

AnalysisWorkstation.c DSP Mathematics and Implementation Reference

August 11, 2026

Abstract

This document explains the digital signal processing calculations implemented by the analysis workstation. For every major calculation, it shows the relevant C code, expresses the same operation mathematically, defines the variables, and explains why the calculation is performed. The emphasis is the active analysis path: spectrogram construction, magnitude/phase/instantaneous-frequency graphs, PSD averaging, frequency-domain crop filtering, constellation sample preparation, carrier-frequency correction, PSK/QAM/ASK timing and alignment, FSK/MSK discriminator recovery, and OFDM cyclic-prefix, carrier, pilot, and constellation processing.

Contents

1	Scope and Signal Model	5
1.1	Important constants	5
2	Spectrogram Coordinate Geometry	6
2.1	Vertical spectrogram fraction to RF frequency	6
2.2	Frequency-selector center and bandwidth	6
2.3	Mouse time position to sample number and seconds	7
2.4	Vertical selector to FFT-bin range	7
3	Spectrogram FFT and Time-Domain Analysis Graphs	7
3.1	Hann window construction	7
3.2	Mapping display columns to IQ blocks	8
3.3	Complex16 normalization, magnitude, and window application	8
3.4	Forward DFT, FFT shift, magnitude, and display dB	9
3.5	Center-bin stripe interpolation	9
4	Selected-Band Magnitude, Phase, and Instantaneous Frequency	10
4.1	Selected-band center, width, and low-pass coefficient	10
4.2	Complex frequency translation to baseband	11
4.3	Phase estimate and instantaneous frequency	11
4.4	Phase-validity magnitude gate	12
4.5	Phase unwrapping	12
4.6	Least-squares carrier-ramp removal	13
4.7	Instantaneous-frequency centering	13

5	PSD / Average Spectrum Construction	14
5.1	Mapping graph pixels to FFT bins	14
5.2	Display normalization	15
5.3	Spectrogram dynamic range and grayscale mapping	15
6	Frequency-Domain Crop Filtering	16
6.1	Block FFT and rectangular frequency selection	16
6.2	Inverse FFT, optional noise muting, and complex16 conversion	17
6.3	Interpretation of the crop filter	17
7	Constellation Input Preparation	17
7.1	Selecting a high-power region	17
7.2	Initial DC removal and selected-band geometry	18
7.3	Frequency translation before constellation filtering	19
7.4	Power-of-two zero padding and FFT-domain low-pass	19
7.5	Second DC removal	20
8	Carrier and Phase-Recovery Primitives	21
8.1	Phase wrapping: <code>ANALYSIS_wrap_phase()</code>	21
8.2	Complex frequency correction	21
8.3	Integer complex power	22
8.4	Mth-power sample-to-sample frequency estimate	23
8.5	Mth-order coherence score	24
8.6	Direct weighted frequency estimate	24
9	FFT-Based Carrier-Frequency Estimation	25
9.1	PSK Mth-power FFT estimator	25
9.2	QAM fourth-moment FFT estimator	26
9.3	Spectral-centroid estimator for ASK/OOK	28
10	Constellation Family Scoring	29
10.1	One-dimensional K-means distortion	29
10.2	PSK fit score	30
10.3	QAM fit score	31
10.4	ASK/OOK fit score and principal axis	32
11	Symbol Sampling and Timing Recovery	33
11.1	Symbol-spaced candidate averaging	33
11.2	Transition-energy timing metric	34
11.3	Active refined timing search	35
12	Residual Carrier Correction and Linear Constellation Alignment	38
12.1	PSK residual frequency from symbol-spaced points	38
12.2	QAM residual frequency	38
12.3	Active PSK/QAM/ASK processing chain	39
12.4	Robust constellation display normalization	42

13 FSK / MSK Discriminator Recovery	42
13.1 Magnitude gate and instantaneous-frequency discriminator	42
13.2 Smoothing	43
13.3 Transition detection	44
13.4 Samples-per-symbol from transition phase coherence	44
13.5 Per-symbol discriminator point	45
13.6 Two-state versus four-state selection	46
13.7 FSK plot normalization	47
14 Robust FSK K-Means	49
14.1 Percentile-bounded initialization	49
14.2 Centroid updates and convergence	49
14.3 Mean squared fitting error	50
15 OFDM Cyclic-Prefix Detection and Generic Recovery	50
15.1 Normalized cyclic-prefix correlation	50
15.2 Search over generic FFT and CP sizes	51
15.3 OFDM symbol FFT and active-carrier threshold	52
15.4 Per-carrier phase order, rotation, and RMS scaling	53
16 Known 128-Point QPSK OFDM Recovery	54
16.1 Sliding CP metric	54
16.2 Fixed waveform parameters	54
16.3 Periodic timing-run score	55
16.4 Fractional carrier offset from CP phase	56
16.5 Integer subcarrier displacement from occupied/guard contrast	56
16.6 Pilot common-phase and linear phase-slope correction	57
16.7 Per-data-carrier QPSK orientation and RMS normalization	58
17 Additional Defined DSP Helpers	59
17.1 Original symbol-timing search	60
17.2 FFT-based transition-energy symbol-rate estimator	61
17.3 Transition-histogram center-offset helper	63
18 End-to-End Active DSP Paths	64
18.1 Spectrogram and graph path	64
18.2 Linear constellation path	65
18.3 FSK/MSK path	65
18.4 OFDM path	65
19 Worked Numerical Examples	65
19.1 2048-point FFT at 2 MS/s	65
19.2 Instantaneous frequency from phase increment	66
19.3 OFDM CP carrier estimate	66
20 Engineering Notes and Interpretation	66
21 Compact Formula Reference	67
22 Core DSP Source Locations	68

23 Core DSP Calculation Listings	68
23.1 Spectrogram FFT calculation	68
23.2 Filtered phase and instantaneous-frequency calculation	69
23.3 Phase unwrap and detrend calculation	70
23.4 Frequency-domain crop calculation	71
23.5 Constellation sample preparation calculation	72
23.6 PSK/QAM/ASK active family dispatch	73
23.7 FSK discriminator and timing calculations	74
23.8 OFDM cyclic-prefix calculation	75
23.9 Known OFDM pilot correction calculation	76

1 Scope and Signal Model

The analysis workstation operates on signed complex16 recordings whose samples are interpreted as

$$x[n] = I[n] + jQ[n]. \quad (1)$$

The stored integer components are converted to normalized floating point by

$$I_f[n] = \frac{I_{16}[n]}{32768}, \quad Q_f[n] = \frac{Q_{16}[n]}{32768}. \quad (2)$$

Three major DSP paths are implemented:

analysis display: IQ → Hann window → FFT → spectrogram/PSD,
 filtered graphs: IQ → frequency shift → low-pass → $\phi[n]$, $f_{\text{inst}}[n]$,
 constellation: IQ → DC removal/filtering → carrier recovery → timing → symbols.

1.1 Important constants

The source defines the principal transform and constellation limits as follows:

```

50 #ifndef M_PI
51 #define M_PI 3.14159265358979323846
52 #endif
53 #define MARGIN 20
54 #define ANALYSIS_MAX_FILES 512
55 #define ANALYSIS_FFT_SIZE 2048
56 #define ANALYSIS_MAX_CONST_POINTS 4096
57 #define ANALYSIS_WORKSPACE_COUNT 5
58 #define ANALYSIS_SIGNAL_FIELD_COUNT 7
59 #define ANALYSIS_SIGNAL_TEXT_MAX 512
60 #define ANALYSIS_SIGNAL_FIELD_NONE -1
61 #define ANALYSIS_SIGNAL_DECIMATION_FIELD 5
62 #define ANALYSIS_SIGNAL_FILENAME_FIELD 6
63 #define ANALYSIS_FILE_SEARCH_TEXT_MAX 256
64 #define ANALYSIS_FILE_SEARCH_ROW_H 34
65 #define ANALYSIS_MULTITHREAD_COUNT 10
66 #define ANALYSIS_TRANSMIT_PASSWORD_MAX 127
67 #define ANALYSIS_TRANSMIT_FIELD_COUNT 5
68 #define ANALYSIS_TRANSMIT_TEXT_MAX 32
69 #define ANALYSIS_TRANSMIT_FIELD_FREQUENCY 0
70 #define ANALYSIS_TRANSMIT_FIELD_SAMPLE_RATE 1
71 #define ANALYSIS_TRANSMIT_FIELD_BANDWIDTH 2
72 #define ANALYSIS_TRANSMIT_FIELD_GAIN 3
73 #define ANALYSIS_TRANSMIT_FIELD_REPEAT 4
74 #define ANALYSIS_NOISE_GRAPH_NONE 0
75 #define ANALYSIS_NOISE_GRAPH_MAG 1
76 #define ANALYSIS_NOISE_GRAPH_INST 2
77 #define ANALYSIS_CONSTELLATION_MODE_OFF 0
78 #define ANALYSIS_CONSTELLATION_MODE_PSK 1
79 #define ANALYSIS_CONSTELLATION_MODE_QAM 2
80 #define ANALYSIS_CONSTELLATION_MODE_ASK_OOK 3
81 #define ANALYSIS_CONSTELLATION_MODE_FSK_MSK 4
82 #define ANALYSIS_CONSTELLATION_MODE_OFDM 5
83 #define ANALYSIS_CONSTELLATION_MODE_COUNT 6
84 #define ANALYSIS_CONSTELLATION_MAX_INPUT 131072U
85 #define ANALYSIS_CONSTELLATION_PSK_BPSK 2
86 #define ANALYSIS_CONSTELLATION_PSK_QPSK 4
87 #define ANALYSIS_CONSTELLATION_PSK_8PSK 8
88 #define ANALYSIS_CONSTELLATION_PSK_OPTION_COUNT 3

```

The principal analysis FFT length is

$$N_{\text{FFT}} = 2048, \quad (3)$$

and the maximum constellation input is

$$N_{\text{const,max}} = 131072. \quad (4)$$

The PSK modes explicitly supported by the constellation path are $M = 2, 4$, and 8 for BPSK, QPSK, and 8PSK.

2 Spectrogram Coordinate Geometry

2.1 Vertical spectrogram fraction to RF frequency

Relevant code (AnalysisWorkstation.c lines 2257–2267):

```
2257 static double ANALYSIS_frequency_from_spec_frac(double frac) {
2258     /*
2259         Purpose: Converts a greyscale spectrogram vertical fraction into RF frequency
2260         Returns: Frequency in Hz
2261     */
2262     frac = ANALYSIS_limit_double(frac, 0.0, 1.0);
2263     double offset_hz = (0.5 - frac) * Global_Analysis_Sample_Rate;
2264     return Global_Analysis_Center_Hz + offset_hz;
2265 }
```

The vertical coordinate is normalized so the top of the spectrogram is fraction 0 and the bottom is fraction 1. The frequency offset is

$$\Delta f = \left(\frac{1}{2} - y \right) F_s, \quad (5)$$

where $y \in [0, 1]$ is the vertical fraction and F_s is the recording sample rate. Therefore,

$$f_{\text{RF}} = f_c + \left(\frac{1}{2} - y \right) F_s. \quad (6)$$

The sign is reversed because screen y increases downward while displayed frequency increases upward.

2.2 Frequency-selector center and bandwidth

Relevant code (AnalysisWorkstation.c lines 2216–2222):

```
2216 double top = Global_Analysis_Filter_Y0;
2217 double bottom = Global_Analysis_Filter_Y1;
2218 double center_y = (top + bottom) * 0.5;
2219 double bw_frac = fabs(bottom - top);
2220 double offset_hz = (0.5 - center_y) * Global_Analysis_Sample_Rate;
2221 double center_hz = Global_Analysis_Center_Hz + offset_hz;
2222 double bw_hz = bw_frac * Global_Analysis_Sample_Rate;
```

For selector endpoints y_0 and y_1 ,

$$y_c = \frac{y_0 + y_1}{2}, \quad (7)$$

$$B = |y_1 - y_0| F_s, \quad (8)$$

$$\Delta f_c = \left(\frac{1}{2} - y_c \right) F_s, \quad (9)$$

$$f_{\text{sel}} = f_c + \Delta f_c. \quad (10)$$

Thus the full sampled bandwidth is treated as the vertical span of the analysis spectrogram.

2.3 Mouse time position to sample number and seconds

Relevant code (AnalysisWorkstation.c lines 2313–2327):

```

2313     double frac = (double)(mouse_x - spec_rect.x) / (double)spec_rect.w;
2315     frac = ANALYSIS_limit_double(frac, 0.0, 1.0);
2317     size_t marker_sample = Global_Analysis_View_Start + (size_t)(frac * (double)Global_Analysis_View_Len);
2319     if (marker_sample >= Global_Analysis_IQ_Count) {
2321         marker_sample = Global_Analysis_IQ_Count - 1;
2323     }
2325     Global_Analysis_Marker_Active = 1;
2326     Global_Analysis_Marker_Sample = marker_sample;
2327     Global_Analysis_Marker_Time = (double)marker_sample / Global_Analysis_Sample_Rate;

```

For horizontal fraction $x \in [0, 1]$ within the current view,

$$n_{\text{marker}} = n_{\text{view}} + \lfloor xL_{\text{view}} \rfloor, \quad (11)$$

where n_{view} is the first sample in the visible region and L_{view} is its sample length. Time is then

$$t_{\text{marker}} = \frac{n_{\text{marker}}}{F_s}. \quad (12)$$

The same geometry is used when a selected time interval is converted into crop sample indices:

```

8301     x0 = ANALYSIS_limit_double(x0, 0.0, 1.0);
8302     x1 = ANALYSIS_limit_double(x1, 0.0, 1.0);
8304 }
8306 *start_sample = Global_Analysis_View_Start + (size_t)(x0 * (double)Global_Analysis_View_Len);
8307 *end_sample = Global_Analysis_View_Start + (size_t)(x1 * (double)Global_Analysis_View_Len);

```

so

$$n_0 = n_{\text{view}} + x_0 L_{\text{view}}, \quad n_1 = n_{\text{view}} + x_1 L_{\text{view}}. \quad (13)$$

2.4 Vertical selector to FFT-bin range

Relevant code (AnalysisWorkstation.c lines 8359–8374):

```

8359     double y0 = Global_Analysis_Filter_Y0;
8360     double y1 = Global_Analysis_Filter_Y1;
8362     if (y1 < y0) {
8364         double tmp = y0;
8365         y0 = y1;
8366         y1 = tmp;
8368     }
8370     int bin_a = (int)((1.0 - y0) * (double)(ANALYSIS_FFT_SIZE - 1));
8371     int bin_b = (int)((1.0 - y1) * (double)(ANALYSIS_FFT_SIZE - 1));
8373     *filter_bin_low = bin_a < bin_b ? bin_a : bin_b;
8374     *filter_bin_high = bin_a > bin_b ? bin_a : bin_b;

```

The FFT display is vertically inverted. A display fraction y maps to shifted FFT bin

$$k(y) = \lfloor (1 - y)(N_{\text{FFT}} - 1) \rfloor. \quad (14)$$

The lower and upper selected bins are then the minimum and maximum of the two endpoint mappings.

3 Spectrogram FFT and Time-Domain Analysis Graphs

3.1 Hann window construction

Relevant code (AnalysisWorkstation.c lines 14944–14948):

```

14944     double window[ANALYSIS_FFT_SIZE];
14946     for (int i = 0; i < ANALYSIS_FFT_SIZE; i++) {
14947         window[i] = 0.5 - 0.5 * cos((2.0 * M_PI * (double)i) / (double)(ANALYSIS_FFT_SIZE - 1));
14948     }

```

The workstation uses a Hann window

$$w[n] = \frac{1}{2} - \frac{1}{2} \cos\left(\frac{2\pi n}{N-1}\right), \quad 0 \leq n < N. \quad (15)$$

The window reduces spectral leakage from the finite 2048-sample analysis block by tapering both block edges toward zero.

3.2 Mapping display columns to IQ blocks

Relevant code (AnalysisWorkstation.c lines 15117–15129):

```

15117     for (int x = 0; x < render_w; x++) {
15118         double frac = (render_w > 1) ? (double)x / (double)(render_w - 1) : 0.0;
15119         size_t start = Global_Analysis_View_Start + (size_t)(frac * (double)Global_Analysis_View_Len);
15121         if (start + ANALYSIS_FFT_SIZE >= Global_Analysis_IQ_Count) {
15123             start =
15124                 Global_Analysis_IQ_Count > ANALYSIS_FFT_SIZE ? Global_Analysis_IQ_Count -
ANALYSIS_FFT_SIZE : 0;
15126         }
15128         thread_starts[x] = start;
15129     }

```

Each rendered horizontal column selects one 2048-sample IQ block. For column x of width W ,

$$u_x = \frac{x}{W-1}, \quad n_x = n_{\text{view}} + \lfloor u_x L_{\text{view}} \rfloor. \quad (16)$$

This is not a fixed-hop STFT. Instead, the visible time span is resampled into exactly the number of display columns, with one FFT block centered by its starting position at each column.

3.3 Complex16 normalization, magnitude, and window application

Relevant code (AnalysisWorkstation.c lines 15185–15197):

```

15185     double sum_mag = 0.0;
15187     for (int k = 0; k < ANALYSIS_FFT_SIZE; k++) {
15188         double I = (double)current_block[k * 2] / 32768.0;
15189         double Q = (double)current_block[k * 2 + 1] / 32768.0;
15191         sum_mag += sqrt(I * I + Q * Q);
15193         in[k][0] = I * window[k];
15194         in[k][1] = Q * window[k];
15195     }
15197     double avg_mag = sum_mag / (double)ANALYSIS_FFT_SIZE;

```

For each sample,

$$x[k] = \frac{I_{16}[k]}{32768} + j \frac{Q_{16}[k]}{32768}. \quad (17)$$

The time-domain magnitude graph initially uses the average envelope magnitude over the block:

$$\bar{r}_x = \frac{1}{N} \sum_{k=0}^{N-1} |x[k]| = \frac{1}{N} \sum_{k=0}^{N-1} \sqrt{I[k]^2 + Q[k]^2}. \quad (18)$$

The FFT input is

$$x_w[k] = x[k]w[k]. \quad (19)$$

3.4 Forward DFT, FFT shift, magnitude, and display dB

Relevant code (AnalysisWorkstation.c lines 15321–15340):

```

15321     fftw_execute(plan);
15322     for (int y = 0; y < ANALYSIS_FFT_SIZE; y++) {
15323         int shifted = (y + ANALYSIS_FFT_SIZE / 2) % ANALYSIS_FFT_SIZE;
15324         double I = out[shifted][0];
15325         double Q = out[shifted][1];
15326         double mag = sqrt(I * I + Q * Q) / (double)ANALYSIS_FFT_SIZE;
15327         double val = 20.0 * log10(mag + 1e-12) + 100.0;
15328         db_img[(size_t)x * ANALYSIS_FFT_SIZE + y] = val;
15329         if (filter_active && y >= filter_bin_low && y <= filter_bin_high) {
15330             double weight = mag;
15331             filter_mag[x] += weight;
15332             filter_re[x] += I;
15333             filter_im[x] += Q;
15334         }
15335     }
15336 }
15337
15338 }
15339
15340 }
```

FFTW computes the forward transform

$$X[m] = \sum_{n=0}^{N-1} x_w[n] e^{-j2\pi mn/N}. \quad (20)$$

The source performs the usual half-length FFT shift using

$$m_{\text{src}} = (m + N/2) \bmod N, \quad (21)$$

so negative frequencies appear on the left and positive frequencies on the right of the shifted spectral array.

The normalized complex magnitude is

$$A[m] = \frac{|X[m]|}{N} = \frac{\sqrt{\Re\{X[m]\}^2 + \Im\{X[m]\}^2}}{N}. \quad (22)$$

The display value is

$$D[m] = 20 \log_{10}(A[m] + 10^{-12}) + 100. \quad (23)$$

The 10^{-12} floor prevents $\log(0)$, and the $+100$ is a display offset; it does not change relative spectral differences.

3.5 Center-bin stripe interpolation

The greyscale spectrogram deliberately hides a guard region around the shifted DC bin and replaces it with a linear interpolation between bins on either side.

Relevant code (AnalysisWorkstation.c lines 9900–9918):

```

9900     double low = db_img[(size_t)x * ANALYSIS_FFT_SIZE + low_bin];
9901     double high = db_img[(size_t)x * ANALYSIS_FFT_SIZE + high_bin];
9902     double denom = (double)(high_bin - low_bin);
9903     double t = denom > 0.0 ? (double)(bin - low_bin) / denom : 0.5;
9904     if (t < 0.0) {
9905         t = 0.0;
9906     }
9907     if (t > 1.0) {
9908         t = 1.0;
9909     }
9910     return low + ((high - low) * t);
9911 }
```

For reference values D_L and D_H at bins k_L and k_H , the interpolated center-bin value is

$$t = \frac{k - k_L}{k_H - k_L}, \quad D_{\text{interp}}[k] = D_L + (D_H - D_L)t. \quad (24)$$

This is a display-only operation. It does not alter the underlying IQ data or the FFT used by the analysis calculations.

4 Selected-Band Magnitude, Phase, and Instantaneous Frequency

When the frequency selector is active, the workstation computes time-domain phase and instantaneous frequency from a frequency-shifted, one-pole low-pass version of the IQ samples.

4.1 Selected-band center, width, and low-pass coefficient

Relevant code (AnalysisWorkstation.c lines 15201–15221):

```

15201     double bin_center = ((double)filter_bin_low + (double)filter_bin_high) * 0.5;
15202     double center_offset_hz = (bin_center - ((double)ANALYSIS_FFT_SIZE * 0.5)) *
15203         (Global_Analysis_Sample_Rate / (double)ANALYSIS_FFT_SIZE);
15204     double bin_width_hz = Global_Analysis_Sample_Rate / (double)ANALYSIS_FFT_SIZE;
15205     double filter_width_hz = (double)(filter_bin_high - filter_bin_low + 1) * bin_width_hz;
15206     double cutoff_hz = filter_width_hz * 0.5;
15208     if (cutoff_hz < bin_width_hz) {
15210         cutoff_hz = bin_width_hz;
15212     }
15214     if (cutoff_hz > Global_Analysis_Sample_Rate * 0.45) {
15216         cutoff_hz = Global_Analysis_Sample_Rate * 0.45;
15218     }
15220     double alpha = (2.0 * M_PI * cutoff_hz) / (Global_Analysis_Sample_Rate + (2.0 * M_PI *
cutoff_hz));
15221     double omega = 2.0 * M_PI * center_offset_hz / Global_Analysis_Sample_Rate;

```

The FFT-bin width is

$$\Delta f_{\text{bin}} = \frac{F_s}{N_{\text{FFT}}}. \quad (25)$$

For selected bins k_L through k_H ,

$$k_c = \frac{k_L + k_H}{2}, \quad (26)$$

$$f_{\text{off}} = \left(k_c - \frac{N_{\text{FFT}}}{2} \right) \Delta f_{\text{bin}}, \quad (27)$$

$$B_{\text{filter}} = (k_H - k_L + 1) \Delta f_{\text{bin}}, \quad (28)$$

$$f_{\text{cut}} = \frac{B_{\text{filter}}}{2}. \quad (29)$$

The cutoff is clamped to at least one FFT-bin width and at most $0.45F_s$.

The one-pole low-pass coefficient is

$$\alpha = \frac{2\pi f_{\text{cut}}}{F_s + 2\pi f_{\text{cut}}}. \quad (30)$$

This corresponds to a first-order discrete low-pass approximation with update

$$y[n] = y[n-1] + \alpha(x[n] - y[n-1]). \quad (31)$$

4.2 Complex frequency translation to baseband

Relevant code (AnalysisWorkstation.c lines 15252–15273):

```

15252     for (int k = 0; k < inst_sample_count; k++) {
15253         double I = (double)current_block[k * 2] / 32768.0;
15254         double Q = (double)current_block[k * 2 + 1] / 32768.0;
15255         double angle = -omega * (double)k;
15256         double c = cos(angle);
15257         double s = sin(angle);
15258         double mix_i = I * c - Q * s;
15259         double mix_q = I * s + Q * c;
15261         if (k == 0) {
15263             lp_i = mix_i;
15264             lp_q = mix_q;
15266         }
15268         else {
15270             lp_i += alpha * (mix_i - lp_i);
15271             lp_q += alpha * (mix_q - lp_q);
15273         }

```

The normalized angular frequency of the selected center is

$$\omega_0 = \frac{2\pi f_{\text{off}}}{F_s}. \quad (32)$$

For local sample index k , the code forms

$$z[k] = x[k]e^{-j\omega_0 k}. \quad (33)$$

Expanded into real and imaginary parts,

$$I_m = I \cos \theta - Q \sin \theta, \quad (34)$$

$$Q_m = I \sin \theta + Q \cos \theta, \quad (35)$$

where $\theta = -\omega_0 k$. The mixer moves the selected frequency center to DC before the one-pole low-pass is applied.

4.3 Phase estimate and instantaneous frequency

Relevant code (AnalysisWorkstation.c lines 15275–15297):

```

15275     phase_sum_i += lp_i;
15276     phase_sum_q += lp_q;
15278     if (k > 0) {
15280         double prod_i = lp_i * prev_i + lp_q * prev_q;
15281         double prod_q = lp_q * prev_i - lp_i * prev_q;
15282         double dphase = ANALYSIS_wrap_phase(atan2(prod_q, prod_i));
15284         inst_freq_sum += dphase * Global_Analysis_Sample_Rate / (2.0 * M_PI);
15285         inst_freq_count++;
15287     }
15289     prev_i = lp_i;
15290     prev_q = lp_q;
15291 }
15293 filter_td_phase[x] = atan2(phase_sum_q, phase_sum_i);
15295 if (inst_freq_count > 0) {
15297     filter_td_inst_freq[x] = inst_freq_sum / (double)inst_freq_count;

```

The per-column phase estimate uses the argument of the vector sum of filtered samples:

$$\phi_x = \arg \left(\sum_k y[k] \right) = \text{atan2} \left(\sum_k Q_y[k], \sum_k I_y[k] \right). \quad (36)$$

Instantaneous phase increment is obtained from

$$p[k] = y[k]y^*[k-1]. \quad (37)$$

Its real and imaginary parts are exactly the two products in the code, and

$$\Delta\phi[k] = \arg p[k]. \quad (38)$$

The corresponding instantaneous frequency is

$$f_{\text{inst}}[k] = \frac{F_s}{2\pi} \Delta\phi[k]. \quad (39)$$

The plotted value for a display column is the mean of these frequency estimates over the local sample subset.

4.4 Phase-validity magnitude gate

Relevant code (AnalysisWorkstation.c lines 15395–15410):

```

15395     double phase_gate = max_mag * 0.15;
15397     if (phase_gate < 1e-9) {
15399         phase_gate = 1e-9;
15401     }
15403     for (int x = 0; x < render_w; x++) {
15404         double avg_mag = Global_Analysis_Mag_Line[x];
15406         if (avg_mag >= phase_gate && (!time_filter_active || (x >= time_col_low && x <= time_col_high))) {
15408             valid_phase[x] = 1;
15410         }

```

Only columns with selected-band magnitude above

$$r_x \geq 0.15 r_{\text{max}} \quad (40)$$

are treated as phase-valid. This avoids attempting to interpret phase in low-amplitude regions where noise dominates the angle.

4.5 Phase unwrapping

Relevant code (AnalysisWorkstation.c lines 15444–15453):

```

15444     double unwrapped = filter_td_phase[seg_start];
15445     double prev_raw = filter_td_phase[seg_start];
15446     filter_td_phase[seg_start] = unwrapped;
15448     for (int x = seg_start + 1; x <= seg_end; x++) {
15449         double raw = filter_td_phase[x];
15450         unwrapped += ANALYSIS_wrap_phase(raw - prev_raw);
15451         filter_td_phase[x] = unwrapped;
15452         prev_raw = raw;
15453     }

```

Within each contiguous valid segment, wrapped phase increments are accumulated:

$$\tilde{\phi}[x] = \tilde{\phi}[x-1] + \text{wrap}_{[-\pi, \pi]}(\phi[x] - \phi[x-1]). \quad (41)$$

This removes the visual 2π discontinuities created by `atan2()`.

4.6 Least-squares carrier-ramp removal

Relevant code (AnalysisWorkstation.c lines 15455–15488):

```

15455     int n = seg_end - seg_start + 1;
15457     if (n >= 2) {
15459         double sum_x = 0.0;
15460         double sum_y = 0.0;
15461         double sum_xx = 0.0;
15462         double sum_xy = 0.0;
15464         for (int x = seg_start; x <= seg_end; x++) {
15465             double dx = (double)(x - seg_start);
15466             double y = filter_td_phase[x];
15468             sum_x += dx;
15469             sum_y += y;
15470             sum_xx += dx * dx;
15471             sum_xy += dx * y;
15472         }
15474         double denom = ((double)n * sum_xx) - (sum_x * sum_x);
15475         double slope = 0.0;
15476         double intercept = sum_y / (double)n;
15478         if (fabs(denom) > 1e-12) {
15480             slope = (((double)n * sum_xy) - (sum_x * sum_y)) / denom;
15481             intercept = (sum_y - slope * sum_x) / (double)n;
15483         }
15485         for (int x = seg_start; x <= seg_end; x++) {
15486             double dx = (double)(x - seg_start);
15487             filter_td_phase[x] -= intercept + slope * dx;
15488         }

```

A residual carrier-frequency error produces an approximately linear phase ramp. The code fits

$$\hat{\phi}(x) = a + bx \quad (42)$$

with ordinary least squares. For n points,

$$b = \frac{n \sum xy - (\sum x)(\sum y)}{n \sum x^2 - (\sum x)^2}, \quad (43)$$

$$a = \frac{\sum y - b \sum x}{n}. \quad (44)$$

The displayed phase becomes

$$\phi_{\text{dev}}(x) = \tilde{\phi}(x) - \hat{\phi}(x), \quad (45)$$

which emphasizes modulation-induced phase deviation instead of steady carrier rotation.

4.7 Instantaneous-frequency centering

Relevant code (AnalysisWorkstation.c lines 15501–15545):

```

15501     double inst_freq_mean = 0.0;
15502     int inst_freq_mean_count = 0;
15504     for (int x = 0; x < render_w; x++) {
15506         if (valid_phase[x]) {
15508             inst_freq_mean += filter_td_inst_freq[x];
15509             inst_freq_mean_count++;
15511         }
15512     }
15514     if (inst_freq_mean_count > 0) {
15516         inst_freq_mean /= (double)inst_freq_mean_count;
15518     }
15520     for (int x = 0; x < render_w; x++) {
15521         double phase = 0.0;
15522         double inst_freq_hz = 0.0;
15524         if (valid_phase[x]) {

```

```

15526         phase = filter_td_phase[x];
15527         inst_freq_hz = filter_td_inst_freq[x] - inst_freq_mean;
15529         if (fabs(phase) > max_phase_abs) {
15531             max_phase_abs = fabs(phase);
15533         }
15535         if (fabs(inst_freq_hz) > max_inst_freq_abs) {
15537             max_inst_freq_abs = fabs(inst_freq_hz);
15539         }
15541     }
15543     Global_Analysis_Phase_Line[x] = (float)phase;
15544     Global_Analysis_InstFreq_Line[x] = (float)inst_freq_hz;
15545 }

```

The mean instantaneous frequency over all valid columns is

$$\bar{f} = \frac{1}{N_v} \sum_{x \in \mathcal{V}} f_x. \quad (46)$$

The graph displays

$$f_{\text{dev}}[x] = f_x - \bar{f}, \quad (47)$$

thereby removing a constant residual frequency offset and showing frequency modulation around the selected center.

5 PSD / Average Spectrum Construction

5.1 Mapping graph pixels to FFT bins

Relevant code (AnalysisWorkstation.c lines 15580–15627):

```

15580     for (int x = 0; x < render_w; x++) {
15581         int bin = render_w > 1 ? (int)((double)x / (double)(render_w - 1)) * (double)(ANALYSIS_FFT_SIZE - 1)
15582             : ANALYSIS_FFT_SIZE / 2;
15584         if (bin < 0) {
15586             bin = 0;
15588         }
15590         if (bin >= ANALYSIS_FFT_SIZE) {
15592             bin = ANALYSIS_FFT_SIZE - 1;
15594         }
15596         if (filter_active && (bin < filter_bin_low || bin > filter_bin_high)) {
15598             Global_Analysis_PSD_Line[x] = 0.0f;
15599             continue;
15601         }
15603         double sum_db = 0.0;
15604         int count_db = 0;
15606         for (int t = 0; t < render_w; t++) {
15608             if (time_filter_active && (t < time_col_low || t > time_col_high)) {
15610                 continue;
15612             }
15614             sum_db += db_img[(size_t)t * ANALYSIS_FFT_SIZE + bin];
15615             count_db++;
15616         }
15618         if (count_db <= 0) {
15620             Global_Analysis_PSD_Line[x] = 0.0f;
15621             continue;
15623         }
15625         double avg_db = sum_db / (double)count_db;
15627         Global_Analysis_PSD_Line[x] = (float)avg_db;

```

For horizontal graph coordinate x ,

$$k_x = \left\lfloor \frac{x}{W-1} (N_{\text{FFT}} - 1) \right\rfloor. \quad (48)$$

For that bin, the workstation averages the already computed logarithmic spectrogram values over selected time columns:

$$\bar{D}[k_x] = \frac{1}{N_t} \sum_{t \in \mathcal{T}} D_t[k_x]. \quad (49)$$

Because the averaging is performed in dB, this is an arithmetic mean of logarithmic magnitude values, not a linear-power Welch PSD estimate.

5.2 Display normalization

Relevant code (AnalysisWorkstation.c lines 15643–15673):

```

15643     double psd_range_db = psd_max_db - psd_min_db;
15645     if (psd_range_db < 1e-9) {
15647         psd_range_db = 1.0;
15649     }
15651     for (int x = 0; x < render_w; x++) {
15653         if (!psd_valid[x]) {
15655             Global_Analysis_PSD_Line[x] = 0.0f;
15656             continue;
15658         }
15660         Global_Analysis_PSD_Line[x] = (float)((Global_Analysis_PSD_Line[x] - psd_min_db) / psd_range_db);
15662         if (Global_Analysis_PSD_Line[x] < 0.0f) {
15664             Global_Analysis_PSD_Line[x] = 0.0f;
15666         }
15668         if (Global_Analysis_PSD_Line[x] > 1.0f) {
15670             Global_Analysis_PSD_Line[x] = 1.0f;
15672         }
15673     }

```

The graph is linearly normalized between its minimum and maximum displayed dB values:

$$P_{\text{disp}}[x] = \frac{\bar{D}[x] - D_{\min}}{D_{\max} - D_{\min}}. \quad (50)$$

The result is clamped to $[0, 1]$ for rendering.

5.3 Spectrogram dynamic range and grayscale mapping

Relevant code (AnalysisWorkstation.c lines 15719–15748):

```

15719     double min_db = max_db - 70.0;
15721     for (int x = 0; x < tex_w; x++) {
15722         int src_x = x;
15724         if (src_x >= render_w) {
15726             src_x = render_w - 1;
15728         }
15730         for (int py = 0; py < tex_h; py++) {
15731             int bin = (int)((1.0 - ((double)py / (double)tex_h)) * (double)(ANALYSIS_FFT_SIZE - 1));
15732             double val = ANALYSIS_spectrogram_display_db(db_img, src_x, bin);
15733             double norm = (val - min_db) / 70.0;
15735             if (filter_active && (bin < filter_bin_low || bin > filter_bin_high)) {
15737                 norm *= 0.30;
15739             }
15741             if (time_filter_active && (src_x < time_col_low || src_x > time_col_high)) {
15743                 norm *= 0.30;
15745             }
15747             pixels[(size_t)py * tex_w + x] = ANALYSIS_gray(norm);
15748         }

```

The greyscale spectrogram uses a 70 dB display window below the strongest displayed value:

$$D_{\min} = D_{\max} - 70 \text{ dB}, \quad g = \frac{D - D_{\min}}{70}. \quad (51)$$

The grayscale helper clamps g to $[0, 1]$ and converts it to an 8-bit intensity

$$c = \lfloor 255g \rfloor. \quad (52)$$

Frequency/time regions outside active selectors are multiplied by 0.30 for display emphasis only.

6 Frequency-Domain Crop Filtering

6.1 Block FFT and rectangular frequency selection

Relevant code (AnalysisWorkstation.c lines 8696–8737):

```

8696     for (int i = 0; i < ANALYSIS_FFT_SIZE; i++) {
8698         if ((size_t)i < got_iq) {
8700             freq_in[i][0] = (double)iq[i * 2] / 32768.0;
8701             freq_in[i][1] = (double)iq[i * 2 + 1] / 32768.0;
8703         }
8705         else {
8707             freq_in[i][0] = 0.0;
8708             freq_in[i][1] = 0.0;
8710         }
8711     }
8713     fftw_execute(forward);
8715     for (int i = 0; i < ANALYSIS_FFT_SIZE; i++) {
8716         freq_in[i][0] = 0.0;
8717         freq_in[i][1] = 0.0;
8718     }
8720     int filter_center_bin = (filter_bin_low + filter_bin_high) / 2;
8721     for (int y = filter_bin_low; y <= filter_bin_high; y++) {
8722         int src_shifted = (y + ANALYSIS_FFT_SIZE / 2) % ANALYSIS_FFT_SIZE;
8723         int centered_y = y - filter_center_bin + (ANALYSIS_FFT_SIZE / 2);
8725         if (centered_y < 0 || centered_y >= ANALYSIS_FFT_SIZE) {
8727             continue;
8729         }
8731         int dst_shifted = (centered_y + ANALYSIS_FFT_SIZE / 2) % ANALYSIS_FFT_SIZE;
8733         freq_in[dst_shifted][0] = freq_out[src_shifted][0];
8734         freq_in[dst_shifted][1] = freq_out[src_shifted][1];
8735     }
8737     fftw_execute(inverse);

```

Each crop block is normalized to complex floating point and zero-padded to $N = 2048$ when the final block is short. The forward FFT is

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}. \quad (53)$$

All destination spectrum bins are first set to zero. Only selected shifted display bins are copied. If

$$k_c = \left\lfloor \frac{k_L + k_H}{2} \right\rfloor, \quad (54)$$

then the selected display bin y is recentered to

$$y' = y - k_c + \frac{N}{2}. \quad (55)$$

The extra half-length modulo operations convert between shifted-display indexing and FFTW's native unshifted order. The operation therefore performs two things simultaneously: a rectangular frequency mask and translation of the selected band's center to DC.

6.2 Inverse FFT, optional noise muting, and complex16 conversion

Relevant code (AnalysisWorkstation.c lines 8737–8775):

```

8737     fftw_execute(inverse);
8739     for (size_t i = 0; i < got_iq; i++) {
8740         double re = time_out[i][0] / (double)ANALYSIS_FFT_SIZE;
8741         double im = time_out[i][1] / (double)ANALYSIS_FFT_SIZE;
8743         if (ANALYSIS_noise_sample_is_muted(absolute_sample + i)) {
8745             re = 0.0;
8746             im = 0.0;
8748         }
8750         if (re > 0.999969) {
8752             re = 0.999969;
8754         }
8756         if (re < -1.0) {
8758             re = -1.0;
8760         }
8762         if (im > 0.999969) {
8764             im = 0.999969;
8766         }
8768         if (im < -1.0) {
8770             im = -1.0;
8772         }
8774         iq[i * 2] = (int16_t)rint(re * 32767.0);
8775         iq[i * 2 + 1] = (int16_t)rint(im * 32767.0);

```

FFTW's inverse transform is unnormalized, so the time-domain result is divided by N :

$$x_f[n] = \frac{1}{N} \text{IFFT}\{X_f[k]\}. \quad (56)$$

If the noise mask marks the sample, both complex components are set to zero. Otherwise, the output is clipped to the representable normalized complex16 range and converted by approximately

$$I_{16} = \text{round}(32767 I_f), \quad Q_{16} = \text{round}(32767 Q_f). \quad (57)$$

6.3 Interpretation of the crop filter

The crop filter is an FFT-domain rectangular mask rather than a tapered FIR. Ideally,

$$H[k] = \begin{cases} 1, & k \in [k_L, k_H], \\ 0, & \text{otherwise.} \end{cases} \quad (58)$$

The corresponding time-domain impulse response is sinc-like. Because blocks are processed independently without overlap-add, discontinuities can occur at block boundaries for difficult signals. This description reflects the implemented mathematics; it is distinct from the recording FIR used elsewhere in RetroSpectrum.

7 Constellation Input Preparation

7.1 Selecting a high-power region

When the visible selection is larger than the maximum constellation input, the code probes 17 positions and chooses a region around the strongest average complex power.

Relevant code (AnalysisWorkstation.c lines 10389–10435):

```

10389     for (int p = 0; p < probes; p++) {
10390         size_t center =
10391             selection_start + (size_t)((((double)p / (double)(probes - 1)) * (double)(selection_length
- 1U));

```

```

10392     size_t probe_start = center > probe_count / 2U ? center - probe_count / 2U : selection_start;
10394     if (probe_start < selection_start) {
10396         probe_start = selection_start;
10398     }
10400     if (probe_start + probe_count > selection_start + selection_length) {
10402         probe_start = selection_start + selection_length - probe_count;
10404     }
10406     clearerr(fp);
10408     if (fseeko(fp, (off_t)(probe_start * 2U * sizeof(int16_t)), SEEK_SET) != 0) {
10410         continue;
10412     }
10414     size_t values = fread(probe, sizeof(int16_t), probe_count * 2U, fp);
10415     size_t samples = values / 2U;
10416     double power = 0.0;
10418     for (size_t n = 0; n < samples; n++) {
10419         double sample_i = (double)probe[n * 2U] / 32768.0;
10420         double sample_q = (double)probe[n * 2U + 1U] / 32768.0;
10421         power += sample_i * sample_i + sample_q * sample_q;
10422     }
10424     if (samples > 0) {
10426         power /= (double)samples;
10428     }
10430     if (power > best_power) {
10432         best_power = power;
10433         best_center = center;
10435     }

```

For a probe of N_p samples, the mean power is

$$P = \frac{1}{N_p} \sum_{n=0}^{N_p-1} (I[n]^2 + Q[n]^2). \quad (59)$$

The highest-power probe determines the center of the bounded constellation input window. This biases constellation recovery toward an active signal burst rather than a quiet part of a long recording.

7.2 Initial DC removal and selected-band geometry

Relevant code (AnalysisWorkstation.c lines 10494–10513):

```

10494     double dc_i = 0.0;
10495     double dc_q = 0.0;
10497     for (size_t n = 0; n < sample_count; n++) {
10498         dc_i += (double)row[n * 2U] / 32768.0;
10499         dc_q += (double)row[n * 2U + 1U] / 32768.0;
10500     }
10502     dc_i /= (double)sample_count;
10503     dc_q /= (double)sample_count;
10505     double center_offset_hz = 0.0;
10506     double bandwidth_hz = Global_Analysis_Sample_Rate * 0.90;
10508     if (filter_active) {
10510         double bin_center = ((double)filter_bin_low + (double)filter_bin_high) * 0.5;
10511         double bin_width_hz = Global_Analysis_Sample_Rate / (double)ANALYSIS_FFT_SIZE;
10512         center_offset_hz = (bin_center - ((double)ANALYSIS_FFT_SIZE * 0.5)) * bin_width_hz;
10513         bandwidth_hz = (double)(filter_bin_high - filter_bin_low + 1) * bin_width_hz;

```

The complex DC estimate is the sample mean

$$\mu = \mu_I + j\mu_Q = \frac{1}{N} \sum_{n=0}^{N-1} x[n]. \quad (60)$$

The samples are later centered by $x[n] - \mu$.

If a frequency filter is active, the selected-bin center and bandwidth are

$$k_c = \frac{k_L + k_H}{2}, \quad (61)$$

$$\Delta f_{\text{bin}} = \frac{F_s}{N_{\text{FFT}}}, \quad (62)$$

$$f_{\text{off}} = \left(k_c - \frac{N_{\text{FFT}}}{2} \right) \Delta f_{\text{bin}}, \quad (63)$$

$$B = (k_H - k_L + 1) \Delta f_{\text{bin}}. \quad (64)$$

7.3 Frequency translation before constellation filtering

Relevant code (AnalysisWorkstation.c lines 10523–10540):

```

10523 double mixer_step = -2.0 * M_PI * center_offset_hz / Global_Analysis_Sample_Rate;
10524 double step_i = cos(mixer_step);
10525 double step_q = sin(mixer_step);
10526 double oscillator_i = 1.0;
10527 double oscillator_q = 0.0;
10529 for (size_t n = 0; n < sample_count; n++) {
10530     double input_i = ((double)raw[n * 2U] / 32768.0) - dc_i;
10531     double input_q = ((double)raw[n * 2U + 1U] / 32768.0) - dc_q;
10533     i_data[n] = input_i * oscillator_i - input_q * oscillator_q;
10534     q_data[n] = input_i * oscillator_q + input_q * oscillator_i;
10536     double next_i = oscillator_i * step_i - oscillator_q * step_q;
10537     double next_q = oscillator_q * step_i + oscillator_i * step_q;
10538     oscillator_i = next_i;
10539     oscillator_q = next_q;
10540 }

```

The mixer step is

$$\omega_m = -\frac{2\pi f_{\text{off}}}{F_s}. \quad (65)$$

A recursive unit oscillator implements $e^{j\omega_m n}$, and the centered input becomes

$$x_m[n] = (x[n] - \mu) e^{-j2\pi f_{\text{off}} n / F_s}. \quad (66)$$

This moves the selected signal center to zero frequency.

7.4 Power-of-two zero padding and FFT-domain low-pass

Relevant code (AnalysisWorkstation.c lines 10544–10640):

```

10544 size_t fft_size = ANALYSIS_constellation_next_power_of_two(sample_count);
10546 if (fft_size < sample_count || fft_size > ANALYSIS_CONSTELLATION_MAX_INPUT) {
10548     free(i_data);
10549     free(q_data);
10550     return 0;
10552 }
10554 fftw_complex *time_data = fftw_malloc(sizeof(fftw_complex) * fft_size);
10555 fftw_complex *frequency_data = fftw_malloc(sizeof(fftw_complex) * fft_size);
10557 if (!time_data || !frequency_data) {
10559     if (time_data) {
10561         fftw_free(time_data);
10563     }
10565     if (frequency_data) {
10567         fftw_free(frequency_data);
10569     }
10570     free(i_data);
10571     free(q_data);
10572     return 0;
10574 }

```

```

10576     for (size_t n = 0; n < fft_size; n++) {
10577         time_data[n][0] = n < sample_count ? i_data[n] : 0.0;
10578         time_data[n][1] = n < sample_count ? q_data[n] : 0.0;
10579     }
10581     fftw_plan forward = fftw_plan_dft_1d((int)fft_size, time_data, frequency_data, FFTW_FORWARD,
10582     FFTW_ESTIMATE);
10582     fftw_plan inverse = fftw_plan_dft_1d((int)fft_size, frequency_data, time_data, FFTW_BACKWARD,
10583     FFTW_ESTIMATE);
10584     if (!forward || !inverse) {
10586         if (forward) {
10588             fftw_destroy_plan(forward);
10590         }
10592         if (inverse) {
10594             fftw_destroy_plan(inverse);
10596         }
10597         fftw_free(time_data);
10598         fftw_free(frequency_data);
10599         free(i_data);
10600         free(q_data);
10601         return 0;
10603     }
10605     fftw_execute(forward);
10607     double cutoff_hz = bandwidth_hz * 0.55;
10609     if (cutoff_hz > Global_Analysis_Sample_Rate * 0.47) {
10611         cutoff_hz = Global_Analysis_Sample_Rate * 0.47;
10613     }
10615     if (cutoff_hz < Global_Analysis_Sample_Rate / (double)fft_size) {
10617         cutoff_hz = Global_Analysis_Sample_Rate / (double)fft_size;
10619     }
10621     for (size_t k = 0; k < fft_size; k++) {
10622         double frequency =
10623             k <= fft_size / 2U
10624             ? ((double)k * Global_Analysis_Sample_Rate / (double)fft_size)
10625             : ((double)((long long)k - (long long)fft_size) * Global_Analysis_Sample_Rate /
10626             (double)fft_size);
10627         if (fabs(frequency) > cutoff_hz) {
10629             frequency_data[k][0] = 0.0;
10630             frequency_data[k][1] = 0.0;
10632         }
10633     }
10635     fftw_execute(inverse);
10637     for (size_t n = 0; n < sample_count; n++) {
10638         i_data[n] = time_data[n][0] / (double)fft_size;
10639         q_data[n] = time_data[n][1] / (double)fft_size;
10640     }

```

The transform size is the next power of two not smaller than the sample count. Samples beyond the actual count are zero-padded.

The cutoff is chosen as

$$f_{\text{cut}} = 0.55B, \quad (67)$$

then constrained to

$$\frac{F_s}{N_{\text{FFT,const}}} \leq f_{\text{cut}} \leq 0.47F_s. \quad (68)$$

Each native FFT bin is assigned signed frequency

$$f_k = \begin{cases} kF_s/N, & k \leq N/2, \\ (k - N)F_s/N, & k > N/2. \end{cases} \quad (69)$$

Bins satisfying $|f_k| > f_{\text{cut}}$ are zeroed. After the inverse FFT, division by N restores FFTW's scaling.

7.5 Second DC removal

Relevant code (AnalysisWorkstation.c lines 10647–10661):

```

10647     dc_i = 0.0;
10648     dc_q = 0.0;
10650     for (size_t n = 0; n < sample_count; n++) {
10651         dc_i += i_data[n];
10652         dc_q += q_data[n];
10653     }
10655     dc_i /= (double)sample_count;
10656     dc_q /= (double)sample_count;
10658     for (size_t n = 0; n < sample_count; n++) {
10659         i_data[n] -= dc_i;
10660         q_data[n] -= dc_q;
10661     }

```

Filtering can leave a small residual mean. The code recomputes

$$\mu_f = \frac{1}{N} \sum_n x_f[n] \quad (70)$$

and outputs

$$x_c[n] = x_f[n] - \mu_f. \quad (71)$$

This is the prepared IQ sequence used by all constellation-family recovery routines.

8 Carrier and Phase-Recovery Primitives

8.1 Phase wrapping: ANALYSIS_wrap_phase()

Relevant code (AnalysisWorkstation.c lines 9800–9813):

```

9800 static double ANALYSIS_wrap_phase(double value) {
9801     /*
9802      * Purpose: Wraps phase into the -pi to pi range
9803      * Returns: Wrapped phase
9804      */
9805     while (value > M_PI) {
9806         value -= 2.0 * M_PI;
9807     }
9808     while (value < -M_PI) {
9809         value += 2.0 * M_PI;
9810     }
9811     return value;
9813 }

```

The function implements the modulo- 2π mapping

$$\phi_w = \text{wrap}_{[-\pi, \pi]}(\phi). \quad (72)$$

Repeatedly adding or subtracting 2π preserves the physical angle while constraining its numerical representation.

8.2 Complex frequency correction

Relevant code (AnalysisWorkstation.c lines 10115–10158):

```

10115 static void ANALYSIS_constellation_apply_frequency_correction(double *i_data, double *q_data, size_t count,
10116                                                                double radians_per_sample) {
10117     /*
10118      * Purpose: Rotates IQ samples to compensate for a frequency offset
10119      * Returns: No value
10120      */
10121     if (!i_data || !q_data || count == 0 || fabs(radians_per_sample) < 1e-15) {
10122         return;
10123     }
10126 }

```

```

10128 double step_i = cos(-radians_per_sample);
10129 double step_q = sin(-radians_per_sample);
10130 double oscillator_i = 1.0;
10131 double oscillator_q = 0.0;
10133 for (size_t n = 0; n < count; n++) {
10134     double input_i = i_data[n];
10135     double input_q = q_data[n];
10137     i_data[n] = input_i * oscillator_i - input_q * oscillator_q;
10138     q_data[n] = input_i * oscillator_q + input_q * oscillator_i;
10140     double next_i = oscillator_i * step_i - oscillator_q * step_q;
10141     double next_q = oscillator_q * step_i + oscillator_i * step_q;
10143     oscillator_i = next_i;
10144     oscillator_q = next_q;
10146     if ((n & 4095U) == 4095U) {
10148         double magnitude = hypot(oscillator_i, oscillator_q);
10150         if (magnitude > 1e-12) {
10152             oscillator_i /= magnitude;
10153             oscillator_q /= magnitude;
10155         }
10157     }
10158 }

```

For an estimated angular offset ω in radians/sample, the correction is

$$y[n] = x[n]e^{-j\omega n}. \quad (73)$$

Rather than evaluating trigonometric functions for every sample, the code forms a one-sample step

$$s = e^{-j\omega} \quad (74)$$

and recursively updates the oscillator

$$o[n+1] = o[n]s. \quad (75)$$

Every 4096 samples the oscillator is renormalized to unit magnitude to limit floating-point drift.

8.3 Integer complex power

Relevant code (AnalysisWorkstation.c lines 10161–10189):

```

10161 static void ANALYSIS_constellation_complex_power(double input_i, double input_q, int order, double *output_i,
10162                                                  double *output_q) {
10163     /*
10164      * Purpose: Raises a complex IQ value to an integer power
10165      * Returns: No value
10166      */
10168     double result_i = 1.0;
10169     double result_q = 0.0;
10171     for (int k = 0; k < order; k++) {
10172         double next_i = result_i * input_i - result_q * input_q;
10173         double next_q = result_i * input_q + result_q * input_i;
10174         result_i = next_i;
10175         result_q = next_q;
10176     }
10178     if (output_i) {
10180         *output_i = result_i;
10182     }
10184     if (output_q) {
10186         *output_q = result_q;
10188     }
10189 }

```

For $z = I + jQ$, repeated complex multiplication computes

$$z^M. \quad (76)$$

If $z = re^{j\phi}$, then

$$z^M = r^M e^{jM\phi}. \quad (77)$$

For M -PSK, raising normalized samples to the modulation order largely removes the discrete symbol phase, leaving the common carrier rotation multiplied by M .

8.4 Mth-power sample-to-sample frequency estimate

Relevant code (AnalysisWorkstation.c lines 10191–10240):

```

10191 static double ANALYSIS_constellation_estimate_mth_frequency(const double *i_data, const double *q_data,
10192 size_t count,
10193                                     int order, double magnitude_gate) {
10194     /*
10195      Purpose: Estimates angular frequency offset using an Mth-power phase measurement
10196      Returns: Estimated radians per sample
10197     */
10198     double sum_i = 0.0;
10199     double sum_q = 0.0;
10200     double previous_i = 0.0;
10201     double previous_q = 0.0;
10202     int have_previous = 0;
10203     for (size_t n = 0; n < count; n++) {
10204         double magnitude = hypot(i_data[n], q_data[n]);
10205         if (magnitude < magnitude_gate) {
10206             have_previous = 0;
10207             continue;
10208         }
10209         double unit_i = i_data[n] / magnitude;
10210         double unit_q = q_data[n] / magnitude;
10211         double powered_i = 0.0;
10212         double powered_q = 0.0;
10213         ANALYSIS_constellation_complex_power(unit_i, unit_q, order, &powered_i, &powered_q);
10214         if (have_previous) {
10215             sum_i += powered_i * previous_i + powered_q * previous_q;
10216             sum_q += powered_q * previous_i - powered_i * previous_q;
10217         }
10218         previous_i = powered_i;
10219         previous_q = powered_q;
10220         have_previous = 1;
10221     }
10222     if (hypot(sum_i, sum_q) < 1e-12) {
10223         return 0.0;
10224     }
10225     return atan2(sum_q, sum_i) / (double)order;
10226 }

```

After amplitude normalization and M th-power transformation, let

$$u[n] = \left(\frac{x[n]}{|x[n]|} \right)^M. \quad (78)$$

The code accumulates the complex adjacent-sample correlation

$$C = \sum_n u[n] u^*[n-1]. \quad (79)$$

The average original-signal angular increment is estimated as

$$\hat{\omega} = \frac{1}{M} \arg(C). \quad (80)$$

Magnitude gating breaks the correlation across low-amplitude regions.

8.5 Mth-order coherence score

Relevant code (AnalysisWorkstation.c lines 10242–10281):

```

10242 static double ANALYSIS_constellation_mth_coherence(const double *i_data, const double *q_data, size_t count,
10243 int order,
10244 double frequency, double magnitude_gate) {
10245     /*
10246     Purpose: Measures Mth-order phase coherence after applying a candidate frequency correction
10247     Returns: Normalized coherence score
10248     */
10249     double sum_i = 0.0;
10250     double sum_q = 0.0;
10251     int used = 0;
10252     double step_i = cos(-frequency);
10253     double step_q = sin(-frequency);
10254     double oscillator_i = 1.0;
10255     double oscillator_q = 0.0;
10256     for (size_t n = 0; n < count; n++) {
10257         double corrected_i = i_data[n] * oscillator_i - q_data[n] * oscillator_q;
10258         double corrected_q = i_data[n] * oscillator_q + q_data[n] * oscillator_i;
10259         double magnitude = hypot(corrected_i, corrected_q);
10260         if (magnitude >= magnitude_gate) {
10261             double powered_i = 0.0;
10262             double powered_q = 0.0;
10263             ANALYSIS_constellation_complex_power(corrected_i / magnitude, corrected_q / magnitude, order,
10264             &powered_i,
10265             &powered_q);
10266             sum_i += powered_i;
10267             sum_q += powered_q;
10268             used++;
10269         }
10270         double next_i = oscillator_i * step_i - oscillator_q * step_q;
10271         double next_q = oscillator_q * step_i + oscillator_i * step_q;
10272         oscillator_i = next_i;
10273         oscillator_q = next_q;
10274     }
10275     return used > 0 ? hypot(sum_i, sum_q) / (double)used : 0.0;
10276 }

```

After applying a candidate frequency correction, the unit-magnitude M th powers are summed. The normalized coherence is

$$\gamma_M = \frac{1}{N_u} \left| \sum_{n \in \mathcal{U}} \left(\frac{y[n]}{|y[n]|} \right)^M \right|. \quad (81)$$

A value near 1 means the M th-power phases align strongly; a small value indicates dispersed phase.

8.6 Direct weighted frequency estimate

Relevant code (AnalysisWorkstation.c lines 10284–10310):

```

10284 static double ANALYSIS_constellation_estimate_direct_frequency(const double *i_data, const double *q_data,
10285 size_t count,
10286 double magnitude_gate) {
10287     /*
10288     Purpose: Estimates direct sample-to-sample angular frequency from gated IQ samples
10289     Returns: Estimated radians per sample
10290     */
10291     double sum_i = 0.0;
10292     double sum_q = 0.0;
10293     for (size_t n = 1; n < count; n++) {
10294         double previous_magnitude = hypot(i_data[n - 1U], q_data[n - 1U]);
10295         double current_magnitude = hypot(i_data[n], q_data[n]);
10296         if (previous_magnitude < magnitude_gate || current_magnitude < magnitude_gate) {
10297             continue;
10298         }
10299         double weight = previous_magnitude * current_magnitude;
10300         sum_i += weight * (i_data[n] * i_data[n - 1U] + q_data[n] * q_data[n - 1U]);
10301     }

```



```

10306     sum_q += weight * (q_data[n] * i_data[n - 1U] - i_data[n] * q_data[n - 1U]);
10307 }
10309 return hypot(sum_i, sum_q) > 1e-12 ? atan2(sum_q, sum_i) : 0.0;
10310 }

```

For adjacent valid samples, the complex product

$$x[n]x^*[n-1] \quad (82)$$

encodes the phase step. RetroSpectrum weights each contribution by

$$w_n = |x[n]| |x[n-1]|, \quad (83)$$

then forms

$$C = \sum_n w_n x[n] x^*[n-1]. \quad (84)$$

The estimate is simply

$$\hat{\omega} = \arg(C). \quad (85)$$

This helper is defined in the source but is not currently called by the active constellation pipeline.

9 FFT-Based Carrier-Frequency Estimation

9.1 PSK Mth-power FFT estimator

Relevant code (AnalysisWorkstation.c lines 11383–11401):

```

11383 for (size_t n = 0; n < count; n++) {
11384     double magnitude = hypot(i_data[n], q_data[n]);
11386     if (magnitude < magnitude_gate) {
11388         continue;
11390     }
11392     double powered_i = 0.0;
11393     double powered_q = 0.0;
11394     double unit_i = i_data[n] / magnitude;
11395     double unit_q = q_data[n] / magnitude;
11396     double window = count > 1U ? 0.5 - 0.5 * cos((2.0 * M_PI * (double)n) / (double)(count - 1U)) : 1.0;
11398     ANALYSIS_constellation_complex_power(unit_i, unit_q, order, &powered_i, &powered_q);
11399     time_data[n][0] = powered_i * window;
11400     time_data[n][1] = powered_q * window;
11401 }

```

For valid PSK samples, the estimator constructs

$$v[n] = w[n] \left(\frac{x[n]}{|x[n]|} \right)^M \quad (86)$$

with a Hann window. If the original carrier offset is f_Δ , the dominant spectral line after the M th power is approximately Mf_Δ .

Peak search and interpolation (lines 11415–11486):

```

11415 double maximum_power = -1.0;
11416 size_t maximum_bin = 0U;
11417 double search_hz = maximum_offset_hz * (double)order;
11419 if (search_hz < Global_Analysis_Sample_Rate / (double)fft_size) {
11421     search_hz = Global_Analysis_Sample_Rate / (double)fft_size;
11423 }
11425 if (search_hz > Global_Analysis_Sample_Rate * 0.48) {
11427     search_hz = Global_Analysis_Sample_Rate * 0.48;
11429 }
11431 for (size_t bin = 0; bin < fft_size; bin++) {

```

```

11432     long long signed_bin = bin <= fft_size / 2U ? (long long)bin : (long long)bin - (long long)fft_size;
11433     double frequency_hz = (double)signed_bin * Global_Analysis_Sample_Rate / (double)fft_size;
11435     if (fabs(frequency_hz) > search_hz) {
11437         continue;
11439     }
11441     double power =
11442         frequency_data[bin][0] * frequency_data[bin][0] + frequency_data[bin][1] * frequency_data[bin][1];
11444     if (power > maximum_power) {
11446         maximum_power = power;
11447         maximum_bin = bin;
11449     }
11450 }
11452 double fractional_bin = 0.0;
11454 if (maximum_bin > 0U && maximum_bin + 1U < fft_size) {
11456     double left_power = frequency_data[maximum_bin - 1U][0] * frequency_data[maximum_bin - 1U][0] +
11457         frequency_data[maximum_bin - 1U][1] * frequency_data[maximum_bin - 1U][1];
11458     double center_power = frequency_data[maximum_bin][0] * frequency_data[maximum_bin][0] +
11459         frequency_data[maximum_bin][1] * frequency_data[maximum_bin][1];
11460     double right_power = frequency_data[maximum_bin + 1U][0] * frequency_data[maximum_bin + 1U][0] +
11461         frequency_data[maximum_bin + 1U][1] * frequency_data[maximum_bin + 1U][1];
11462     double left_log = log(left_power + 1e-30);
11463     double center_log = log(center_power + 1e-30);
11464     double right_log = log(right_power + 1e-30);
11465     double denominator = left_log - 2.0 * center_log + right_log;
11467     if (fabs(denominator) > 1e-12) {
11469         fractional_bin = 0.5 * (left_log - right_log) / denominator;
11470         fractional_bin = ANALYSIS_limit_double(fractional_bin, -0.5, 0.5);
11472     }
11474 }
11476 long long signed_peak_bin =
11477     maximum_bin <= fft_size / 2U ? (long long)maximum_bin : (long long)maximum_bin - (long long)fft_size;
11478 double powered_frequency_hz =
11479     ((double)signed_peak_bin + fractional_bin) * Global_Analysis_Sample_Rate / (double)fft_size;
11480 double carrier_frequency_hz = powered_frequency_hz / (double)order;
11482 fftw_destroy_plan(plan);
11483 fftw_free(time_data);
11484 fftw_free(frequency_data);
11486 return 2.0 * M_PI * carrier_frequency_hz / Global_Analysis_Sample_Rate;

```

The estimator searches only the allowed offset range. Peak power is

$$P[k] = |V[k]|^2. \quad (87)$$

A three-bin log-parabolic interpolation estimates a sub-bin displacement

$$\delta = \frac{1}{2} \frac{\ln P_{k-1} - \ln P_{k+1}}{\ln P_{k-1} - 2 \ln P_k + \ln P_{k+1}}, \quad (88)$$

clamped to $[-0.5, 0.5]$. The powered-signal frequency is

$$f_M = (k_s + \delta) \frac{F_s}{N}, \quad (89)$$

where k_s is the signed peak bin. The carrier estimate is

$$\hat{f}_\Delta = \frac{f_M}{M}, \quad \hat{\omega} = \frac{2\pi \hat{f}_\Delta}{F_s}. \quad (90)$$

9.2 QAM fourth-moment FFT estimator

Relevant code (AnalysisWorkstation.c lines 11531–11582):

```

11531     double average_power = 0.0;
11533     for (size_t n = 0; n < count; n++) {
11534         average_power += i_data[n] * i_data[n] + q_data[n] * q_data[n];

```

```

11535     }
11536     double rms = sqrt(average_power / (double)count);
11537     if (rms < 1e-12) {
11538         fftw_free(time_data);
11539         fftw_free(frequency_data);
11540         return 0.0;
11541     }
11542     for (size_t n = 0; n < fft_size; n++) {
11543         time_data[n][0] = 0.0;
11544         time_data[n][1] = 0.0;
11545     }
11546     for (size_t n = 0; n < count; n++) {
11547         double magnitude = hypot(i_data[n], q_data[n]);
11548         if (magnitude < magnitude_gate) {
11549             continue;
11550         }
11551         double normalized_i = i_data[n] / rms;
11552         double normalized_q = q_data[n] / rms;
11553         double normalized_magnitude = hypot(normalized_i, normalized_q);
11554         /* Limit impulsive samples without discarding QAM amplitude levels. */
11555         if (normalized_magnitude > 4.0) {
11556             normalized_i *= 4.0 / normalized_magnitude;
11557             normalized_q *= 4.0 / normalized_magnitude;
11558         }
11559         double powered_i = 0.0;
11560         double powered_q = 0.0;
11561         double window = count > 1U ? 0.5 - 0.5 * cos((2.0 * M_PI * (double)n) / (double)(count - 1U)) : 1.0;
11562         /* Square QAM has a non-zero fourth moment at four times the CFO. */
11563         ANALYSIS_constellation_complex_power(normalized_i, normalized_q, 4, &powered_i, &powered_q);
11564         time_data[n][0] = powered_i * window;
11565         time_data[n][1] = powered_q * window;
11566     }

```

The QAM estimator first computes RMS amplitude

$$r_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_n |x[n]|^2}, \quad (91)$$

normalizes by this RMS, limits impulsive normalized magnitudes to 4, and computes the fourth power

$$v[n] = w[n] \left(\frac{x[n]}{r_{\text{RMS}}} \right)^4. \quad (92)$$

Square QAM has a useful non-zero fourth-order moment, so carrier rotation appears at approximately four times the carrier offset.

Peak conversion (lines 11594–11665):

```

11594     fftw_execute(plan);
11595     double search_hz = maximum_offset_hz * 4.0;
11596     if (search_hz < Global_Analysis_Sample_Rate / (double)fft_size) {
11597         search_hz = Global_Analysis_Sample_Rate / (double)fft_size;
11598     }
11599     if (search_hz > Global_Analysis_Sample_Rate * 0.48) {
11600         search_hz = Global_Analysis_Sample_Rate * 0.48;
11601     }
11602     double maximum_power = -1.0;
11603     size_t maximum_bin = 0U;
11604     for (size_t bin = 0; bin < fft_size; bin++) {
11605         long long signed_bin = bin <= fft_size / 2U ? (long long)bin : (long long)bin - (long long)fft_size;
11606         double frequency_hz = (double)signed_bin * Global_Analysis_Sample_Rate / (double)fft_size;
11607         if (fabs(frequency_hz) > search_hz) {
11608             continue;
11609         }
11610         double power =
11611             frequency_data[bin][0] * frequency_data[bin][0] + frequency_data[bin][1] * frequency_data[bin][1];
11612         if (power > maximum_power) {

```

```

11628     maximum_power = power;
11629     maximum_bin = bin;
11631 }
11632 }
11633 double fractional_bin = 0.0;
11634 size_t left_bin = maximum_bin == 0U ? fft_size - 1U : maximum_bin - 1U;
11635 size_t right_bin = maximum_bin + 1U == fft_size ? 0U : maximum_bin + 1U;
11636 double left_power = frequency_data[left_bin][0] * frequency_data[left_bin][0] +
11637     frequency_data[left_bin][1] * frequency_data[left_bin][1];
11638 double center_power = frequency_data[maximum_bin][0] * frequency_data[maximum_bin][0] +
11639     frequency_data[maximum_bin][1] * frequency_data[maximum_bin][1];
11640 double right_power = frequency_data[right_bin][0] * frequency_data[right_bin][0] +
11641     frequency_data[right_bin][1] * frequency_data[right_bin][1];
11642 double left_log = log(left_power + 1e-30);
11643 double center_log = log(center_power + 1e-30);
11644 double right_log = log(right_power + 1e-30);
11645 double denominator = left_log - 2.0 * center_log + right_log;
11646 if (fabs(denominator) > 1e-12) {
11647     fractional_bin = 0.5 * (left_log - right_log) / denominator;
11648     fractional_bin = ANALYSIS_limit_double(fractional_bin, -0.5, 0.5);
11649 }
11650 long long signed_peak_bin =
11651     maximum_bin <= fft_size / 2U ? (long long)maximum_bin : (long long)maximum_bin - (long long)fft_size;
11652 double fourth_power_frequency_hz =
11653     ((double)signed_peak_bin + fractional_bin) * Global_Analysis_Sample_Rate / (double)fft_size;
11654 double carrier_frequency_hz = fourth_power_frequency_hz / 4.0;
11655 fftw_destroy_plan(plan);
11656 fftw_free(time_data);
11657 fftw_free(frequency_data);
11658 return 2.0 * M_PI * carrier_frequency_hz / Global_Analysis_Sample_Rate;

```

The same log-parabolic peak interpolation is used. The final conversion divides the fourth-power line by four:

$$\hat{f}_{\Delta} = \frac{1}{4}(k_s + \delta)\frac{F_s}{N}, \quad \hat{\omega} = \frac{2\pi\hat{f}_{\Delta}}{F_s}. \quad (93)$$

9.3 Spectral-centroid estimator for ASK/OOK

Relevant code (AnalysisWorkstation.c lines 11754–11816):

```

11754 size_t selected_count = 0U;
11755 for (size_t bin = 0; bin < fft_size; bin++) {
11756     long long signed_bin = bin <= fft_size / 2U ? (long long)bin : (long long)bin - (long long)fft_size;
11757     double frequency_hz = (double)signed_bin * Global_Analysis_Sample_Rate / (double)fft_size;
11758     if (fabs(frequency_hz) <= maximum_offset_hz) {
11759         selected_power[selected_count++] =
11760             frequency_data[bin][0] * frequency_data[bin][0] + frequency_data[bin][1] *
11761             frequency_data[bin][1];
11762     }
11763 }
11764 if (selected_count == 0U) {
11765     fftw_destroy_plan(plan);
11766     fftw_free(time_data);
11767     fftw_free(frequency_data);
11768     free(selected_power);
11769     return 0.0;
11770 }
11771 qsort(selected_power, selected_count, sizeof(double), ANALYSIS_constellation_double_compare);
11772 double noise_power = selected_power[(size_t)(0.20 * (double)(selected_count - 1U))];
11773 double weighted_frequency = 0.0;
11774 double weight_sum = 0.0;
11775 for (size_t bin = 0; bin < fft_size; bin++) {
11776     long long signed_bin = bin <= fft_size / 2U ? (long long)bin : (long long)bin - (long long)fft_size;
11777     double frequency_hz = (double)signed_bin * Global_Analysis_Sample_Rate / (double)fft_size;
11778     if (fabs(frequency_hz) > maximum_offset_hz) {
11779         continue;
11780     }
11781     double power =

```

```

11794     frequency_data[bin][0] * frequency_data[bin][0] + frequency_data[bin][1] * frequency_data[bin][1];
11795     double weight = power - 2.0 * noise_power;
11797     if (weight > 0.0) {
11799         weighted_frequency += frequency_hz * weight;
11800         weight_sum += weight;
11802     }
11803 }
11805 fftw_destroy_plan(plan);
11806 fftw_free(time_data);
11807 fftw_free(frequency_data);
11808 free(selected_power);
11810 if (weight_sum <= 1e-18) {
11812     return 0.0;
11814 }
11816 return 2.0 * M_PI * (weighted_frequency / weight_sum) / Global_Analysis_Sample_Rate;

```

Within the allowed frequency range, the 20th-percentile spectral power is used as a noise estimate P_N . Each bin receives positive weight

$$w_k = \max(P[k] - 2P_N, 0). \quad (94)$$

The estimated spectral center is the weighted centroid

$$\hat{f}_c = \frac{\sum_k f_k w_k}{\sum_k w_k}, \quad (95)$$

which is returned as angular frequency

$$\hat{\omega} = \frac{2\pi \hat{f}_c}{F_s}. \quad (96)$$

10 Constellation Family Scoring

10.1 One-dimensional K-means distortion

Relevant code (AnalysisWorkstation.c lines 10709–10745):

```

10709     double centers[8] = {0.0};
10711     for (int level = 0; level < level_count; level++) {
10712         centers[level] = minimum + ((double)level + 0.5) * (maximum - minimum) / (double)level_count;
10713     }
10715     for (int iteration = 0; iteration < 10; iteration++) {
10716         double sums[8] = {0.0};
10717         int counts[8] = {0};
10719         for (int i = 0; i < count; i++) {
10720             int best_level = 0;
10721             double best_distance = fabs(values[i] - centers[0]);
10723             for (int level = 1; level < level_count; level++) {
10724                 double distance = fabs(values[i] - centers[level]);
10726                 if (distance < best_distance) {
10728                     best_distance = distance;
10729                     best_level = level;
10731                 }
10732             }
10734             sums[best_level] += values[i];
10735             counts[best_level]++;
10736         }
10738         for (int level = 0; level < level_count; level++) {
10740             if (counts[level] > 0) {
10742                 centers[level] = sums[level] / (double)counts[level];
10744             }
10745         }

```

For K levels, initial centroids are uniformly distributed across the observed range. Each iteration assigns sample v_i to the nearest centroid

$$a_i = \arg \min_k |v_i - c_k| \quad (97)$$

and updates

$$c_k = \frac{1}{N_k} \sum_{i:a_i=k} v_i. \quad (98)$$

Normalized error (lines 10748–10769):

```

10748     double error = 0.0;
10749     double variance = 0.0;
10751     for (int i = 0; i < count; i++) {
10752         double best_distance = fabs(values[i] - centers[0]);
10754         for (int level = 1; level < level_count; level++) {
10755             double distance = fabs(values[i] - centers[level]);
10757             if (distance < best_distance) {
10759                 best_distance = distance;
10761             }
10762         }
10764         error += best_distance * best_distance;
10765         double centered = values[i] - mean;
10766         variance += centered * centered;
10767     }
10769     return variance > 1e-12 ? error / variance : 1.0;

```

The distortion is normalized by total variance:

$$E_K = \frac{\sum_i \min_k (v_i - c_k)^2}{\sum_i (v_i - \bar{v})^2}. \quad (99)$$

Smaller values indicate cleaner K -level clustering.

10.2 PSK fit score

Relevant code (AnalysisWorkstation.c lines 10784–10820):

```

10784     double sum_i = 0.0;
10785     double sum_q = 0.0;
10786     double radius_sum = 0.0;
10787     double radius_squared_sum = 0.0;
10788     int used = 0;
10790     for (int p = 0; p < count; p++) {
10791         double radius = hypot(points[p].i, points[p].q);
10793         if (radius < 1e-9) {
10795             continue;
10797         }
10799         double powered_i = 0.0;
10800         double powered_q = 0.0;
10801         ANALYSIS_constellation_complex_power(points[p].i / radius, points[p].q / radius, order, &powered_i,
&powered_q);
10802         sum_i += powered_i;
10803         sum_q += powered_q;
10804         radius_sum += radius;
10805         radius_squared_sum += radius * radius;
10806         used++;
10807     }
10809     if (used < 8) {
10811         return 0.0;
10813     }
10815     double coherence = hypot(sum_i, sum_q) / (double)used;
10816     double mean_radius = radius_sum / (double)used;
10817     double variance = radius_squared_sum / (double)used - mean_radius * mean_radius;
10818     double relative_variance = variance > 0.0 ? variance / (mean_radius * mean_radius + 1e-12) : 0.0;
10820     return coherence / (1.0 + 3.0 * relative_variance);

```

For each nonzero constellation point, its unit phasor is raised to order M . Phase coherence is

$$C_M = \frac{1}{N} \left| \sum_p \left(\frac{z_p}{|z_p|} \right)^M \right|. \quad (100)$$

The radial relative variance is

$$v_r = \frac{\text{Var}(r)}{\bar{r}^2 + \epsilon}. \quad (101)$$

The score is

$$S_{\text{PSK}} = \frac{C_M}{1 + 3v_r}. \quad (102)$$

This rewards equally spaced phase clusters with nearly constant radius.

10.3 QAM fit score

Relevant code (AnalysisWorkstation.c lines 10835–10886):

```

10835     double mean_i = 0.0;
10836     double mean_q = 0.0;
10838     for (int p = 0; p < count; p++) {
10839         mean_i += points[p].i;
10840         mean_q += points[p].q;
10841     }
10843     mean_i /= (double)count;
10844     mean_q /= (double)count;
10846     double fourth_i = 0.0;
10847     double fourth_q = 0.0;
10849     for (int p = 0; p < count; p++) {
10850         double powered_i = 0.0;
10851         double powered_q = 0.0;
10852         ANALYSIS_constellation_complex_power(points[p].i - mean_i, points[p].q - mean_q, 4, &powered_i,
&powered_q);
10853         fourth_i += powered_i;
10854         fourth_q += powered_q;
10855     }
10857     double phase = 0.25 * ANALYSIS_wrap_phase(atan2(fourth_q, fourth_i) - M_PI);
10858     double rotation_i = cos(-phase);
10859     double rotation_q = sin(-phase);
10860     double axis_i[512];
10861     double axis_q[512];
10862     int used = count > 512 ? 512 : count;
10864     for (int p = 0; p < used; p++) {
10865         double centered_i = points[p].i - mean_i;
10866         double centered_q = points[p].q - mean_q;
10867         axis_i[p] = centered_i * rotation_i - centered_q * rotation_q;
10868         axis_q[p] = centered_i * rotation_q + centered_q * rotation_i;
10869     }
10871     double best_distortion = 1.0;
10872     const int levels[] = {2, 4, 8};
10874     for (size_t index = 0; index < sizeof(levels) / sizeof(levels[0]); index++) {
10875         double distortion = 0.5 * (ANALYSIS_constellation_kmeans_1d(axis_i, used, levels[index]) +
ANALYSIS_constellation_kmeans_1d(axis_q, used, levels[index]));
10876
10877         distortion *= 1.0 + 0.025 * (double)(levels[index] - 2);
10879         if (distortion < best_distortion) {
10881             best_distortion = distortion;
10883         }
10884     }
10886     return 1.0 / (0.02 + best_distortion);

```

The constellation is centered, and the aggregate fourth moment determines a square-QAM rotation:

$$\theta = \frac{1}{4} \text{wrap} \left(\arg \sum_p (z_p - \mu)^4 - \pi \right). \quad (103)$$

After rotating by $-\theta$, independent 1-D K-means distortions are computed on the I and Q axes for 2, 4, and 8 levels. For a candidate level count L ,

$$E_L = \frac{1}{2}(E_{I,L} + E_{Q,L}) [1 + 0.025(L - 2)]. \quad (104)$$

The best distortion is converted into

$$S_{\text{QAM}} = \frac{1}{0.02 + E_{\text{min}}}. \quad (105)$$

10.4 ASK/OOK fit score and principal axis

Relevant code (AnalysisWorkstation.c lines 10901–10957):

```

10901     double mean_i = 0.0;
10902     double mean_q = 0.0;
10904     for (int p = 0; p < count; p++) {
10905         mean_i += points[p].i;
10906         mean_q += points[p].q;
10907     }
10909     mean_i /= (double)count;
10910     mean_q /= (double)count;
10912     double covariance_ii = 0.0;
10913     double covariance_qq = 0.0;
10914     double covariance_iq = 0.0;
10916     for (int p = 0; p < count; p++) {
10917         double centered_i = points[p].i - mean_i;
10918         double centered_q = points[p].q - mean_q;
10919         covariance_ii += centered_i * centered_i;
10920         covariance_qq += centered_q * centered_q;
10921         covariance_iq += centered_i * centered_q;
10922     }
10924     double phase = 0.5 * atan2(2.0 * covariance_iq, covariance_ii - covariance_qq);
10925     double rotation_i = cos(-phase);
10926     double rotation_q = sin(-phase);
10927     double axis[512];
10928     double axis_variance = 0.0;
10929     double perpendicular_variance = 0.0;
10930     int used = count > 512 ? 512 : count;
10932     for (int p = 0; p < used; p++) {
10933         double centered_i = points[p].i - mean_i;
10934         double centered_q = points[p].q - mean_q;
10935         double along = centered_i * rotation_i - centered_q * rotation_q;
10936         double perpendicular = centered_i * rotation_q + centered_q * rotation_i;
10937         axis[p] = along;
10938         axis_variance += along * along;
10939         perpendicular_variance += perpendicular * perpendicular;
10940     }
10942     double best_distortion = 1.0;
10943     const int levels[] = {2, 4, 8};
10945     for (size_t index = 0; index < sizeof(levels) / sizeof(levels[0]); index++) {
10946         double distortion = ANALYSIS_constellation_kmeans_1d(axis, used, levels[index]);
10947         distortion *= 1.0 + 0.025 * (double)(levels[index] - 2);
10949         if (distortion < best_distortion) {
10951             best_distortion = distortion;
10953         }
10954     }
10956     double line_ratio = axis_variance / (perpendicular_variance + axis_variance * 0.01 + 1e-12);
10957     return line_ratio / (0.03 + best_distortion);

```

The centered I/Q covariance terms are

$$C_{II} = \sum I_c^2, \quad (106)$$

$$C_{QQ} = \sum Q_c^2, \quad (107)$$

$$C_{IQ} = \sum I_c Q_c. \quad (108)$$

The principal-axis angle is

$$\theta = \frac{1}{2} \text{atan2}(2C_{IQ}, C_{II} - C_{QQ}). \quad (109)$$

After rotation, the ratio of energy along the main axis to perpendicular energy is combined with a 1-D amplitude-level K-means distortion. The implemented score is

$$S_{\text{ASK}} = \frac{V_{\parallel}}{V_{\perp} + 0.01V_{\parallel} + \epsilon} \frac{1}{0.03 + E_{\min}}. \quad (110)$$

A clean ASK/OOK constellation should be strongly one-dimensional and cluster into discrete amplitude levels.

11 Symbol Sampling and Timing Recovery

11.1 Symbol-spaced candidate averaging

Relevant code (AnalysisWorkstation.c lines 10990–11031):

```

10990     size_t symbol_stride = available > (size_t)maximum_points ? available / (size_t)maximum_points : 1U;
10991     int half_window = samples_per_symbol / 10;
10993     if (half_window < 0) {
10995         half_window = 0;
10997     }
10999     if (half_window > 16) {
11001         half_window = 16;
11003     }
11005     int point_count = 0;
11007     for (size_t symbol = 0; symbol < available && point_count < maximum_points; symbol += symbol_stride) {
11008         size_t center = first + symbol * (size_t)samples_per_symbol;
11009         double sum_i = 0.0;
11010         double sum_q = 0.0;
11011         int samples = 0;
11013         for (int k = -half_window; k <= half_window; k++) {
11014             long long index = (long long)center + (long long)k;
11016             if (index < 0 || (size_t)index >= count) {
11018                 continue;
11020             }
11022             sum_i += i_data[index];
11023             sum_q += q_data[index];
11024             samples++;
11025         }
11027         if (samples > 0) {
11029             points[point_count].i = sum_i / (double)samples;
11030             points[point_count].q = sum_q / (double)samples;
11031             point_count++;

```

Given samples-per-symbol S and offset o , candidate centers occur at

$$n_m = n_0 + o + mS. \quad (111)$$

Rather than using one raw sample, the code averages a small window around each center:

$$z_m = \frac{1}{2H+1} \sum_{k=-H}^H x[n_m + k], \quad (112)$$

with $H \approx S/10$ but capped at 16. This reduces noise while retaining symbol-center information.

11.2 Transition-energy timing metric

Relevant code (AnalysisWorkstation.c lines 11052–11143):

```

11052     size_t region_start = guard > 1U ? guard : 1U;
11053     size_t region_end = count - guard;
11054     double total_energy = 0.0;
11055     for (size_t n = region_start; n < region_end; n++) {
11056         double delta_i = i_data[n] - i_data[n - 1U];
11057         double delta_q = q_data[n] - q_data[n - 1U];
11058         total_energy += delta_i * delta_i + delta_q * delta_q;
11059     }
11060     if (total_energy <= 1e-18) {
11061         return 1.0;
11062     }
11063     int half_window = samples_per_symbol / 30;
11064     if (half_window < 2) {
11065         half_window = 2;
11066     }
11067     if (half_window > 8) {
11068         half_window = 8;
11069     }
11070     long long first_boundary = (long long)guard + (long long)center_offset - samples_per_symbol / 2;
11071     while (first_boundary < (long long)region_start) {
11072         first_boundary += samples_per_symbol;
11073     }
11074     double captured_energy = 0.0;
11075     double center_energy = 0.0;
11076     size_t covered_samples = 0U;
11077     int boundary_count = 0;
11078     int center_count = 0;
11079     for (long long boundary = first_boundary; boundary < (long long)region_end; boundary +=
11080         samples_per_symbol) {
11081         boundary_count++;
11082         for (int k = -half_window; k <= half_window; k++) {
11083             long long index = boundary + k;
11084             if (index <= 0 || index < (long long)region_start || index >= (long long)region_end) {
11085                 continue;
11086             }
11087             double delta_i = i_data[index] - i_data[index - 1];
11088             double delta_q = q_data[index] - q_data[index - 1];
11089             captured_energy += delta_i * delta_i + delta_q * delta_q;
11090             covered_samples++;
11091         }
11092     }
11093     size_t first_center = guard + (size_t)center_offset;
11094     for (size_t center = first_center; center < region_end; center += (size_t)samples_per_symbol) {
11095         if (center == 0U) {
11096             continue;
11097         }
11098         double delta_i = i_data[center] - i_data[center - 1U];
11099         double delta_q = q_data[center] - q_data[center - 1U];
11100         center_energy += delta_i * delta_i + delta_q * delta_q;
11101         center_count++;
11102     }
11103     if (boundary_count < 2 || covered_samples == 0U) {
11104         return 0.1;
11105     }
11106     double region_samples = (double)(region_end - region_start);
11107     double coverage_fraction = (double)covered_samples / region_samples;
11108     double captured_fraction = captured_energy / total_energy;
11109     double concentration = captured_fraction / (coverage_fraction + 1e-12);
11110     double average_energy = total_energy / region_samples;
11111     double center_average = center_count > 0 ? center_energy / (double)center_count : average_energy;
11112     double center_penalty = 1.0 + center_average / (average_energy + 1e-18);
11113     return concentration * sqrt(ANALYSIS_limit_double(captured_fraction, 0.0, 1.0)) / center_penalty;

```

The basic transition energy is

$$e[n] = |x[n] - x[n - 1]|^2 = (\Delta I[n])^2 + (\Delta Q[n])^2. \quad (113)$$

For a timing candidate, the code measures how much total transition energy is captured near predicted symbol boundaries. If E_B is captured boundary energy, E_T is total energy, and c is the fraction of samples covered by the boundary windows,

$$f_{\text{capt}} = \frac{E_B}{E_T}, \quad (114)$$

$$C = \frac{f_{\text{capt}}}{c + \epsilon}. \quad (115)$$

It also computes a center penalty from transition energy at proposed symbol centers. The returned timing score is

$$S_t = \frac{C \sqrt{\text{clip}(f_{\text{capt}}, 0, 1)}}{1 + E_{\text{center,avg}}/E_{\text{avg}}}. \quad (116)$$

The intended behavior is high transition concentration near boundaries and low transition energy at symbol centers.

11.3 Active refined timing search

The active linear-modulation path calls `ANALYSIS_constellation_find_symbol_timing_v2()`.

Search bounds (AnalysisWorkstation.c lines 12245–12291):

```

12245     double minimum_rate_hz = effective_bandwidth / 64.0;
12246     double maximum_rate_hz = effective_bandwidth * 2.0;
12248     if (minimum_rate_hz < 100.0) {
12250         minimum_rate_hz = 100.0;
12252     }
12254     if (maximum_rate_hz > Global_Analysis_Sample_Rate * 0.5) {
12256         maximum_rate_hz = Global_Analysis_Sample_Rate * 0.5;
12258     }
12260     if (maximum_rate_hz <= minimum_rate_hz) {
12262         maximum_rate_hz = minimum_rate_hz * 2.0;
12264     }
12266     int minimum_sps = (int)floor(Global_Analysis_Sample_Rate / maximum_rate_hz) - 2;
12267     int maximum_sps = (int)ceil(Global_Analysis_Sample_Rate / minimum_rate_hz) + 2;
12269     if (minimum_sps < 2) {
12271         minimum_sps = 2;
12273     }
12275     if (minimum_sps > 2048) {
12277         minimum_sps = 2048;
12279     }
12281     if (maximum_sps > 2048) {
12283         maximum_sps = 2048;
12285     }
12287     if (maximum_sps < minimum_sps) {
12289         maximum_sps = minimum_sps;
12291     }

```

The candidate symbol-rate range is based on selected signal bandwidth B :

$$R_{\min} = \max(B/64, 100 \text{ Hz}), \quad R_{\max} = \min(2B, F_s/2). \quad (117)$$

These rates become samples-per-symbol limits

$$S_{\min} \approx \left\lfloor \frac{F_s}{R_{\max}} \right\rfloor - 2, \quad S_{\max} \approx \left\lceil \frac{F_s}{R_{\min}} \right\rceil + 2, \quad (118)$$

with final clamping to $[2, 2048]$.

Folded transition-energy search (lines 12339–12511):

```
12339     for (size_t index = 0; index < transition_count; index++) {
12340         size_t n = timing_start + index;
12341         double previous_magnitude = hypot(i_data[n - 1U], q_data[n - 1U]);
12342         double current_magnitude = hypot(i_data[n], q_data[n]);
12343         if (previous_magnitude < magnitude_gate || current_magnitude < magnitude_gate) {
12344             transition_energy[index] = 0.0;
12345             continue;
12346         }
12347         double delta_i = i_data[n] - i_data[n - 1U];
12348         double delta_q = q_data[n] - q_data[n - 1U];
12349         transition_energy[index] = delta_i * delta_i + delta_q * delta_q;
12350     }
12351     Type_Analysis_Constellation_Point candidate[384];
12352     double best_score = -1.0;
12353     for (int sps = minimum_sps; sps <= maximum_sps; sps++) {
12354         memset(histogram, 0, (size_t)sps * sizeof(double));
12355         memset(histogram_count, 0, (size_t)sps * sizeof(int));
12356         memset(smoothed, 0, (size_t)sps * sizeof(double));
12357         int phase = (int)(timing_start % (size_t)sps);
12358         for (size_t index = 0; index < transition_count; index++) {
12359             histogram[phase] += transition_energy[index];
12360             histogram_count[phase]++;
12361             phase++;
12362             if (phase >= sps) {
12363                 phase = 0;
12364             }
12365         }
12366         for (phase = 0; phase < sps; phase++) {
12367             if (histogram_count[phase] > 0) {
12368                 histogram[phase] /= (double)histogram_count[phase];
12369             }
12370         }
12371         int smoothing_radius = sps / 100;
12372         if (smoothing_radius < 1) {
12373             smoothing_radius = 1;
12374         }
12375         if (smoothing_radius > 6) {
12376             smoothing_radius = 6;
12377         }
12378         double mean_smoothed = 0.0;
12379         double maximum_smoothed = -1.0;
12380         int boundary_phase = 0;
12381         for (phase = 0; phase < sps; phase++) {
12382             for (int delta = -smoothing_radius; delta <= smoothing_radius; delta++) {
12383                 int folded_phase = phase + delta;
12384                 while (folded_phase < 0) {
12385                     folded_phase += sps;
12386                 }
12387                 while (folded_phase >= sps) {
12388                     folded_phase -= sps;
12389                 }
12390                 smoothed[phase] += histogram[folded_phase];
12391             }
12392             mean_smoothed += smoothed[phase];
12393             if (smoothed[phase] > maximum_smoothed) {
12394                 maximum_smoothed = smoothed[phase];
12395                 boundary_phase = phase;
12396             }
12397         }
12398         mean_smoothed /= (double)sps;
12399         if (mean_smoothed <= 1e-18 || maximum_smoothed <= 0.0) {
12400             continue;
12401         }
12402         double concentration = maximum_smoothed / mean_smoothed;
12403         int center_phase = (boundary_phase + sps / 2) % sps;
12404         int guard_phase = (int)(guard % (size_t)sps);
12405         int offset = center_phase - guard_phase;
12406         while (offset < 0) {
12407             offset += sps;
12408         }
```

```

12446     }
12448     while (offset >= sps) {
12449         offset -= sps;
12450     }
12452     int candidate_count =
12453         ANALYSIS_constellation_collect_candidate(i_data, q_data, count, guard, sps, offset, candidate,
384);
12454     double cluster_score = 0.0;
12456     if (family == ANALYSIS_CONSTELLATION_MODE_PSK) {
12458         cluster_score = ANALYSIS_constellation_psk_score(candidate, candidate_count, psk_order);
12460     }
12462     else if (family == ANALYSIS_CONSTELLATION_MODE_QAM) {
12464         cluster_score = ANALYSIS_constellation_qam_score(candidate, candidate_count);
12466     }
12468     else {
12470         cluster_score = ANALYSIS_constellation_ask_score(candidate, candidate_count);
12472     }
12474     if (cluster_score <= 0.0) {
12476         continue;
12478     }
12480     /*
12481     * A real symbol period concentrates transition energy at one folded
12482     * phase. Multiples can also concentrate transitions, so prefer the
12483     * shortest period that still produces clean modulation-family
12484     * clusters. This selects 125 samples/symbol for a 2.5 MS/s capture of
12485     * a 20 ksymbol/s signal instead of 250, 375, or 500.
12486     */
12487     double timing_excess = concentration - 1.0;
12489     if (timing_excess < 0.0001) {
12491         timing_excess = 0.0001;
12493     }
12495     double score = cluster_score * timing_excess / (double)sps;
12497     if (score > best_score) {
12499         best_score = score;
12500         *best_samples_per_symbol = sps;
12501         *best_offset = offset;
12503     }
12504 }
12506 free(transition_energy);
12507 free(histogram);
12508 free(histogram_count);
12509 free(smoothed);
12511 return best_score > 0.0;

```

For each candidate S , transition energies are folded modulo S into a phase histogram

$$H_S[p] = \text{mean}\{e[n] : n \bmod S = p\}. \quad (119)$$

A short circular smoothing identifies the boundary phase p_b with maximum transition energy. The symbol center is taken approximately half a symbol later:

$$p_c = (p_b + S/2) \bmod S. \quad (120)$$

The timing concentration is

$$C_S = \frac{\max_p \tilde{H}_S[p]}{\text{mean}_p \tilde{H}_S[p]}. \quad (121)$$

The code also evaluates the modulation-family cluster score at that timing and ranks candidates by

$$S_{\text{total}} = S_{\text{cluster}} \frac{\max(C_S - 1, 10^{-4})}{S}. \quad (122)$$

The $1/S$ term deliberately favors the shortest symbol period that still produces clean clusters, reducing selection of integer multiples of the real period.

12 Residual Carrier Correction and Linear Constellation Alignment

12.1 PSK residual frequency from symbol-spaced points

Relevant code (AnalysisWorkstation.c lines 12527–12560):

```

12527     double sum_i = 0.0;
12528     double sum_q = 0.0;
12529     double previous_i = 0.0;
12530     double previous_q = 0.0;
12531     int have_previous = 0;
12533     for (int p = 0; p < count; p++) {
12534         double magnitude = hypot(points[p].i, points[p].q);
12536         if (magnitude < 1e-12) {
12538             have_previous = 0;
12539             continue;
12541         }
12543         double powered_i = 0.0;
12544         double powered_q = 0.0;
12545         ANALYSIS_constellation_complex_power(points[p].i / magnitude, points[p].q / magnitude, order,
12546         &powered_i,
12547                                     &powered_q);
12548         if (have_previous) {
12550             sum_i += powered_i * previous_i + powered_q * previous_q;
12551             sum_q += powered_q * previous_i - powered_i * previous_q;
12553         }
12555         previous_i = powered_i;
12556         previous_q = powered_q;
12557         have_previous = 1;
12558     }
12560     return hypot(sum_i, sum_q) > 1e-12 ? atan2(sum_q, sum_i) / ((double)order * (double)samples_per_symbol) :
0.0;

```

For symbol-spaced points z_m , the normalized M th powers are correlated between adjacent symbols:

$$C = \sum_m u_m u_{m-1}^*, \quad u_m = \left(\frac{z_m}{|z_m|} \right)^M. \quad (123)$$

Because adjacent points are S input samples apart, the residual angular frequency per input sample is

$$\hat{\omega}_r = \frac{\arg C}{MS}. \quad (124)$$

12.2 QAM residual frequency

Relevant code (AnalysisWorkstation.c lines 12570–12641):

```

12570     const int block_symbols = 32;
12572     if (!points || count < block_symbols * 4 || samples_per_symbol < 1 || symbol_rate_hz <= 0.0) {
12574         return 0.0;
12576     }
12578     int block_count = count / block_symbols;
12579     double previous_i = 0.0;
12580     double previous_q = 0.0;
12581     int have_previous = 0;
12582     double correlation_i = 0.0;
12583     double correlation_q = 0.0;
12584     double correlation_power = 0.0;
12586     for (int block = 0; block < block_count; block++) {
12587         double block_i = 0.0;
12588         double block_q = 0.0;
12590         for (int symbol = 0; symbol < block_symbols; symbol++) {
12591             const Type_Analysis_Constellation_Point *point = &points[block * block_symbols + symbol];
12592             double powered_i = 0.0;
12593             double powered_q = 0.0;
12595             ANALYSIS_constellation_complex_power(point->i, point->q, 4, &powered_i, &powered_q);

```

```

12596     block_i += powered_i;
12597     block_q += powered_q;
12598 }
12600 block_i /= (double)block_symbols;
12601 block_q /= (double)block_symbols;
12603 if (have_previous) {
12605     correlation_i += block_i * previous_i + block_q * previous_q;
12606     correlation_q += block_q * previous_i - block_i * previous_q;
12607     correlation_power += hypot(block_i, block_q) * hypot(previous_i, previous_q);
12609 }
12611 previous_i = block_i;
12612 previous_q = block_q;
12613 have_previous = 1;
12614 }
12616 double coherence = correlation_power > 1e-18 ? hypot(correlation_i, correlation_q) / correlation_power :
0.0;
12618 if (coherence < 0.12) {
12620     return 0.0;
12622 }
12624 double radians_per_sample =
12625     atan2(correlation_q, correlation_i) / (4.0 * (double)block_symbols * (double)samples_per_symbol);
12626 double residual_hz = radians_per_sample * Global_Analysis_Sample_Rate / (2.0 * M_PI);
12627 double maximum_residual_hz = symbol_rate_hz / 128.0;
12629 if (maximum_residual_hz < 20.0) {
12631     maximum_residual_hz = 20.0;
12633 }
12635 if (fabs(residual_hz) > maximum_residual_hz) {
12637     return 0.0;
12639 }
12641 return radians_per_sample;

```

QAM points are processed in blocks of 32 symbols. Each block forms the average fourth moment

$$b_j = \frac{1}{32} \sum_{m=0}^{31} z_{32j+m}^4. \quad (125)$$

Adjacent block moments are correlated. A normalized coherence

$$\gamma = \frac{|\sum_j b_j b_{j-1}^*|}{\sum_j |b_j| |b_{j-1}|} \quad (126)$$

rejects weak estimates below 0.12. Since the correlation spans $4 \times 32 \times S$ phase multiplication/sample spacing, the residual per input sample is

$$\hat{\omega}_r = \frac{\arg C}{4 \cdot 32 \cdot S}. \quad (127)$$

An additional limit rejects residual offsets larger than $\max(R_s/128, 20 \text{ Hz})$.

12.3 Active PSK/QAM/ASK processing chain

Carrier estimate and timing (AnalysisWorkstation.c lines 12657–12713):

```

12657 double power = 0.0;
12659 for (size_t n = 0; n < count; n++) {
12660     power += i_data[n] * i_data[n] + q_data[n] * q_data[n];
12661 }
12663 double rms = sqrt(power / (double)count);
12664 double gate = rms * 0.15;
12665 int psk_order = selected_psk_order;
12667 if (psk_order != ANALYSIS_CONSTELLATION_PSK_BPSK && psk_order != ANALYSIS_CONSTELLATION_PSK_QPSK &&
12668     psk_order != ANALYSIS_CONSTELLATION_PSK_8PSK) {
12670     psk_order = ANALYSIS_CONSTELLATION_PSK_BPSK;
12672 }

```

```

12674 double maximum_offset_hz = bandwidth_hz * 0.75;
12676 if (maximum_offset_hz < Global_Analysis_Sample_Rate / 4096.0) {
12678     maximum_offset_hz = Global_Analysis_Sample_Rate / 4096.0;
12680 }
12682 double frequency = 0.0;
12684 if (family == ANALYSIS_CONSTELLATION_MODE_PSK) {
12686     frequency = ANALYSIS_constellation_estimate_psk_frequency_fft(i_data, q_data, count, psk_order, gate,
12687                                                                maximum_offset_hz);
12689 }
12691 else if (family == ANALYSIS_CONSTELLATION_MODE_QAM) {
12693     frequency = ANALYSIS_constellation_estimate_qam_frequency_fft(i_data, q_data, count, gate,
12694                                                                maximum_offset_hz);
12695 }
12697 else {
12699     frequency = ANALYSIS_constellation_estimate_spectral_center(i_data, q_data, count, maximum_offset_hz);
12701 }
12703 ANALYSIS_constellation_apply_frequency_correction(i_data, q_data, count, frequency);
12705 int samples_per_symbol = 0;
12706 int offset = 0;
12708 if (!ANALYSIS_constellation_find_symbol_timing_v2(i_data, q_data, count, bandwidth_hz, family, psk_order,
12709 gate,
12709                                                                &samples_per_symbol, &offset)) {
12711     return;
12713 }

```

The input RMS and magnitude gate are

$$r_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_n |x[n]|^2}, \quad r_{\text{gate}} = 0.15 r_{\text{RMS}}. \quad (128)$$

A family-specific carrier estimator is selected: M th-power FFT for PSK, fourth-moment FFT for QAM, and spectral centroid for ASK/OOK. After correction, the refined timing search produces S and symbol-center offset o .

Residual correction and final rotation (lines 12729–12858):

```

12729 Type_Analysis_Constellation_Point output[ANALYSIS_MAX_CONST_POINTS];
12730 int output_count = ANALYSIS_constellation_collect_candidate(i_data, q_data, count, guard,
12731 samples_per_symbol,
12731                                                                offset, output, ANALYSIS_MAX_CONST_POINTS);
12733 if (output_count < 4) {
12735     return;
12737 }
12739 double residual_frequency = 0.0;
12741 if (family == ANALYSIS_CONSTELLATION_MODE_PSK) {
12743     residual_frequency =
12744     ANALYSIS_constellation_psk_residual_frequency(output, output_count, psk_order,
12745     samples_per_symbol);
12746 }
12748 else if (family == ANALYSIS_CONSTELLATION_MODE_QAM) {
12750     double symbol_rate_hz = Global_Analysis_Sample_Rate / (double)samples_per_symbol;
12751     residual_frequency =
12752     ANALYSIS_constellation_qam_residual_frequency(output, output_count, samples_per_symbol,
12753     symbol_rate_hz);
12754 }
12756 else {
12758     residual_frequency = ANALYSIS_constellation_psk_residual_frequency(output, output_count, 1,
12759     samples_per_symbol);
12760 }
12762 if (fabs(residual_frequency) > 1e-15) {
12764     ANALYSIS_constellation_apply_frequency_correction(i_data, q_data, count, residual_frequency);
12766     output_count = ANALYSIS_constellation_collect_candidate(i_data, q_data, count, guard,
12767     samples_per_symbol,
12768     offset, output, ANALYSIS_MAX_CONST_POINTS);
12769 }
12770 if (output_count < 4) {
12772     return;
12774 }

```



```

12776 double rotation_phase = 0.0;
12777 if (family == ANALYSIS_CONSTELLATION_MODE_PSK) {
12780     double sum_i = 0.0;
12781     double sum_q = 0.0;
12783     for (int p = 0; p < output_count; p++) {
12784         double radius = hypot(output[p].i, output[p].q);
12786         if (radius > gate) {
12788             double powered_i = 0.0;
12789             double powered_q = 0.0;
12790             ANALYSIS_constellation_complex_power(output[p].i / radius, output[p].q / radius, psk_order,
12791 &powered_i,
12792 &powered_q);
12793             sum_i += powered_i;
12794             sum_q += powered_q;
12795         }
12796     }
12798     rotation_phase = atan2(sum_q, sum_i) / (double)psk_order;
12800 }
12802 else if (family == ANALYSIS_CONSTELLATION_MODE_QAM) {
12804     double sum_i = 0.0;
12805     double sum_q = 0.0;
12807     for (int p = 0; p < output_count; p++) {
12808         double powered_i = 0.0;
12809         double powered_q = 0.0;
12810         ANALYSIS_constellation_complex_power(output[p].i, output[p].q, 4, &powered_i, &powered_q);
12811         sum_i += powered_i;
12812         sum_q += powered_q;
12813     }
12815     rotation_phase = ANALYSIS_wrap_phase(atan2(sum_q, sum_i) - M_PI) * 0.25;
12817 }
12819 else {
12821     double mean_i = 0.0;
12822     double mean_q = 0.0;
12824     for (int p = 0; p < output_count; p++) {
12825         mean_i += output[p].i;
12826         mean_q += output[p].q;
12827     }
12829     mean_i /= (double)output_count;
12830     mean_q /= (double)output_count;
12832     double covariance_ii = 0.0;
12833     double covariance_qq = 0.0;
12834     double covariance_iq = 0.0;
12836     for (int p = 0; p < output_count; p++) {
12837         double centered_i = output[p].i - mean_i;
12838         double centered_q = output[p].q - mean_q;
12839         covariance_ii += centered_i * centered_i;
12840         covariance_qq += centered_q * centered_q;
12841         covariance_iq += centered_i * centered_q;
12842     }
12844     rotation_phase = 0.5 * atan2(2.0 * covariance_iq, covariance_ii - covariance_qq);
12846 }
12848 double rotation_i = cos(-rotation_phase);
12849 double rotation_q = sin(-rotation_phase);
12851 Global_Analysis_Const_Count = output_count;
12853 for (int p = 0; p < output_count; p++) {
12854     Global_Analysis_Const_I[p] = (float)(output[p].i * rotation_i - output[p].q * rotation_q);
12855     Global_Analysis_Const_Q[p] = (float)(output[p].i * rotation_q + output[p].q * rotation_i);
12856 }
12858 ANALYSIS_constellation_normalize_output(family == ANALYSIS_CONSTELLATION_MODE_ASK_OOK);

```

After collecting symbol points, the code estimates a second residual frequency and, if nonzero, re-corrects the entire IQ sequence before recollecting symbols.

Final orientation is family-specific:

- PSK: $\theta = \arg(\sum(z/|z|)^M)/M$.
- QAM: $\theta = \frac{1}{4} \text{wrap}(\arg \sum z^4 - \pi)$.
- ASK/OOK: $\theta = \frac{1}{2} \text{atan2}(2C_{IQ}, C_{II} - C_{QQ})$ from the principal covariance axis.

Every point is then rotated by $e^{-j\theta}$ and passed to the common display normalizer.

12.4 Robust constellation display normalization

Relevant code (AnalysisWorkstation.c lines 11280–11333):

```

11280     if (!preserve_origin) {
11281         double mean_i = 0.0;
11282         double mean_q = 0.0;
11283         for (int p = 0; p < count; p++) {
11284             mean_i += Global_Analysis_Const_I[p];
11285             mean_q += Global_Analysis_Const_Q[p];
11286         }
11287         mean_i /= (double)count;
11288         mean_q /= (double)count;
11289         for (int p = 0; p < count; p++) {
11290             Global_Analysis_Const_I[p] = (float)((double)Global_Analysis_Const_I[p] - mean_i);
11291             Global_Analysis_Const_Q[p] = (float)((double)Global_Analysis_Const_Q[p] - mean_q);
11292         }
11293     }
11294     double *components = malloc((size_t)count * sizeof(double));
11295     if (!components) {
11296         return;
11297     }
11298     for (int p = 0; p < count; p++) {
11299         double absolute_i = fabs((double)Global_Analysis_Const_I[p]);
11300         double absolute_q = fabs((double)Global_Analysis_Const_Q[p]);
11301         components[p] = absolute_i > absolute_q ? absolute_i : absolute_q;
11302     }
11303     qsort(components, (size_t)count, sizeof(double), ANALYSIS_constellation_double_compare);
11304     int percentile_index = (int)((double)(count - 1) * 0.985);
11305     double scale = components[percentile_index];
11306     free(components);
11307     if (scale < 1e-12) {
11308         return;
11309     }
11310     scale /= 0.86;
11311     for (int p = 0; p < count; p++) {
11312         double normalized_i = (double)Global_Analysis_Const_I[p] / scale;
11313         double normalized_q = (double)Global_Analysis_Const_Q[p] / scale;
11314         Global_Analysis_Const_I[p] = (float)ANALYSIS_limit_double(normalized_i, -1.0, 1.0);
11315         Global_Analysis_Const_Q[p] = (float)ANALYSIS_limit_double(normalized_q, -1.0, 1.0);
11316     }

```

Unless the origin must be preserved for ASK/OOK, the output constellation mean is subtracted. The scale statistic for each point is

$$c_p = \max(|I_p|, |Q_p|). \quad (129)$$

The 98.5th percentile $c_{0.985}$ is used instead of the absolute maximum, making the display resistant to a small number of outliers. The scale is

$$s = \frac{c_{0.985}}{0.86}, \quad (130)$$

and display coordinates are

$$I'_p = \text{clip}(I_p/s, -1, 1), \quad Q'_p = \text{clip}(Q_p/s, -1, 1). \quad (131)$$

13 FSK / MSK Discriminator Recovery

13.1 Magnitude gate and instantaneous-frequency discriminator

Relevant code (AnalysisWorkstation.c lines 13072–13100):

```

13072     double average_power = 0.0;
13074     for (size_t n = 0U; n < count; n++) {
13075         average_power += i_data[n] * i_data[n] + q_data[n] * q_data[n];
13076     }
13078     average_power /= (double)count;
13079     double magnitude_gate = sqrt(average_power) * 0.18;
13080     size_t valid_count = 0U;
13082     for (size_t n = 1U; n < count; n++) {
13083         double current_magnitude = hypot(i_data[n], q_data[n]);
13084         double previous_magnitude = hypot(i_data[n - 1U], q_data[n - 1U]);
13085         size_t index = n - 1U;
13087         if (current_magnitude < magnitude_gate || previous_magnitude < magnitude_gate) {
13089             frequency_hz[index] = 0.0;
13090             continue;
13092         }
13094         double product_i = i_data[n] * i_data[n - 1U] + q_data[n] * q_data[n - 1U];
13095         double product_q = q_data[n] * i_data[n - 1U] - i_data[n] * q_data[n - 1U];
13096         double phase_increment = atan2(product_q, product_i);
13098         frequency_hz[index] = phase_increment * sample_rate / (2.0 * M_PI);
13099         valid[index] = 1U;
13100         valid_count++;

```

Average complex power and the amplitude gate are

$$P_{\text{avg}} = \frac{1}{N} \sum_n |x[n]|^2, \quad r_{\text{gate}} = 0.18 \sqrt{P_{\text{avg}}}. \quad (132)$$

For adjacent gated samples,

$$\Delta\phi[n] = \arg(x[n]x^*[n-1]), \quad (133)$$

so the discriminator output is

$$f[n] = \frac{F_s}{2\pi} \Delta\phi[n]. \quad (134)$$

This converts FSK/MSK frequency states into a one-dimensional frequency trajectory.

13.2 Smoothing

Relevant code (AnalysisWorkstation.c lines 13140–13184):

```

13140     int smoothing_radius = 2;
13142     if (bandwidth_hz > 0.0) {
13144         smoothing_radius = (int)llround(sample_rate / (8.0 * bandwidth_hz));
13146     }
13148     if (smoothing_radius < 1) {
13150         smoothing_radius = 1;
13152     }
13154     if (smoothing_radius > 24) {
13156         smoothing_radius = 24;
13158     }
13160     for (size_t n = 0U; n < frequency_count; n++) {
13161         size_t first = n > (size_t)smoothing_radius ? n - (size_t)smoothing_radius : 0U;
13162         size_t last = n + (size_t)smoothing_radius + 1U;
13164         if (last > frequency_count) {
13166             last = frequency_count;
13168         }
13170         double sum = 0.0;
13171         size_t used = 0U;
13173         for (size_t sample = first; sample < last; sample++) {
13175             if (valid[sample]) {
13177                 sum += frequency_hz[sample];
13178                 used++;
13180             }
13181         }
13183         smoothed_hz[n] = used > 0U ? sum / (double)used : fallback_center;
13184     }

```

The smoothing radius is approximately

$$R \approx \frac{F_s}{8B} \quad (135)$$

when bandwidth is known, clamped to 1 through 24 samples. The smoothed discriminator is a valid-sample moving average

$$\tilde{f}[n] = \frac{1}{N_n} \sum_{m=n-R}^{n+R} f[m]. \quad (136)$$

Invalid/gated samples are excluded.

13.3 Transition detection

Relevant code (AnalysisWorkstation.c lines 13199–13230):

```

13199     for (size_t n = 1U; n < frequency_count; n++) {
13201         if (!valid[n] || !valid[n - 1U]) {
13203             continue;
13205         }
13207         double strength = fabs(smoothed_hz[n] - smoothed_hz[n - 1U]);
13208         transition_strength[n] = strength;
13210         if (strength > 0.0 && isfinite(strength)) {
13212             strength_values[strength_count++] = strength;
13214         }
13215     }
13217     if (strength_count < 8U) {
13219         free(strength_values);
13220         free(frequency_hz);
13221         free(smoothed_hz);
13222         free(transition_strength);
13223         free(valid);
13224         return;
13226     }
13228     qsort(strength_values, strength_count, sizeof(double), ANALYSIS_constellation_double_compare);
13229     size_t threshold_index = (size_t)(0.85 * (double)(strength_count - 1U));
13230     double transition_threshold = strength_values[threshold_index];

```

Transition strength is

$$d[n] = |\tilde{f}[n] - \tilde{f}[n - 1]|. \quad (137)$$

The 85th percentile of positive transition strengths is used as the detection threshold. Later logic keeps local maxima and enforces a minimum separation, so timing estimation is driven by strong discriminator transitions.

13.4 Samples-per-symbol from transition phase coherence

Relevant code (AnalysisWorkstation.c lines 13288–13374):

```

13288     if (transition_count >= 4U) {
13290         int minimum_sps = 4;
13291         int maximum_sps = 2048;
13293         if (bandwidth_hz > 0.0) {
13295             minimum_sps = (int)floor(sample_rate / (2.0 * bandwidth_hz));
13296             maximum_sps = (int)ceil(sample_rate * 16.0 / bandwidth_hz);
13298         }
13300         if (minimum_sps < 4) {
13302             minimum_sps = 4;
13304         }
13306         if (maximum_sps > 2048) {
13308             maximum_sps = 2048;
13310         }
13312         int count_limited_maximum = (int)(frequency_count / 8U);
13314         if (maximum_sps > count_limited_maximum) {

```

```

13316     maximum_sps = count_limited_maximum;
13317 }
13318
13320     double best_score = -1.0;
13322     for (int candidate_sps = minimum_sps; candidate_sps <= maximum_sps; candidate_sps++) {
13323         double phase_i = 0.0;
13324         double phase_q = 0.0;
13325         double weight_sum = 0.0;
13327         for (size_t transition = 0U; transition < transition_count; transition++) {
13328             double weight = transition_weights[transition];
13329             double phase = 2.0 * M_PI * (double)transition_samples[transition] / (double)candidate_sps;
13330             phase_i += weight * cos(phase);
13331             phase_q += weight * sin(phase);
13332             weight_sum += weight;
13333         }
13335         if (weight_sum <= 0.0) {
13337             continue;
13339         }
13341         double coherence = hypot(phase_i, phase_q) / weight_sum;
13342         double score = coherence + 0.01 * log((double)candidate_sps);
13344         if (score > best_score) {
13346             best_score = score;
13347             samples_per_symbol = candidate_sps;
13349         }
13350     }
13352     if (samples_per_symbol >= 4) {
13354         double phase_i = 0.0;
13355         double phase_q = 0.0;
13357         for (size_t transition = 0U; transition < transition_count; transition++) {
13358             double weight = transition_weights[transition];
13359             double phase = 2.0 * M_PI * (double)transition_samples[transition] /
(double)samples_per_symbol;
13360             phase_i += weight * cos(phase);
13361             phase_q += weight * sin(phase);
13362         }
13364         double boundary_phase = atan2(phase_q, phase_i);
13366         if (boundary_phase < 0.0) {
13368             boundary_phase += 2.0 * M_PI;
13370         }
13372         boundary_offset = boundary_phase * (double)samples_per_symbol / (2.0 * M_PI);
13374     }

```

For a candidate symbol period S , each detected transition at sample n_j is mapped to a unit phasor around the candidate period:

$$e^{j2\pi n_j/S}. \quad (138)$$

With transition-strength weights w_j ,

$$C(S) = \frac{\left| \sum_j w_j e^{j2\pi n_j/S} \right|}{\sum_j w_j}. \quad (139)$$

The implemented ranking adds a small preference for larger candidate S :

$$\text{score}(S) = C(S) + 0.01 \ln S. \quad (140)$$

For the selected S , the weighted phasor angle gives the boundary offset

$$o_b = \frac{S}{2\pi} \arg \left(\sum_j w_j e^{j2\pi n_j/S} \right). \quad (141)$$

13.5 Per-symbol discriminator point

Relevant code (AnalysisWorkstation.c lines 13417–13485):

```

13417     double center_sample = boundary_offset + 0.5 * (double)samples_per_symbol;
13419     if (center_sample < (double)run_start) {
13421         double steps = ceil(((double)run_start - center_sample) / (double)samples_per_symbol);
13422         center_sample += steps * (double)samples_per_symbol;
13424     }
13426     int half_window = samples_per_symbol / 4;
13428     if (half_window < 2) {
13430         half_window = 2;
13432     }
13434     while (center_sample < (double)run_end && symbol_count < frequency_count) {
13435         long center_index = lround(center_sample);
13436         long first_start = center_index - half_window;
13437         long first_end = center_index;
13438         long second_start = center_index;
13439         long second_end = center_index + half_window;
13441         if (first_start < (long)run_start) {
13443             first_start = (long)run_start;
13445         }
13447         if (second_end > (long)run_end) {
13449             second_end = (long)run_end;
13451         }
13453         double first_sum = 0.0;
13454         double second_sum = 0.0;
13455         size_t first_count = 0U;
13456         size_t second_count = 0U;
13458         for (long sample = first_start; sample < first_end; sample++) {
13460             if (sample >= 0 && valid[(size_t)sample]) {
13462                 first_sum += frequency_hz[(size_t)sample];
13463                 first_count++;
13465             }
13466         }
13468         for (long sample = second_start; sample < second_end; sample++) {
13470             if (sample >= 0 && valid[(size_t)sample]) {
13472                 second_sum += frequency_hz[(size_t)sample];
13473                 second_count++;
13475             }
13476         }
13478         if (first_count > 0U && second_count > 0U) {
13480             double first_average = first_sum / (double)first_count;
13481             double second_average = second_sum / (double)second_count;
13483             symbol_values[symbol_count] = (first_sum + second_sum) / (double)(first_count +
second_count);
13484             symbol_transitions[symbol_count] = second_average - first_average;
13485             symbol_count++;

```

The symbol center is placed half a symbol after the estimated boundary. A window on each side of the center produces two frequency means, $\bar{f}_1$ and $\bar{f}_2$. The plotted symbol state is

$$f_s = \frac{\sum f_{\text{first}} + \sum f_{\text{second}}}{N_1 + N_2}, \quad (142)$$

and the transition coordinate is

$$\Delta f_s = \bar{f}_2 - \bar{f}_1. \quad (143)$$

Thus the FSK constellation uses frequency state on one axis and within-symbol frequency change on the other.

13.6 Two-state versus four-state selection

Relevant code (AnalysisWorkstation.c lines 13523–13582):

```

13523     double centroids_two[4] = {0.0, 0.0, 0.0, 0.0};
13524     double centroids_four[4] = {0.0, 0.0, 0.0, 0.0};
13525     size_t populations_two[4] = {0U, 0U, 0U, 0U};
13526     size_t populations_four[4] = {0U, 0U, 0U, 0U};
13527     double error_two =

```

```

13528     ANALYSIS_constellation_fsk_kmeans_1d(symbol_values, symbol_count, 2, centroids_two, populations_two,
13529     NULL);
13529     double error_four =
13530     ANALYSIS_constellation_fsk_kmeans_1d(symbol_values, symbol_count, 4, centroids_four,
13531     populations_four, NULL);
13531     int level_count = 2;
13532     double *centroids = centroids_two;
13533     if (error_two >= 0.0 && error_four >= 0.0) {
13534         double gaps[3] = {centroids_four[1] - centroids_four[0], centroids_four[2] - centroids_four[1],
13535         centroids_four[3] - centroids_four[2]};
13536         double minimum_gap = gaps[0];
13537         double maximum_gap = gaps[0];
13538         for (int gap = 1; gap < 3; gap++) {
13539             if (gaps[gap] < minimum_gap) {
13540                 minimum_gap = gaps[gap];
13541             }
13542             if (gaps[gap] > maximum_gap) {
13543                 maximum_gap = gaps[gap];
13544             }
13545         }
13546         size_t minimum_population = symbol_count / 50U;
13547         if (minimum_population < 4U) {
13548             minimum_population = 4U;
13549         }
13550         int populations_valid = 1;
13551         for (int level = 0; level < 4; level++) {
13552             if (populations_four[level] < minimum_population) {
13553                 populations_valid = 0;
13554             }
13555         }
13556         double gap_uniformity = maximum_gap > 0.0 ? minimum_gap / maximum_gap : 0.0;
13557         if (populations_valid && minimum_gap > 1e-9 && gap_uniformity >= 0.35 && error_four < error_two *
13558         0.45) {
13559             level_count = 4;
13560             centroids = centroids_four;
13561         }

```

The symbol frequency values are fit with both $K = 2$ and $K = 4$ robust K-means. Four-level FSK is accepted only when all four populations are sufficiently occupied, adjacent centroid gaps are reasonably uniform, and

$$E_4 < 0.45E_2. \quad (144)$$

This prevents a four-cluster model from being chosen simply because additional centroids always reduce unconstrained fitting error.

13.7 FSK plot normalization

Relevant code (AnalysisWorkstation.c lines 13600–13709):

```

13600     double plot_center = 0.5 * (centroids[0] + centroids[level_count - 1]);
13601     double half_span = 0.5 * (centroids[level_count - 1] - centroids[0]);
13602     double plot_scale = half_span / 0.82;
13603     double outer_span = centroids[level_count - 1] - centroids[0];
13604     double minimum_level_gap = outer_span;
13605     for (int level = 0; level < level_count - 1; level++) {
13606         double gap = centroids[level + 1] - centroids[level];
13607         if (gap < minimum_level_gap) {
13608             minimum_level_gap = gap;
13609         }
13610     }
13611     if (!isfinite(plot_scale) || plot_scale <= 1e-9 || minimum_level_gap <= 1e-9) {
13612         free(symbol_values);
13613         free(symbol_transitions);
13614         free(transition_samples);
13615         free(transition_weights);
13616         free(frequency_hz);
13617         free(smoothed_hz);

```

```

13624     free(transition_strength);
13625     free(valid);
13626     return;
13627 }
13628
13629 size_t output_stride = symbol_count > ANALYSIS_MAX_CONST_POINTS ? symbol_count /
ANALYSIS_MAX_CONST_POINTS : 1U;
13632 if (output_stride < 1U) {
13633     output_stride = 1U;
13634 }
13635
13636 int output_count = 0;
13637 for (size_t symbol = 0U; symbol < symbol_count && output_count < ANALYSIS_MAX_CONST_POINTS; symbol++) {
13638     if ((symbol % output_stride) != 0U) {
13639         continue;
13640     }
13641     double value = symbol_values[symbol];
13642     int nearest = 0;
13643     double nearest_distance = fabs(value - centroids[0]);
13644     for (int level = 1; level < level_count; level++) {
13645         double distance = fabs(value - centroids[level]);
13646         if (distance < nearest_distance) {
13647             nearest = level;
13648             nearest_distance = distance;
13649         }
13650     }
13651     double local_gap = minimum_level_gap;
13652     if (nearest > 0) {
13653         double gap = centroids[nearest] - centroids[nearest - 1];
13654         if (gap < local_gap) {
13655             local_gap = gap;
13656         }
13657     }
13658     if (nearest + 1 < level_count) {
13659         double gap = centroids[nearest + 1] - centroids[nearest];
13660         if (gap < local_gap) {
13661             local_gap = gap;
13662         }
13663     }
13664     if (nearest_distance > 0.45 * local_gap) {
13665         continue;
13666     }
13667     double normalized_state = (value - plot_center) / plot_scale;
13668     double normalized_transition = symbol_transitions[symbol] / outer_span;
13669     if (!isfinite(normalized_state) || !isfinite(normalized_transition) || fabs(normalized_transition) >
0.60) {
13670         continue;
13671     }
13672     Global_Analysis_Const_I[output_count] = (float)ANALYSIS_limit_double(normalized_state, -0.98, 0.98);
13673     Global_Analysis_Const_Q[output_count] = (float)ANALYSIS_limit_double(normalized_transition, -0.55,
0.55);
13674     output_count++;
13675 }
13676 Global_Analysis_Const_Count = output_count;

```

With outer centroids $c_{\min}$ and $c_{\max}$,

$$f_c = \frac{c_{\min} + c_{\max}}{2}, \quad (145)$$

$$h = \frac{c_{\max} - c_{\min}}{2}, \quad (146)$$

$$s = h/0.82. \quad (147)$$

The displayed state is

$$I_{\text{plot}} = \frac{f_s - f_c}{s}, \quad (148)$$

and transition is normalized by the full outer-state span. Points too far from their nearest centroid or with excessive transition coordinate are discarded before clipping to the display bounds.

14 Robust FSK K-Means

14.1 Percentile-bounded initialization

Relevant code (AnalysisWorkstation.c lines 12900–12916):

```

12900    qsort(sorted, finite_count, sizeof(double), ANALYSIS_constellation_double_compare);
12901    size_t low_index = (size_t)(0.01 * (double)(finite_count - 1U));
12902    size_t high_index = (size_t)(0.99 * (double)(finite_count - 1U));
12903    double robust_low = sorted[low_index];
12904    double robust_high = sorted[high_index];
12905    if (!isfinite(robust_low) || !isfinite(robust_high) || robust_high <= robust_low) {
12906        free(sorted);
12907        return -1.0;
12908    }
12909    for (int level = 0; level < level_count; level++) {
12910        centroids[level] = robust_low + (robust_high - robust_low) * (double)level / (double)(level_count -
129111);
12912    }
12913    }

```

The 1st and 99th percentiles define robust bounds $f_{0.01}$ and $f_{0.99}$. Initial centroids are uniformly spaced between them:

$$c_k = f_{0.01} + \frac{k}{K-1}(f_{0.99} - f_{0.01}). \quad (149)$$

Values outside the robust interval are excluded from the fit.

14.2 Centroid updates and convergence

Relevant code (AnalysisWorkstation.c lines 12918–12982):

```

12918    for (int iteration = 0; iteration < 48; iteration++) {
12919        double sums[4] = {0.0, 0.0, 0.0, 0.0};
12920        size_t counts[4] = {0U, 0U, 0U, 0U};
12921        for (size_t n = 0U; n < count; n++) {
12922            double value = values[n];
12923            if (!isfinite(value) || value < robust_low || value > robust_high) {
12924                continue;
12925            }
12926            int nearest = 0;
12927            double nearest_distance = fabs(value - centroids[0]);
12928            for (int level = 1; level < level_count; level++) {
12929                double distance = fabs(value - centroids[level]);
12930                if (distance < nearest_distance) {
12931                    nearest = level;
12932                    nearest_distance = distance;
12933                }
12934            }
12935            sums[nearest] += value;
12936            counts[nearest]++;
12937        }
12938        double movement = 0.0;
12939        for (int level = 0; level < level_count; level++) {
12940            if (counts[level] == 0U) {
12941                continue;
12942            }
12943            double next = sums[level] / (double)counts[level];
12944            movement += fabs(next - centroids[level]);
12945            centroids[level] = next;
12946        }
12947        for (int left = 0; left < level_count - 1; left++) {
12948            for (int right = left + 1; right < level_count; right++) {
12949                if (centroids[right] < centroids[left]) {
12950                    double swap = centroids[left];
12951                    centroids[left] = centroids[right];
12952                    centroids[right] = swap;
12953                }
12954            }
12955        }
12956    }
12957    }

```

```

12975     }
12977     if (movement < 1e-6) {
12979         break;
12981     }
12982 }

```

Nearest-centroid assignment and mean updates implement standard 1-D K-means. Centroids are explicitly sorted after each iteration. Convergence is declared when total centroid movement falls below 10^{-6} .

14.3 Mean squared fitting error

Relevant code (AnalysisWorkstation.c lines 12988–13039):

```

12988     double squared_error = 0.0;
12989     size_t used = 0U;
12991     for (size_t n = 0U; n < count; n++) {
12992         double value = values[n];
12994         if (!isfinite(value) || value < robust_low || value > robust_high) {
12996             if (assignments) {
12998                 assignments[n] = -1;
13000             }
13002             continue;
13004         }
13006         int nearest = 0;
13007         double nearest_distance = fabs(value - centroids[0]);
13009         for (int level = 1; level < level_count; level++) {
13010             double distance = fabs(value - centroids[level]);
13012             if (distance < nearest_distance) {
13014                 nearest = level;
13015                 nearest_distance = distance;
13017             }
13018         }
13020         if (assignments) {
13022             assignments[n] = nearest;
13024         }
13026         populations[nearest]++;
13027         squared_error += nearest_distance * nearest_distance;
13028         used++;
13029     }
13031     free(sorted);
13033     if (used == 0U) {
13035         return -1.0;
13037     }
13039     return squared_error / (double)used;

```

For all retained values,

$$E = \frac{1}{N_u} \sum_i \min_k (f_i - c_k)^2. \quad (150)$$

Population counts and optional assignments are also produced for the later 2-FSK versus 4-FSK decision.

15 OFDM Cyclic-Prefix Detection and Generic Recovery

15.1 Normalized cyclic-prefix correlation

Relevant code (AnalysisWorkstation.c lines 13728–13755):

```

13728     int symbol_size = fft_size + cp_size;
13729     double sum_i = 0.0;
13730     double sum_q = 0.0;
13731     double power_a = 0.0;
13732     double power_b = 0.0;
13733     int blocks = 0;
13735     for (size_t start = (size_t)offset; start + (size_t)symbol_size <= count && blocks < 48;

```

```

13736     start += (size_t)symbol_size) {
13737         for (int k = 0; k < cp_size; k++) {
13738             size_t a = start + (size_t)k;
13739             size_t b = start + (size_t)fft_size + (size_t)k;
13740             sum_i += i_data[a] * i_data[b] + q_data[a] * q_data[b];
13741             sum_q += q_data[a] * i_data[b] - i_data[a] * q_data[b];
13742             power_a += i_data[a] * i_data[a] + q_data[a] * q_data[a];
13743             power_b += i_data[b] * i_data[b] + q_data[b] * q_data[b];
13744         }
13745         blocks++;
13746     }
13747     if (phase_out) {
13748         *phase_out = atan2(sum_q, sum_i);
13749     }
13750     return power_a > 1e-12 && power_b > 1e-12 ? hypot(sum_i, sum_q) / sqrt(power_a * power_b) : 0.0;
13751 }

```

For FFT size N and cyclic-prefix length L , the prefix is correlated with the corresponding tail of the OFDM symbol:

$$C = \sum_{k=0}^{L-1} x[n+k]x^*[n+N+k]. \quad (151)$$

The normalized score is

$$\rho = \frac{|C|}{\sqrt{(\sum |x[n+k]|^2)(\sum |x[n+N+k]|^2)}}. \quad (152)$$

This is a complex normalized correlation. The phase $\arg C$ also contains fractional carrier-frequency information.

15.2 Search over generic FFT and CP sizes

Relevant code (AnalysisWorkstation.c lines 13763–13816):

```

13763     const int fft_candidates[] = {64, 128, 256, 512, 1024, 2048};
13764     const int cp_divisors[] = {4, 8, 16, 32};
13765     double best_score = 0.0;
13766     double best_phase = 0.0;
13767     int best_fft = 0;
13768     int best_cp = 0;
13769     int best_offset = 0;
13770     for (size_t fft_index = 0; fft_index < sizeof(fft_candidates) / sizeof(fft_candidates[0]); fft_index++) {
13771         int fft_size = fft_candidates[fft_index];
13772         if ((size_t)(fft_size * 6) > count) {
13773             continue;
13774         }
13775         for (size_t cp_index = 0; cp_index < sizeof(cp_divisors) / sizeof(cp_divisors[0]); cp_index++) {
13776             int cp_size = fft_size / cp_divisors[cp_index];
13777             int symbol_size = fft_size + cp_size;
13778             int offset_step = symbol_size / 48;
13779             if (offset_step < 1) {
13780                 offset_step = 1;
13781             }
13782             for (int offset = 0; offset < symbol_size; offset += offset_step) {
13783                 double phase = 0.0;
13784                 double score =
13785                     ANALYSIS_constellation_ofdm_cp_score(i_data, q_data, count, fft_size, cp_size, offset,
13786                     &phase);
13787                 if (score > best_score) {
13788                     best_score = score;
13789                     best_phase = phase;
13790                     best_fft = fft_size;
13791                     best_cp = cp_size;
13792                     best_offset = offset;
13793                 }
13794             }
13795         }
13796     }

```

```

13807     }
13809     if (best_fft == 0 || best_score < 0.08) {
13811         return;
13813     }
13815     double frequency = -best_phase / (double)best_fft;
13816     ANALYSIS_constellation_apply_frequency_correction(i_data, q_data, count, frequency);

```

The generic path tests FFT sizes 64, 128, 256, 512, 1024, 2048 and CP ratios 1/4, 1/8, 1/16, 1/32. Candidate offsets are sampled across one full symbol. The highest cyclic-prefix score wins.

If ϕ_{CP} is the best aggregate CP correlation phase, the implemented fractional frequency correction is

$$\hat{\omega} = -\frac{\phi_{CP}}{N}. \quad (153)$$

This follows because the two correlated CP copies are separated by N samples, so a carrier offset contributes phase ωN between them.

15.3 OFDM symbol FFT and active-carrier threshold

Relevant code (AnalysisWorkstation.c lines 13873–13931):

```

13873     for (int symbol = 0; symbol < symbol_count; symbol++) {
13874         size_t start = (size_t)best_offset + (size_t)symbol * (size_t)symbol_size + (size_t)best_cp;
13876         if (start + (size_t)best_fft > count) {
13878             symbol_count = symbol;
13879             break;
13881         }
13883         for (int n = 0; n < best_fft; n++) {
13884             time_data[n][0] = i_data[start + (size_t)n];
13885             time_data[n][1] = q_data[start + (size_t)n];
13886         }
13888         fftw_execute(plan);
13890         for (int bin = 0; bin < best_fft; bin++) {
13891             double point_i = frequency_data[bin][0] / (double)best_fft;
13892             double point_q = frequency_data[bin][1] / (double)best_fft;
13893             symbols[(size_t)symbol * (size_t)best_fft + (size_t)bin].i = point_i;
13894             symbols[(size_t)symbol * (size_t)best_fft + (size_t)bin].q = point_q;
13895             average_power[bin] += point_i * point_i + point_q * point_q;
13896         }
13897     }
13899     if (symbol_count < 3) {
13901         fftw_destroy_plan(plan);
13902         fftw_free(time_data);
13903         fftw_free(frequency_data);
13904         free(average_power);
13905         free(sorted_power);
13906         free(symbols);
13907         return;
13909     }
13911     for (int bin = 0; bin < best_fft; bin++) {
13912         average_power[bin] /= (double)symbol_count;
13913         sorted_power[bin] = average_power[bin];
13914     }
13916     qsort(sorted_power, (size_t)best_fft, sizeof(double), ANALYSIS_constellation_double_compare);
13917     double maximum_power = sorted_power[best_fft - 1];
13918     double noise_power = sorted_power[best_fft / 10];
13919     double threshold = noise_power * 5.0;
13921     if (threshold < maximum_power * 0.0001) {
13923         threshold = maximum_power * 0.0001;
13925     }
13927     if (threshold > maximum_power * 0.50) {
13929         threshold = maximum_power * 0.50;
13931     }

```

After removing the CP, each symbol is transformed with an N -point FFT and scaled by $1/N$. Average carrier power is

$$\bar{P}[k] = \frac{1}{N_s} \sum_s |X_s[k]|^2. \quad (154)$$

A low sorted-power percentile estimates the noise floor. The active-carrier threshold begins at five times that noise estimate and is clamped between 10^{-4} and 0.5 of the maximum carrier power.

15.4 Per-carrier phase order, rotation, and RMS scaling

Relevant code (AnalysisWorkstation.c lines 13952–13999):

```

13952     for (int symbol = 0; symbol < symbol_count; symbol++) {
13953         Type_Analysis_Constellation_Point point = symbols[(size_t)symbol * (size_t)best_fft +
(size_t)bin];
13954         double magnitude = hypot(point.i, point.q);
13955         if (magnitude > 1e-12) {
13956             double powered_i = 0.0;
13957             double powered_q = 0.0;
13958             double unit_i = point.i / magnitude;
13959             double unit_q = point.q / magnitude;
13960             ANALYSIS_constellation_complex_power(unit_i, unit_q, 2, &powered_i, &powered_q);
13961             second_i += powered_i;
13962             second_q += powered_q;
13963             ANALYSIS_constellation_complex_power(unit_i, unit_q, 4, &powered_i, &powered_q);
13964             fourth_i += powered_i;
13965             fourth_q += powered_q;
13966             phase_samples++;
13967         }
13968         bin_power += point.i * point.i + point.q * point.q;
13969     }
13970     double second_coherence = phase_samples > 0 ? hypot(second_i, second_q) / (double)phase_samples : 0.0;
13971     double fourth_coherence = phase_samples > 0 ? hypot(fourth_i, fourth_q) / (double)phase_samples : 0.0;
13972     int order = second_coherence >= 0.90 * fourth_coherence ? 2 : 4;
13973     double phase = order == 2 ? 0.5 * atan2(second_q, second_i) : 0.25 * atan2(fourth_q, fourth_i);
13974     double rotation_i = cos(-phase);
13975     double rotation_q = sin(-phase);
13976     double bin_rms = sqrt(bin_power / (double)symbol_count);
13977     if (bin_rms < 1e-12) {
13978         continue;
13979     }
13980     for (int symbol = 0; symbol < symbol_count && output_count < ANALYSIS_MAX_CONST_POINTS; symbol++) {
13981         Type_Analysis_Constellation_Point point = symbols[(size_t)symbol * (size_t)best_fft +
(size_t)bin];
13982         Global_Analysis_Const_I[output_count] = (float)((point.i * rotation_i - point.q * rotation_q) /
bin_rms);
13983         Global_Analysis_Const_Q[output_count] = (float)((point.i * rotation_q + point.q * rotation_i) /
bin_rms);
13984         output_count++;
13985     }
13986     Global_Analysis_Const_Count = output_count;
13987     ANALYSIS_constellation_normalize_output(0);

```

For each active carrier, both second- and fourth-order phase coherences are computed across OFDM symbols. The carrier is treated as effectively order 2 when

$$C_2 \geq 0.90C_4, \quad (155)$$

otherwise order 4 is used. The orientation estimate is

$$\theta = \begin{cases} \frac{1}{2} \arg \sum (z/|z|)^2, & M = 2, \\ \frac{1}{4} \arg \sum (z/|z|)^4, & M = 4. \end{cases} \quad (156)$$

Each carrier's points are rotated by $e^{-j\theta}$ and divided by

$$r_{\text{RMS},k} = \sqrt{\frac{1}{N_s} \sum_s |X_s[k]|^2}. \quad (157)$$

16 Known 128-Point QPSK OFDM Recovery

The source contains a specialized recovery path for a known 128-point, 32-sample cyclic-prefix QPSK OFDM waveform. If it fails its confidence tests, the generic OFDM path described above is used.

16.1 Sliding CP metric

Relevant code (AnalysisWorkstation.c lines 14024–14063):

```

14024 double sum_i = 0.0;
14025 double sum_q = 0.0;
14026 double power_a = 0.0;
14027 double power_b = 0.0;
14029 for (int k = 0; k < cp_size; k++) {
14030     size_t a = (size_t)k;
14031     size_t b = (size_t)fft_size + (size_t)k;
14032     sum_i += i_data[a] * i_data[b] + q_data[a] * q_data[b];
14033     sum_q += q_data[a] * i_data[b] - i_data[a] * q_data[b];
14034     power_a += i_data[a] * i_data[a] + q_data[a] * q_data[a];
14035     power_b += i_data[b] * i_data[b] + q_data[b] * q_data[b];
14036 }
14038 for (size_t start = 0; start < metric_count; start++) {
14039     corr_i[start] = sum_i;
14040     corr_q[start] = sum_q;
14041     metrics[start] = power_a > 1e-15 && power_b > 1e-15 ? hypot(sum_i, sum_q) / sqrt(power_a * power_b) :
0.0;
14043     if (start + 1U >= metric_count) {
14044         break;
14045     }
14047     size_t remove_a = start;
14049     size_t remove_b = start + (size_t)fft_size;
14050     size_t add_a = start + (size_t)cp_size;
14051     size_t add_b = add_a + (size_t)fft_size;
14052     sum_i -= i_data[remove_a] * i_data[remove_b] + q_data[remove_a] * q_data[remove_b];
14053     sum_q -= q_data[remove_a] * i_data[remove_b] - i_data[remove_a] * q_data[remove_b];
14054     power_a -= i_data[remove_a] * i_data[remove_a] + q_data[remove_a] * q_data[remove_a];
14055     power_b -= i_data[remove_b] * i_data[remove_b] + q_data[remove_b] * q_data[remove_b];
14056     sum_i += i_data[add_a] * i_data[add_b] + q_data[add_a] * q_data[add_b];
14057     sum_q += q_data[add_a] * i_data[add_b] - i_data[add_a] * q_data[add_b];
14058     power_a += i_data[add_a] * i_data[add_a] + q_data[add_a] * q_data[add_a];
14059     power_b += i_data[add_b] * i_data[add_b] + q_data[add_b] * q_data[add_b];
14060 }
14063

```

The same normalized CP correlation is evaluated at every possible start position. Rather than recomputing all L products for every shift, the code updates the correlation and two powers with a sliding-window recurrence: subtract the leaving pair and add the entering pair. This reduces the per-position work from $O(L)$ to $O(1)$ after initialization.

16.2 Fixed waveform parameters

Relevant code (AnalysisWorkstation.c lines 14122–14132):

```

14122 const int fft_size = 128;
14123 const int cp_size = 32;
14124 const int symbol_size = fft_size + cp_size;
14125 const int logical_carriers[48] = {-24, -23, -22, -21, -20, -19, -18, -17, -16, -15, -14, -13, -12, -11,
-10, -9,

```

```

14126             -8, -7, -6, -5, -4, -3, -2, -1, 1, 2, 3, 4, 5, 6,
14127             7, 8,
14127             9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22,
14128             23, 24};
14128     const int data_carriers[44] = {-24, -23, -22, -20, -19, -18, -17, -16, -15, -14, -13, -12, -11, -10, -9,
14129             -8, -6, -5, -4, -3, -2, -1, 1, 2, 3, 4, 5, 6, 8, 9,
14130             10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 22, 23, 24};
14131     const int pilot_carriers[4] = {-21, -7, 7, 21};
14132     const double pilot_values[4] = {1.0, 1.0, 1.0, -1.0};

```

The known waveform uses

$$N = 128, \quad L = 32, \quad N_{\text{sym}} = 160. \quad (158)$$

Forty-eight logical occupied carriers are defined, of which 44 are data carriers and four are pilots at $-21, -7, +7, +21$. Their expected pilot signs are $(+1, +1, +1, -1)$.

16.3 Periodic timing-run score

Relevant code (AnalysisWorkstation.c lines 14202–14259):

```

14202     for (int phase = 0; phase < symbol_size; phase++) {
14203         size_t run_start = 0U;
14204         int run_length = 0;
14205         double run_sum = 0.0;
14207         for (size_t position = (size_t)phase; position < metric_count; position += (size_t)symbol_size) {
14209             if (metrics[position] >= timing_threshold) {
14211                 if (run_length == 0) {
14213                     run_start = position;
14215                 }
14217                 run_length++;
14218                 run_sum += metrics[position];
14220             }
14222             else {
14224                 if (run_length >= 6) {
14226                     double average = run_sum / (double)run_length;
14227                     double score = (double)run_length * average * average;
14229                     if (score > best_run_score ||
14230                         (fabs(score - best_run_score) <= 1e-12 && run_start < best_run_start)) {
14232                         best_run_score = score;
14233                         best_run_start = run_start;
14234                         best_run_length = run_length;
14236                     }
14238                 }
14240                 run_length = 0;
14241                 run_sum = 0.0;
14243             }
14244         }
14246         if (run_length >= 6) {
14248             double average = run_sum / (double)run_length;
14249             double score = (double)run_length * average * average;
14251             if (score > best_run_score || (fabs(score - best_run_score) <= 1e-12 && run_start <
14253                 best_run_start)) {
14254                 best_run_score = score;
14255                 best_run_start = run_start;
14256                 best_run_length = run_length;
14257             }
14259         }

```

For each of the 160 possible symbol phases, CP metric values are examined every 160 samples. Contiguous runs above threshold are scored by

$$S_{\text{run}} = L_{\text{run}} \bar{\rho}^2, \quad (159)$$

where L_{run} is run length and $\bar{\rho}$ is the run's mean CP metric. A run must contain at least six symbols. This favors long, consistently correlated symbol sequences.

16.4 Fractional carrier offset from CP phase

Relevant code (AnalysisWorkstation.c lines 14302–14322):

```

14302  /* CP phase gives the fractional carrier offset modulo one subcarrier spacing. */
14303  double aggregate_i = 0.0;
14304  double aggregate_q = 0.0;
14306  for (int symbol = 0; symbol < symbol_count; symbol++) {
14307      aggregate_i += corr_i[starts[symbol]];
14308      aggregate_q += corr_q[starts[symbol]];
14309  }
14311  if (hypot(aggregate_i, aggregate_q) <= 1e-15) {
14313      free(starts);
14314      free(metrics);
14315      free(corr_i);
14316      free(corr_q);
14317      return 0;
14319  }
14321  double frequency = -atan2(aggregate_q, aggregate_i) / (double)fft_size;
14322  ANALYSIS_constellation_apply_frequency_correction(i_data, q_data, count, frequency);

```

CP correlations at all selected symbol starts are coherently summed:

$$C_{\text{agg}} = \sum_s C_s. \quad (160)$$

The correction is

$$\hat{\omega} = -\frac{\arg C_{\text{agg}}}{128}. \quad (161)$$

The complete IQ sequence is rotated by this angular increment before the OFDM symbol FFTs are evaluated.

16.5 Integer subcarrier displacement from occupied/guard contrast

Relevant code (AnalysisWorkstation.c lines 14424–14489):

```

14424  int best_shift = 0;
14425  double best_shift_score = -1.0;
14426  double best_shift_power = 0.0;
14427  double average_total_power = total_power / (double)fft_size;
14429  for (int shift = -fft_size / 4; shift <= fft_size / 4; shift++) {
14430      double active_power = 0.0;
14431      double guard_power = 0.0;
14433      for (int carrier = 0; carrier < 48; carrier++) {
14434          int bin = (logical_carriers[carrier] + shift) % fft_size;
14436          if (bin < 0) {
14438              bin += fft_size;
14440          }
14442          active_power += average_power[bin];
14443      }
14445      for (int carrier = -32; carrier <= -25; carrier++) {
14446          int bin = (carrier + shift) % fft_size;
14448          if (bin < 0) {
14450              bin += fft_size;
14452          }
14454          guard_power += average_power[bin];
14455      }
14457      for (int carrier = 25; carrier <= 32; carrier++) {
14458          int bin = (carrier + shift) % fft_size;
14460          if (bin < 0) {
14462              bin += fft_size;
14464          }
14466          guard_power += average_power[bin];
14467      }
14469      int dc_bin = shift % fft_size;
14471      if (dc_bin < 0) {

```



```

14473         dc_bin += fft_size;
14475     }
14477     double active_mean = active_power / 48.0;
14478     double guard_mean = guard_power / 16.0;
14479     double denominator = guard_mean + 0.25 * average_power[dc_bin] + 0.05 * average_total_power + 1e-15;
14480     double score = active_mean / denominator;
14482     if (score > best_shift_score || (fabs(score - best_shift_score) <= 1e-12 && active_power >
best_shift_power)) {
14484         best_shift_score = score;
14485         best_shift_power = active_power;
14486         best_shift = shift;
14488     }
14489 }

```

For each integer shift, average power is summed across the expected 48 occupied carriers and 16 edge guard carriers. The score is

$$S_{\text{shift}} = \frac{\bar{P}_{\text{active}}}{\bar{P}_{\text{guard}} + 0.25\bar{P}_{\text{DC}} + 0.05\bar{P}_{\text{all}} + \epsilon}. \quad (162)$$

This uses both occupied-band energy and empty-edge contrast, reducing ambiguity that would occur if only total active-bin power were compared.

16.6 Pilot common-phase and linear phase-slope correction

Relevant code (AnalysisWorkstation.c lines 14586–14690):

```

14586     for (int pass = 0; pass < 2; pass++) {
14587         for (int symbol = 0; symbol < symbol_count; symbol++) {
14588             double error_i[4] = {0.0};
14589             double error_q[4] = {0.0};
14590             int valid_pilots = 0;
14592             for (int pilot = 0; pilot < 4; pilot++) {
14593                 int bin = (pilot_carriers[pilot] + best_shift) % fft_size;
14595                 if (bin < 0) {
14597                     bin += fft_size;
14599                 }
14601                 Type_Analysis_Constellation_Point point = symbols[(size_t)symbol * (size_t)fft_size +
(size_t)bin];
14602                 double point_i = point.i * pilot_values[pilot];
14603                 double point_q = point.q * pilot_values[pilot];
14604                 double magnitude = hypot(point_i, point_q);
14606                 if (magnitude <= 1e-12) {
14608                     continue;
14610                 }
14612                 point_i /= magnitude;
14613                 point_q /= magnitude;
14615                 error_i[pilot] = point_i * reference_i[pilot] + point_q * reference_q[pilot];
14616                 error_q[pilot] = point_q * reference_i[pilot] - point_i * reference_q[pilot];
14617                 valid_pilots++;
14618             }
14620             if (valid_pilots < 3) {
14622                 continue;
14624             }
14626             double best_slope = 0.0;
14627             double best_coherence = -1.0;
14628             double best_sum_i = 0.0;
14629             double best_sum_q = 0.0;
14631             for (int slope_step = -100; slope_step <= 100; slope_step++) {
14632                 double slope = (double)slope_step * 0.0025;
14633                 double sum_i = 0.0;
14634                 double sum_q = 0.0;
14636                 for (int pilot = 0; pilot < 4; pilot++) {
14638                     if (hypot(error_i[pilot], error_q[pilot]) <= 1e-12) {
14640                         continue;
14642                     }
14644                     double angle = -slope * (double)pilot_carriers[pilot];

```

```

14645         double rotation_i = cos(angle);
14646         double rotation_q = sin(angle);
14647         sum_i += error_i[pilot] * rotation_i - error_q[pilot] * rotation_q;
14648         sum_q += error_i[pilot] * rotation_q + error_q[pilot] * rotation_i;
14649     }
14651     double coherence = hypot(sum_i, sum_q);
14653     if (coherence > best_coherence) {
14655         best_coherence = coherence;
14656         best_slope = slope;
14657         best_sum_i = sum_i;
14658         best_sum_q = sum_q;
14660     }
14661 }
14663 if (best_coherence < 0.40 * (double)valid_pilots) {
14665     continue;
14667 }
14669 double common_phase = atan2(best_sum_q, best_sum_i);
14671 for (int bin = 0; bin < fft_size; bin++) {
14672     int carrier = bin - best_shift;
14674     while (carrier > fft_size / 2) {
14675         carrier -= fft_size;
14676     }
14678     while (carrier < -fft_size / 2) {
14679         carrier += fft_size;
14680     }
14682     double phase = common_phase + best_slope * (double)carrier;
14683     double rotation_i = cos(-phase);
14684     double rotation_q = sin(-phase);
14685     Type_Analysis_Constellation_Point *point = &symbols[(size_t)symbol * (size_t)fft_size +
(size_t)bin];
14686     double corrected_i = point->i * rotation_i - point->q * rotation_q;
14687     double corrected_q = point->i * rotation_q + point->q * rotation_i;
14688     point->i = corrected_i;
14689     point->q = corrected_q;
14690 }

```

For each symbol, the four pilot errors are normalized against a reference pilot phasor. A candidate linear phase slope b is searched on a grid. For pilot carrier index k_p , the candidate de-slope is

$$e^{-jbk_p}. \quad (163)$$

The slope maximizing coherent pilot-sum magnitude is selected. Common phase is then

$$\phi_c = \arg \sum_p e_p e^{-jbk_p}. \quad (164)$$

Every subcarrier k in that OFDM symbol is corrected by

$$X'_s[k] = X_s[k] e^{-j(\phi_c + bk)}. \quad (165)$$

The common term corrects symbol-wide phase error; the linear term corrects phase slope across frequency caused by residual timing/sampling displacement.

16.7 Per-data-carrier QPSK orientation and RMS normalization

Relevant code (AnalysisWorkstation.c lines 14732–14801):

```

14732     double bin_phase[44] = {0.0};
14733     double bin_rms[44] = {0.0};
14735     for (int carrier_index = 0; carrier_index < 44; carrier_index++) {
14736         int bin = (data_carriers[carrier_index] + best_shift) % fft_size;
14738         if (bin < 0) {
14740             bin += fft_size;
14742         }
14744         double fourth_i = 0.0;

```

```

14745     double fourth_q = 0.0;
14746     double power = 0.0;
14748     for (int symbol = 0; symbol < symbol_count; symbol++) {
14749         Type_Analysis_Constellation_Point point = symbols[(size_t)symbol * (size_t)fft_size +
14750             (size_t)bin];
14751         double magnitude = hypot(point.i, point.q);
14752         if (magnitude > 1e-12) {
14753             double powered_i = 0.0;
14754             double powered_q = 0.0;
14755             ANALYSIS_constellation_complex_power(point.i / magnitude, point.q / magnitude, 4, &powered_i,
14756                 &powered_q);
14757             fourth_i += powered_i;
14758             fourth_q += powered_q;
14759         }
14760         power += point.i * point.i + point.q * point.q;
14761     }
14762     bin_phase[carrier_index] = 0.25 * atan2(fourth_q, fourth_i);
14763     bin_rms[carrier_index] = sqrt(power / (double)symbol_count);
14764     if (bin_rms[carrier_index] < 1e-12) {
14765         bin_rms[carrier_index] = 1.0;
14766     }
14767 }
14768 int output_count = 0;
14769 for (int symbol = 0; symbol < symbol_count && output_count < ANALYSIS_MAX_CONST_POINTS; symbol++) {
14770     for (int carrier_index = 0; carrier_index < 44 && output_count < ANALYSIS_MAX_CONST_POINTS;
14771         carrier_index++) {
14772         int bin = (data_carriers[carrier_index] + best_shift) % fft_size;
14773         if (bin < 0) {
14774             bin += fft_size;
14775         }
14776         Type_Analysis_Constellation_Point point = symbols[(size_t)symbol * (size_t)fft_size +
14777             (size_t)bin];
14778         double rotation_i = cos(-bin_phase[carrier_index]);
14779         double rotation_q = sin(-bin_phase[carrier_index]);
14780         double corrected_i = (point.i * rotation_i - point.q * rotation_q) / bin_rms[carrier_index];
14781         double corrected_q = (point.i * rotation_q + point.q * rotation_i) / bin_rms[carrier_index];
14782         Global_Analysis_Const_I[output_count] = (float)corrected_i;
14783         Global_Analysis_Const_Q[output_count] = (float)corrected_q;
14784         output_count++;
14785     }
14786 }
14787 Global_Analysis_Const_Count = output_count;
14788 ANALYSIS_constellation_normalize_output(0);

```

For each data carrier, the fourth-order phase estimate is

$$\theta_k = \frac{1}{4} \arg \sum_s \left(\frac{X_s[k]}{|X_s[k]|} \right)^4. \quad (166)$$

The per-carrier RMS is

$$r_k = \sqrt{\frac{1}{N_s} \sum_s |X_s[k]|^2}. \quad (167)$$

The final point sent to the display is

$$Z_s[k] = \frac{X_s[k] e^{-j\theta_k}}{r_k}. \quad (168)$$

These points are then passed through the common robust constellation display normalizer.

17 Additional Defined DSP Helpers

Several mathematically relevant static helpers are present in the source but are not called by the current active constellation-building path. They are retained here because they are part of `AnalysisWorkstation.c` and may be useful for future recovery logic.

17.1 Original symbol-timing search

Relevant code (AnalysisWorkstation.c lines 11169–11263):

```

11169     double estimated_sps = 1.35 * Global_Analysis_Sample_Rate / effective_bandwidth;
11170     int minimum_sps = (int)floor(estimated_sps * 0.35);
11171     int maximum_sps = (int)ceil(estimated_sps * 8.0);
11173     if (minimum_sps < 2) {
11175         minimum_sps = 2;
11177     }
11179     if (maximum_sps > 1024) {
11181         maximum_sps = 1024;
11183     }
11185     if (maximum_sps <= minimum_sps) {
11187         maximum_sps = minimum_sps + 1;
11189     }
11191     int sps_step = (maximum_sps - minimum_sps) / 96;
11193     if (sps_step < 1) {
11195         sps_step = 1;
11197     }
11199     size_t guard = count / 40U;
11201     if (guard < 128U) {
11203         guard = 128U;
11205     }
11207     if (guard * 2U >= count) {
11209         guard = 0U;
11211     }
11213     double best_score = -1.0;
11214     Type_Analysis_Constellation_Point candidate[384];
11216     for (int sps = minimum_sps; sps <= maximum_sps; sps += sps_step) {
11217         int offset_step = sps / 32;
11219         if (offset_step < 1) {
11221             offset_step = 1;
11223         }
11225         for (int offset = 0; offset < sps; offset += offset_step) {
11226             int candidate_count =
11227                 ANALYSIS_constellation_collect_candidate(i_data, q_data, count, guard, sps, offset,
candidate, 384);
11228             double score = 0.0;
11230             if (family == ANALYSIS_CONSTELLATION_MODE_PSK) {
11232                 score = ANALYSIS_constellation_psk_score(candidate, candidate_count, psk_order);
11234             }
11236             else if (family == ANALYSIS_CONSTELLATION_MODE_QAM) {
11238                 score = ANALYSIS_constellation_qam_score(candidate, candidate_count);
11240             }
11242             else {
11244                 score = ANALYSIS_constellation_ask_score(candidate, candidate_count);
11246             }
11248             double transition_score =
11249                 ANALYSIS_constellation_transition_timing_score(i_data, q_data, count, guard, sps, offset);
11250             transition_score = ANALYSIS_limit_double(transition_score, 0.05, 25.0);
11251             score *= sqrt(transition_score);
11253             if (score > best_score) {
11255                 best_score = score;
11256                 *best_samples_per_symbol = sps;
11257                 *best_offset = offset;
11259             }
11260         }
11261     }
11263     return best_score > 0.0;

```

This earlier search derives an approximate samples-per-symbol range from

$$S_{\text{est}} \approx 1.35 \frac{F_s}{B}, \quad (169)$$

then evaluates family cluster scores over candidate periods and offsets, multiplying each cluster score by the square root of the transition-timing score. The active path now uses the refined folded-transition search instead.

17.2 FFT-based transition-energy symbol-rate estimator

Relevant code (AnalysisWorkstation.c lines 11869–11930):

```

11869     double mean_difference = 0.0;
11871     for (size_t n = 1; n < count; n++) {
11872         double previous_magnitude = hypot(i_data[n - 1U], q_data[n - 1U]);
11873         double current_magnitude = hypot(i_data[n], q_data[n]);
11874         double value = 0.0;
11876         if (previous_magnitude >= magnitude_gate && current_magnitude >= magnitude_gate) {
11878             double delta_i = i_data[n] - i_data[n - 1U];
11879             double delta_q = q_data[n] - q_data[n - 1U];
11880             value = delta_i * delta_i + delta_q * delta_q;
11882         }
11884         difference[n - 1U] = value;
11885         mean_difference += value;
11886     }
11888     mean_difference /= (double)difference_count;
11890     for (size_t n = 0; n < fft_size; n++) {
11892         if (n < difference_count) {
11894             double value = difference[n] - mean_difference;
11896             if (n > 0U && n + 1U < difference_count) {
11898                 value = (difference[n - 1U] + difference[n] + difference[n + 1U]) / 3.0 - mean_difference;
11900             }
11902             double window = difference_count > 1U
11903                 ? 0.5 - 0.5 * cos((2.0 * M_PI * (double)n) / (double)(difference_count - 1U))
11904                 : 1.0;
11905             time_data[n][0] = value * window;
11906             time_data[n][1] = 0.0;
11908         }
11910         else {
11912             time_data[n][0] = 0.0;
11913             time_data[n][1] = 0.0;
11915         }
11916     }
11918     free(difference);
11920     fftw_plan plan = fftw_plan_dft_1d((int)fft_size, time_data, frequency_data, FFTW_FORWARD, FFTW_ESTIMATE);
11922     if (!plan) {
11924         fftw_free(time_data);
11925         fftw_free(frequency_data);
11926         return 0.0;
11928     }
11930     fftw_execute(plan);

```

This helper forms the transition-energy sequence

$$d[n] = |x[n] - x[n - 1]|^2, \quad (170)$$

subtracts its mean, applies a Hann window, and takes an FFT. Repetition at symbol boundaries produces spectral lines at the symbol rate and its harmonics.

Harmonic scoring (lines 11932–12086):

```

11932     double minimum_rate_hz = bandwidth_hz > 0.0 ? bandwidth_hz / 64.0 : Global_Analysis_Sample_Rate / 4096.0;
11933     double maximum_rate_hz = bandwidth_hz > 0.0 ? bandwidth_hz * 8.0 : Global_Analysis_Sample_Rate / 8.0;
11935     if (minimum_rate_hz < 100.0) {
11937         minimum_rate_hz = 100.0;
11939     }
11941     if (maximum_rate_hz > Global_Analysis_Sample_Rate * 0.25) {
11943         maximum_rate_hz = Global_Analysis_Sample_Rate * 0.25;
11945     }
11947     if (maximum_rate_hz <= minimum_rate_hz) {
11949         maximum_rate_hz = minimum_rate_hz * 2.0;
11951     }
11953     Type_Analysis_Constellation_Rate_Peak peaks[24];
11954     int peak_count = 0;
11955     double maximum_peak_power = 0.0;
11957     memset(peaks, 0, sizeof(peaks));
11959     for (size_t bin = 1U; bin + 1U <= fft_size / 2U; bin++) {

```

```
11960     double frequency_hz = (double)bin * Global_Analysis_Sample_Rate / (double)fft_size;
11962     if (frequency_hz < minimum_rate_hz || frequency_hz > maximum_rate_hz) {
11964         continue;
11966     }
11968     double power =
11969         frequency_data[bin][0] * frequency_data[bin][0] + frequency_data[bin][1] * frequency_data[bin][1];
11970     double left_power = frequency_data[bin - 1U][0] * frequency_data[bin - 1U][0] +
11971         frequency_data[bin - 1U][1] * frequency_data[bin - 1U][1];
11972     double right_power = frequency_data[bin + 1U][0] * frequency_data[bin + 1U][0] +
11973         frequency_data[bin + 1U][1] * frequency_data[bin + 1U][1];
11975     if (power <= left_power || power < right_power) {
11977         continue;
11979     }
11981     if (peak_count >= 24 && power <= peaks[23].normalized_power) {
11983         continue;
11985     }
11987     int insertion = peak_count;
11989     if (insertion > 23) {
11991         insertion = 23;
11993     }
11995     while (insertion > 0 && peaks[insertion - 1].normalized_power < power) {
11997         if (insertion < 24) {
11999             peaks[insertion] = peaks[insertion - 1];
12001         }
12002         insertion--;
12003     }
12005     if (insertion < 24) {
12007         peaks[insertion].frequency_hz = frequency_hz;
12008         peaks[insertion].normalized_power = power;
12010         if (peak_count < 24) {
12012             peak_count++;
12014         }
12016     }
12018     if (power > maximum_peak_power) {
12020         maximum_peak_power = power;
12022     }
12023 }
12025 fftw_destroy_plan(plan);
12026 fftw_free(time_data);
12027 fftw_free(frequency_data);
12029 if (peak_count <= 0 || maximum_peak_power <= 0.0) {
12031     return bandwidth_hz > 0.0 ? bandwidth_hz * 0.5 : 0.0;
12033 }
12035 for (int p = 0; p < peak_count; p++) {
12036     peaks[p].normalized_power /= maximum_peak_power;
12037 }
12039 double best_rate_hz = 0.0;
12040 double best_score = -1.0;
12042 for (int source = 0; source < peak_count; source++) {
12043     for (int divisor = 1; divisor <= 12; divisor++) {
12044         double candidate_rate_hz = peaks[source].frequency_hz / (double)divisor;
12045         double samples_per_symbol = Global_Analysis_Sample_Rate / candidate_rate_hz;
12047         if (candidate_rate_hz < 100.0 || samples_per_symbol < 2.0 || samples_per_symbol > 2048.0) {
12049             continue;
12051         }
12053         double score = 0.0;
12055         for (int p = 0; p < peak_count; p++) {
12056             int harmonic = (int)llround(peaks[p].frequency_hz / candidate_rate_hz);
12058             if (harmonic < 1 || harmonic > 16) {
12060                 continue;
12062             }
12064             double relative_error =
12065                 fabs(peaks[p].frequency_hz - (double)harmonic * candidate_rate_hz) / candidate_rate_hz;
12067             if (relative_error < 0.03) {
12069                 score += peaks[p].normalized_power / pow((double)harmonic, 0.35) *
12070                     exp(-pow(relative_error / 0.012, 2.0));
12072             }
12073         }
12075         score *= 1.0 + 0.03 * log(candidate_rate_hz);
12077         if (score > best_score) {
```

```

12079         best_score = score;
12080         best_rate_hz = candidate_rate_hz;
12082     }
12083 }
12084 }
12086 return best_rate_hz;

```

The source detects local spectral maxima over an allowed rate range, then tests candidate fundamental rates formed by dividing peak frequencies by integers 1 through 12. A peak near harmonic h contributes approximately

$$\frac{P_h}{h^{0.35}} \exp \left[- \left(\frac{\epsilon_h}{0.012} \right)^2 \right], \quad (171)$$

where ϵ_h is relative harmonic-frequency error. The highest aggregate harmonic score is returned as the symbol-rate estimate. This helper is currently defined but not called by the active linear-family path.

17.3 Transition-histogram center-offset helper

Relevant code (AnalysisWorkstation.c lines 12119–12199):

```

12119 for (size_t n = start; n < end; n++) {
12120     double delta_i = i_data[n] - i_data[n - 1U];
12121     double delta_q = q_data[n] - q_data[n - 1U];
12122     int phase = (int)(n % (size_t)samples_per_symbol);
12124     histogram[phase] += delta_i * delta_i + delta_q * delta_q;
12125     histogram_count[phase]++;
12126 }
12128 for (int phase = 0; phase < samples_per_symbol; phase++) {
12130     if (histogram_count[phase] > 0) {
12132         histogram[phase] /= (double)histogram_count[phase];
12134     }
12135 }
12137 int smoothing_radius = samples_per_symbol / 100;
12139 if (smoothing_radius < 1) {
12141     smoothing_radius = 1;
12143 }
12145 if (smoothing_radius > 6) {
12147     smoothing_radius = 6;
12149 }
12151 double mean_smoothed = 0.0;
12152 double maximum_smoothed = -1.0;
12153 int boundary_phase = 0;
12155 for (int phase = 0; phase < samples_per_symbol; phase++) {
12156     for (int offset = -smoothing_radius; offset <= smoothing_radius; offset++) {
12157         int index = phase + offset;
12159         while (index < 0) {
12160             index += samples_per_symbol;
12161         }
12163         while (index >= samples_per_symbol) {
12164             index -= samples_per_symbol;
12165         }
12167         smoothed[phase] += histogram[index];
12168     }
12170     mean_smoothed += smoothed[phase];
12172     if (smoothed[phase] > maximum_smoothed) {
12174         maximum_smoothed = smoothed[phase];
12175         boundary_phase = phase;
12177     }
12178 }
12180 mean_smoothed /= (double)samples_per_symbol;
12182 int center_phase = (boundary_phase + samples_per_symbol / 2) % samples_per_symbol;
12183 int guard_phase = (int)(guard % (size_t)samples_per_symbol);
12184 int relative_offset = center_phase - guard_phase;
12186 while (relative_offset < 0) {
12187     relative_offset += samples_per_symbol;
12188 }
12190 while (relative_offset >= samples_per_symbol) {

```

```

12191     relative_offset -= samples_per_symbol;
12192 }
12193 *relative_center_offset = relative_offset;
12194 if (transition_concentration) {
12195     *transition_concentration = maximum_smoothed / (mean_smoothed + 1e-18);

```

For a known samples-per-symbol value S , transition energy is folded by sample phase modulo S . After circular smoothing, the largest phase is treated as a symbol boundary and the center is placed half a symbol away. Transition concentration is

$$C = \frac{H_{\max}}{\bar{H}}. \quad (172)$$

This helper is also currently defined but not called by the active path because the refined timing routine performs the same style of calculation internally while searching S .

18 End-to-End Active DSP Paths

18.1 Spectrogram and graph path

For display column x , the principal operations are

$$n_x = n_0 + \frac{x}{W-1}L, \quad (173)$$

$$x_w[n] = x[n + n_x]w[n], \quad (174)$$

$$X_x[k] = \text{DFT}\{x_w[n]\}, \quad (175)$$

$$D_x[k] = 20 \log_{10} \left(\frac{|X_x[k]|}{N} + 10^{-12} \right) + 100. \quad (176)$$

If a frequency selector is active, an additional branch computes

$$z[n] = x[n]e^{-j2\pi f_{\text{off}}n/F_s}, \quad (177)$$

$$y[n] = y[n-1] + \alpha(z[n] - y[n-1]), \quad (178)$$

$$\phi_x = \arg \sum_n y[n], \quad (179)$$

$$f_x = \text{mean} \left\{ \frac{F_s}{2\pi} \arg(y[n]y^*[n-1]) \right\}. \quad (180)$$

Phase is then unwrapped and linearly detrended; instantaneous frequency is mean-centered.

18.2 Linear constellation path

The active PSK/QAM/ASK chain can be summarized as

complex16 IQ → power-selected input window
 → DC removal
 → selected-center frequency translation
 → FFT-domain low-pass
 → family-specific CFO estimate
 → frequency correction
 → folded transition-energy timing search
 → symbol-center averaging
 → residual CFO correction
 → family orientation correction
 → robust display normalization.

18.3 FSK/MSK path

prepared IQ → $f[n] = \frac{F_s}{2\pi} \arg(x[n]x^*[n-1])$
 → moving-average smoothing
 → transition peak detection
 → periodic transition-phase timing
 → per-symbol discriminator means
 → 2/4-level K-means
 → frequency-state constellation.

18.4 OFDM path

prepared IQ → cyclic-prefix normalized correlation
 → symbol timing and fractional CFO
 → CP removal and symbol FFT
 → carrier occupancy / integer shift
 → pilot common-phase and slope correction
 → per-carrier phase/RMS normalization
 → constellation points.

19 Worked Numerical Examples

19.1 2048-point FFT at 2 MS/s

For an example recording with

$$F_s = 2.0 \text{ MHz}, \quad N = 2048, \quad (181)$$

the FFT-bin spacing is

$$\Delta f = \frac{2,000,000}{2048} = 976.5625 \text{ Hz}. \quad (182)$$

A 100-bin frequency selector therefore represents approximately

$$B = 100(976.5625) = 97.65625 \text{ kHz.} \quad (183)$$

If its shifted center is 200 bins above the display center,

$$f_{\text{off}} = 200(976.5625) = 195.3125 \text{ kHz,} \quad (184)$$

and the normalized mixer angular frequency is

$$\omega_0 = 2\pi \frac{195312.5}{2000000} \approx 0.61359 \text{ rad/sample.} \quad (185)$$

19.2 Instantaneous frequency from phase increment

At $F_s = 2 \text{ MHz}$, a measured adjacent-sample phase increment of 0.1 rad corresponds to

$$f_{\text{inst}} = \frac{2,000,000}{2\pi}(0.1) \approx 31.83 \text{ kHz.} \quad (186)$$

A negative phase increment gives the corresponding negative frequency deviation.

19.3 OFDM CP carrier estimate

For the known waveform, $N = 128$. If the aggregate CP correlation phase is $\phi_{CP} = 0.64 \text{ rad}$, the correction is

$$\hat{\omega} = -\frac{0.64}{128} = -0.005 \text{ rad/sample.} \quad (187)$$

At $F_s = 2 \text{ MHz}$, this corresponds to

$$\hat{f} = \frac{\hat{\omega}F_s}{2\pi} \approx -1.592 \text{ kHz.} \quad (188)$$

20 Engineering Notes and Interpretation

- The live analysis FFT uses a Hann window. The crop frequency filter, by contrast, is a rectangular FFT-domain mask.
- The quantity labeled PSD in the UI is formed by averaging display dB values across spectrogram time columns, then min-max normalizing. It is an average log-magnitude spectrum rather than an absolute calibrated power spectral density in units such as W/Hz or dB/Hz.
- The phase and instantaneous-frequency graphs are intentionally meaningful only when the frequency selector is active. The selected band is mixed to baseband and low-pass filtered before phase calculations.
- Linear phase detrending removes residual carrier rotation from the plotted phase so the graph emphasizes modulation phase deviation.
- The active PSK/QAM/ASK timing routine is `ANALYSIS.constellation.find_symbol_timing_v2()`; several earlier estimator helpers remain defined but are not called by the current build path.
- PSK carrier recovery uses an M th-power method; QAM uses a fourth-order moment; ASK/OOK uses a spectral centroid. These are modulation-family-specific ways of reducing data-dependent phase ambiguity before estimating carrier offset.

- FSK/MSK recovery does not plot raw I/Q symbols. It first converts IQ phase increments into instantaneous frequency, then plots recovered frequency state versus a local transition coordinate.
- The specialized OFDM routine explicitly targets a 128-point, 32-sample-CP QPSK waveform with 44 data carriers and four fixed pilots. Other OFDM is handled by the generic CP-search path.
- FFTW forward and backward transforms are unnormalized. Every inverse-FFT path that returns time-domain amplitudes therefore divides by the transform length.

21 Compact Formula Reference

Operation	Implemented mathematics
Complex16 normalization	$x[n] = I_{16}[n]/32768 + jQ_{16}[n]/32768$
Spectrogram frequency	$f = f_c + (1/2 - y)F_s$
FFT-bin spacing	$\Delta f = F_s/N$
Hann window	$w[n] = \frac{1}{2} - \frac{1}{2} \cos(2\pi n/(N - 1))$
FFT magnitude	$A[k] = X[k] /N$
Display dB	$D[k] = 20 \log_{10}(A[k] + 10^{-12}) + 100$
Selected-band mixer	$z[n] = x[n]e^{-j2\pi f_0 n/F_s}$
One-pole low-pass	$y[n] = y[n - 1] + \alpha(z[n] - y[n - 1])$
LP coefficient	$\alpha = 2\pi f_c/(F_s + 2\pi f_c)$
Instantaneous frequency	$f[n] = F_s \arg(x[n]x^*[n - 1])/(2\pi)$
Phase unwrap	$\tilde{\phi}[n] = \tilde{\phi}[n - 1] + \text{wrap}(\phi[n] - \phi[n - 1])$
Phase detrend	$\phi_{dev}[x] = \tilde{\phi}[x] - (a + bx)$
Crop FFT mask	$H[k] = 1$ in selected bins, 0 elsewhere; selected center shifted to DC
DC removal	$x_c[n] = x[n] - N^{-1} \sum_m x[m]$
Mth-power CFO	$\hat{\omega} = M^{-1} \arg \sum u[n]u^*[n - 1]$
PSK FFT CFO	peak of $\text{FFT}\{w[n](x[n]/ x[n])^M\}$, divided by M
QAM FFT CFO	peak of $\text{FFT}\{w[n](x[n]/r_{RMS})^4\}$, divided by 4
Spectral center	$\hat{f} = \sum f_k w_k / \sum w_k$
PSK score	$C_M/(1 + 3v_r)$
QAM score	$1/(0.02 + E_{\min})$
ASK principal axis	$\theta = \frac{1}{2} \text{atan2}(2C_{IQ}, C_{II} - C_{QQ})$
Transition energy	$e[n] = x[n] - x[n - 1] ^2$
Timing concentration	$C_S = \max \tilde{H}_S / \text{mean } \tilde{H}_S$
PSK residual CFO	$\arg \sum u_m u_{m-1}^* / (MS)$
FSK discriminator	$f[n] = F_s \arg(x[n]x^*[n - 1])/(2\pi)$
FSK timing coherence	$C(S) = \sum w_j e^{j2\pi n_j/S} / \sum w_j$
OFDM CP correlation	$\rho = \sum x[n + k]x^*[n + N + k] / \sqrt{P_A P_B}$

Operation	Implemented mathematics
OFDM CP CFO	$\hat{\omega} = -\arg(C)/N$
OFDM pilot correction	$X'[k] = X[k]e^{-j(\phi_c+bk)}$

22 Core DSP Source Locations

The following source ranges contain the principal DSP implementations documented above. This section uses the same source-line numbering shown in every code block, so it can be used as a navigation map for the current AnalysisWorkstation.c.

Function / calculation	Source lines	Role
ANALYSIS_frequency_from_spec_frac()	2257–2268	display fraction to RF frequency
ANALYSIS_get_current_time_range()	8267–8336	time selector to samples
ANALYSIS_get_current_filter_bins()	8338–8401	frequency selector to bins
ANALYSIS_process_crop_frequency_and_noise()	8570–8805	FFT crop filter / recenter
ANALYSIS_wrap_phase()	9800–9814	phase wrapping
ANALYSIS_spectrogram_display_db()	9851–9919	center-stripe display interpolation
ANALYSIS_constellation_apply_frequency_correction()	10115–10159	complex CFO correction
ANALYSIS_constellation_complex_power()	10161–10189	z^M helper
ANALYSIS_constellation_estimate_mth_frequency()	10191–10240	Mth-power CFO helper
ANALYSIS_constellation_mth_coherence()	10242–10282	phase coherence helper
ANALYSIS_constellation_estimate_direct_frequency()	10284–10310	direct CFO helper
ANALYSIS_constellation_prepare_samples()	10312–10667	DC removal, mixing, low-pass
ANALYSIS_constellation_kmeans_1d()	10669–10770	normalized 1-D clustering
ANALYSIS_constellation_psk_score()	10772–10821	PSK family fit
ANALYSIS_constellation_qam_score()	10823–10887	QAM family fit
ANALYSIS_constellation_ask_score()	10889–10958	ASK/OOK family fit
ANALYSIS_constellation_collect_candidate()	10960–11037	symbol-center samples
ANALYSIS_constellation_transition_timing_score()	11039–11144	boundary transition score
ANALYSIS_constellation_find_symbol_timing_v2()	12208–12512	active symbol timing
ANALYSIS_constellation_psk_residual_frequency()	12514–12561	residual PSK CFO
ANALYSIS_constellation_qam_residual_frequency()	12563–12642	residual QAM CFO
ANALYSIS_constellation_build_linear_family()	12644–12859	PSK/QAM/ASK pipeline
ANALYSIS_constellation_fsk_kmeans_1d()	12861–13040	robust FSK state clustering
ANALYSIS_constellation_build_fsk_family()	13042–13719	FSK/MSK recovery
ANALYSIS_constellation_ofdm_cp_score()	13721–13755	normalized CP score
ANALYSIS_constellation_build_ofdm_family_generic()	13757–14007	generic OFDM recovery
ANALYSIS_constellation_ofdm_cp_metrics()	14009–14066	sliding CP metric
ANALYSIS_constellation_build_known_ofdm_qpsk()	14112–14812	known OFDM recovery
ANALYSIS_build_selected_constellation()	14829–14892	family dispatch / cache
ANALYSIS_render_workstation_data()	14894–15767	live spectrogram/graphs/PSD

23 Core DSP Calculation Listings

This appendix repeats compact core calculation portions from the active processing paths so the equations above can be compared directly with the implementation without including unrelated UI, allocation, and error-handling code.

23.1 Spectrogram FFT calculation

```
15185 double sum_mag = 0.0;
```

```

15187     for (int k = 0; k < ANALYSIS_FFT_SIZE; k++) {
15188         double I = (double)current_block[k * 2] / 32768.0;
15189         double Q = (double)current_block[k * 2 + 1] / 32768.0;
15191         sum_mag += sqrt(I * I + Q * Q);
15193         in[k][0] = I * window[k];
15194         in[k][1] = Q * window[k];
15195     }
15197     double avg_mag = sum_mag / (double)ANALYSIS_FFT_SIZE;

15321     fftw_execute(plan);
15323     for (int y = 0; y < ANALYSIS_FFT_SIZE; y++) {
15324         int shifted = (y + ANALYSIS_FFT_SIZE / 2) % ANALYSIS_FFT_SIZE;
15325         double I = out[shifted][0];
15326         double Q = out[shifted][1];
15327         double mag = sqrt(I * I + Q * Q) / (double)ANALYSIS_FFT_SIZE;
15328         double val = 20.0 * log10(mag + 1e-12) + 100.0;
15330         db_img[(size_t)x * ANALYSIS_FFT_SIZE + y] = val;
15332         if (filter_active && y >= filter_bin_low && y <= filter_bin_high) {
15334             double weight = mag;
15335             filter_mag[x] += weight;
15336             filter_re[x] += I;
15337             filter_im[x] += Q;
15339         }
15340     }

```

23.2 Filtered phase and instantaneous-frequency calculation

```

15201     double bin_center = ((double)filter_bin_low + (double)filter_bin_high) * 0.5;
15202     double center_offset_hz = (bin_center - ((double)ANALYSIS_FFT_SIZE * 0.5)) *
15203         (Global_Analysis_Sample_Rate / (double)ANALYSIS_FFT_SIZE);
15204     double bin_width_hz = Global_Analysis_Sample_Rate / (double)ANALYSIS_FFT_SIZE;
15205     double filter_width_hz = (double)(filter_bin_high - filter_bin_low + 1) * bin_width_hz;
15206     double cutoff_hz = filter_width_hz * 0.5;
15208     if (cutoff_hz < bin_width_hz) {
15210         cutoff_hz = bin_width_hz;
15212     }
15214     if (cutoff_hz > Global_Analysis_Sample_Rate * 0.45) {
15216         cutoff_hz = Global_Analysis_Sample_Rate * 0.45;
15218     }
15220     double alpha = (2.0 * M_PI * cutoff_hz) / (Global_Analysis_Sample_Rate + (2.0 * M_PI *
cutoff_hz));
15221     double omega = 2.0 * M_PI * center_offset_hz / Global_Analysis_Sample_Rate;
15222     double phase_sum_i = 0.0;
15223     double phase_sum_q = 0.0;
15224     double inst_freq_sum = 0.0;
15225     int inst_freq_count = 0;
15226     double lp_i = 0.0;
15227     double lp_q = 0.0;
15228     double prev_i = 0.0;
15229     double prev_q = 0.0;
15230     size_t samples_per_column =
15231         render_w > 0 ? Global_Analysis_View_Len / (size_t)render_w : (size_t)ANALYSIS_FFT_SIZE;
15232     int inst_sample_count = (int)samples_per_column;
15234     if (inst_sample_count < 8) {
15236         inst_sample_count = 8;
15238     }
15240     if (inst_sample_count > 512) {
15242         inst_sample_count = 512;
15244     }
15246     if (inst_sample_count > ANALYSIS_FFT_SIZE) {
15248         inst_sample_count = ANALYSIS_FFT_SIZE;
15250     }
15252     for (int k = 0; k < inst_sample_count; k++) {
15253         double I = (double)current_block[k * 2] / 32768.0;
15254         double Q = (double)current_block[k * 2 + 1] / 32768.0;
15255         double angle = -omega * (double)k;
15256         double c = cos(angle);

```

```

15257     double s = sin(angle);
15258     double mix_i = I * c - Q * s;
15259     double mix_q = I * s + Q * c;
15261     if (k == 0) {
15263         lp_i = mix_i;
15264         lp_q = mix_q;
15266     }
15268     else {
15270         lp_i += alpha * (mix_i - lp_i);
15271         lp_q += alpha * (mix_q - lp_q);
15273     }
15275     phase_sum_i += lp_i;
15276     phase_sum_q += lp_q;
15278     if (k > 0) {
15280         double prod_i = lp_i * prev_i + lp_q * prev_q;
15281         double prod_q = lp_q * prev_i - lp_i * prev_q;
15282         double dphase = ANALYSIS_wrap_phase(atan2(prod_q, prod_i));
15284         inst_freq_sum += dphase * Global_Analysis_Sample_Rate / (2.0 * M_PI);
15285         inst_freq_count++;
15287     }
15289     prev_i = lp_i;
15290     prev_q = lp_q;
15291 }
15293 filter_td_phase[x] = atan2(phase_sum_q, phase_sum_i);
15295 if (inst_freq_count > 0) {
15297     filter_td_inst_freq[x] = inst_freq_sum / (double)inst_freq_count;
15299 }

```

23.3 Phase unwrap and detrend calculation

```

15444     double unwrapped = filter_td_phase[seg_start];
15445     double prev_raw = filter_td_phase[seg_start];
15446     filter_td_phase[seg_start] = unwrapped;
15448     for (int x = seg_start + 1; x <= seg_end; x++) {
15449         double raw = filter_td_phase[x];
15450         unwrapped += ANALYSIS_wrap_phase(raw - prev_raw);
15451         filter_td_phase[x] = unwrapped;
15452         prev_raw = raw;
15453     }
15455     int n = seg_end - seg_start + 1;
15457     if (n >= 2) {
15459         double sum_x = 0.0;
15460         double sum_y = 0.0;
15461         double sum_xx = 0.0;
15462         double sum_xy = 0.0;
15464         for (int x = seg_start; x <= seg_end; x++) {
15465             double dx = (double)(x - seg_start);
15466             double y = filter_td_phase[x];
15468             sum_x += dx;
15469             sum_y += y;
15470             sum_xx += dx * dx;
15471             sum_xy += dx * y;
15472         }
15474         double denom = ((double)n * sum_xx) - (sum_x * sum_x);
15475         double slope = 0.0;
15476         double intercept = sum_y / (double)n;
15478         if (fabs(denom) > 1e-12) {
15480             slope = (((double)n * sum_xy) - (sum_x * sum_y)) / denom;
15481             intercept = (sum_y - slope * sum_x) / (double)n;
15483         }
15485         for (int x = seg_start; x <= seg_end; x++) {
15486             double dx = (double)(x - seg_start);
15487             filter_td_phase[x] -= intercept + slope * dx;
15488         }
15490     }
15492     else {
15494         filter_td_phase[seg_start] = 0.0;
15496     }

```

```

15498     seg_start = seg_end + 1;
15499 }
15501 double inst_freq_mean = 0.0;
15502 int inst_freq_mean_count = 0;
15504 for (int x = 0; x < render_w; x++) {
15506     if (valid_phase[x]) {
15508         inst_freq_mean += filter_td_inst_freq[x];
15509         inst_freq_mean_count++;
15511     }
15512 }
15514 if (inst_freq_mean_count > 0) {
15516     inst_freq_mean /= (double)inst_freq_mean_count;
15518 }
15520 for (int x = 0; x < render_w; x++) {
15521     double phase = 0.0;
15522     double inst_freq_hz = 0.0;
15524     if (valid_phase[x]) {
15526         phase = filter_td_phase[x];
15527         inst_freq_hz = filter_td_inst_freq[x] - inst_freq_mean;
15529         if (fabs(phase) > max_phase_abs) {
15531             max_phase_abs = fabs(phase);
15533         }
15535         if (fabs(inst_freq_hz) > max_inst_freq_abs) {
15537             max_inst_freq_abs = fabs(inst_freq_hz);
15539         }
15541     }
15543     Global_Analysis_Phase_Line[x] = (float)phase;
15544     Global_Analysis_InstFreq_Line[x] = (float)inst_freq_hz;
15545 }

```

23.4 Frequency-domain crop calculation

```

8696     for (int i = 0; i < ANALYSIS_FFT_SIZE; i++) {
8698         if ((size_t)i < got_iq) {
8700             freq_in[i][0] = (double)iq[i * 2] / 32768.0;
8701             freq_in[i][1] = (double)iq[i * 2 + 1] / 32768.0;
8703         }
8705         else {
8707             freq_in[i][0] = 0.0;
8708             freq_in[i][1] = 0.0;
8710         }
8711     }
8713     fftw_execute(forward);
8715     for (int i = 0; i < ANALYSIS_FFT_SIZE; i++) {
8716         freq_in[i][0] = 0.0;
8717         freq_in[i][1] = 0.0;
8718     }
8720     int filter_center_bin = (filter_bin_low + filter_bin_high) / 2;
8721     for (int y = filter_bin_low; y <= filter_bin_high; y++) {
8722         int src_shifted = (y + ANALYSIS_FFT_SIZE / 2) % ANALYSIS_FFT_SIZE;
8723         int centered_y = y - filter_center_bin + (ANALYSIS_FFT_SIZE / 2);
8725         if (centered_y < 0 || centered_y >= ANALYSIS_FFT_SIZE) {
8727             continue;
8729         }
8731         int dst_shifted = (centered_y + ANALYSIS_FFT_SIZE / 2) % ANALYSIS_FFT_SIZE;
8733         freq_in[dst_shifted][0] = freq_out[src_shifted][0];
8734         freq_in[dst_shifted][1] = freq_out[src_shifted][1];
8735     }
8737     fftw_execute(inverse);
8739     for (size_t i = 0; i < got_iq; i++) {
8740         double re = time_out[i][0] / (double)ANALYSIS_FFT_SIZE;
8741         double im = time_out[i][1] / (double)ANALYSIS_FFT_SIZE;
8743         if (ANALYSIS_noise_sample_is_muted(absolute_sample + i)) {
8745             re = 0.0;
8746             im = 0.0;
8748         }
8750         if (re > 0.999969) {
8752             re = 0.999969;

```

```

8754     }
8755     if (re < -1.0) {
8756         re = -1.0;
8757     }
8758     if (im > 0.999969) {
8759         im = 0.999969;
8760     }
8761     if (im < -1.0) {
8762         im = -1.0;
8763     }
8764     iq[i * 2] = (int16_t)rint(re * 32767.0);
8765     iq[i * 2 + 1] = (int16_t)rint(im * 32767.0);

```

23.5 Constellation sample preparation calculation

```

10494 double dc_i = 0.0;
10495 double dc_q = 0.0;
10496 for (size_t n = 0; n < sample_count; n++) {
10497     dc_i += (double)raw[n * 2U] / 32768.0;
10498     dc_q += (double)raw[n * 2U + 1U] / 32768.0;
10499 }
10500 dc_i /= (double)sample_count;
10501 dc_q /= (double)sample_count;
10502 double center_offset_hz = 0.0;
10503 double bandwidth_hz = Global_Analysis_Sample_Rate * 0.90;
10504 if (filter_active) {
10505     double bin_center = ((double)filter_bin_low + (double)filter_bin_high) * 0.5;
10506     double bin_width_hz = Global_Analysis_Sample_Rate / (double)ANALYSIS_FFT_SIZE;
10507     center_offset_hz = (bin_center - ((double)ANALYSIS_FFT_SIZE * 0.5)) * bin_width_hz;
10508     bandwidth_hz = (double)(filter_bin_high - filter_bin_low + 1) * bin_width_hz;
10509 }
10510 if (output_bandwidth_hz) {
10511     *output_bandwidth_hz = bandwidth_hz;
10512 }
10513 double mixer_step = -2.0 * M_PI * center_offset_hz / Global_Analysis_Sample_Rate;
10514 double step_i = cos(mixer_step);
10515 double step_q = sin(mixer_step);
10516 double oscillator_i = 1.0;
10517 double oscillator_q = 0.0;
10518 for (size_t n = 0; n < sample_count; n++) {
10519     double input_i = ((double)raw[n * 2U] / 32768.0) - dc_i;
10520     double input_q = ((double)raw[n * 2U + 1U] / 32768.0) - dc_q;
10521     i_data[n] = input_i * oscillator_i - input_q * oscillator_q;
10522     q_data[n] = input_i * oscillator_q + input_q * oscillator_i;
10523     double next_i = oscillator_i * step_i - oscillator_q * step_q;
10524     double next_q = oscillator_q * step_i + oscillator_i * step_q;
10525     oscillator_i = next_i;
10526     oscillator_q = next_q;
10527 }

```

```

10605 fftw_execute(forward);
10606 double cutoff_hz = bandwidth_hz * 0.55;
10607 if (cutoff_hz > Global_Analysis_Sample_Rate * 0.47) {
10608     cutoff_hz = Global_Analysis_Sample_Rate * 0.47;
10609 }
10610 if (cutoff_hz < Global_Analysis_Sample_Rate / (double)fft_size) {
10611     cutoff_hz = Global_Analysis_Sample_Rate / (double)fft_size;
10612 }
10613 for (size_t k = 0; k < fft_size; k++) {
10614     double frequency =
10615         k <= fft_size / 2U
10616         ? ((double)k * Global_Analysis_Sample_Rate / (double)fft_size)
10617         : ((double)((long long)k - (long long)fft_size) * Global_Analysis_Sample_Rate /
10618           (double)fft_size);
10619     if (fabs(frequency) > cutoff_hz) {
10620         frequency_data[k][0] = 0.0;
10621         frequency_data[k][1] = 0.0;

```



```

10632     }
10633 }
10635 fftw_execute(inverse);
10637 for (size_t n = 0; n < sample_count; n++) {
10638     i_data[n] = time_data[n][0] / (double)fft_size;
10639     q_data[n] = time_data[n][1] / (double)fft_size;
10640 }

```

23.6 PSK/QAM/ASK active family dispatch

```

12674 double maximum_offset_hz = bandwidth_hz * 0.75;
12676 if (maximum_offset_hz < Global_Analysis_Sample_Rate / 4096.0) {
12678     maximum_offset_hz = Global_Analysis_Sample_Rate / 4096.0;
12680 }
12682 double frequency = 0.0;
12684 if (family == ANALYSIS_CONSTELLATION_MODE_PSK) {
12686     frequency = ANALYSIS_constellation_estimate_psk_frequency_fft(i_data, q_data, count, psk_order, gate,
12687                                                                    maximum_offset_hz);
12689 }
12691 else if (family == ANALYSIS_CONSTELLATION_MODE_QAM) {
12693     frequency = ANALYSIS_constellation_estimate_qam_frequency_fft(i_data, q_data, count, gate,
12694                                                                    maximum_offset_hz);
12696 }
12698 else {
12699     frequency = ANALYSIS_constellation_estimate_spectral_center(i_data, q_data, count, maximum_offset_hz);
12701 }
12703 ANALYSIS_constellation_apply_frequency_correction(i_data, q_data, count, frequency);
12705 int samples_per_symbol = 0;
12706 int offset = 0;
12708 if (!ANALYSIS_constellation_find_symbol_timing_v2(i_data, q_data, count, bandwidth_hz, family, psk_order,
12709 gate,
12710                                                                    &samples_per_symbol, &offset)) {
12711     return;
12713 }

```

```

12739 double residual_frequency = 0.0;
12741 if (family == ANALYSIS_CONSTELLATION_MODE_PSK) {
12743     residual_frequency =
12744         ANALYSIS_constellation_psk_residual_frequency(output, output_count, psk_order,
12745 samples_per_symbol);
12746 }
12748 else if (family == ANALYSIS_CONSTELLATION_MODE_QAM) {
12750     double symbol_rate_hz = Global_Analysis_Sample_Rate / (double)samples_per_symbol;
12751     residual_frequency =
12752         ANALYSIS_constellation_qam_residual_frequency(output, output_count, samples_per_symbol,
12753 symbol_rate_hz);
12754 }
12756 else {
12758     residual_frequency = ANALYSIS_constellation_psk_residual_frequency(output, output_count, 1,
12759 samples_per_symbol);
12760 }
12762 if (fabs(residual_frequency) > 1e-15) {
12764     ANALYSIS_constellation_apply_frequency_correction(i_data, q_data, count, residual_frequency);
12765     output_count = ANALYSIS_constellation_collect_candidate(i_data, q_data, count, guard,
12766 samples_per_symbol,
12767                                                                    offset, output, ANALYSIS_MAX_CONST_POINTS);
12768 }
12770 if (output_count < 4) {
12772     return;
12774 }
12776 double rotation_phase = 0.0;
12778 if (family == ANALYSIS_CONSTELLATION_MODE_PSK) {
12780     double sum_i = 0.0;
12781     double sum_q = 0.0;
12783     for (int p = 0; p < output_count; p++) {
12784         double radius = hypot(output[p].i, output[p].q);
12786         if (radius > gate) {

```

```

12788         double powered_i = 0.0;
12789         double powered_q = 0.0;
12790         ANALYSIS_constellation_complex_power(output[p].i / radius, output[p].q / radius, psk_order,
        &powered_i,
12791                                         &powered_q);
12792         sum_i += powered_i;
12793         sum_q += powered_q;
12794     }
12795 }
12796 }
12797 rotation_phase = atan2(sum_q, sum_i) / (double)psk_order;
12798 }
12800 else if (family == ANALYSIS_CONSTELLATION_MODE_QAM) {
12801     double sum_i = 0.0;
12802     double sum_q = 0.0;
12803     for (int p = 0; p < output_count; p++) {
12804         double powered_i = 0.0;
12805         double powered_q = 0.0;
12806         ANALYSIS_constellation_complex_power(output[p].i, output[p].q, 4, &powered_i, &powered_q);
12807         sum_i += powered_i;
12808         sum_q += powered_q;
12809     }
12810     rotation_phase = ANALYSIS_wrap_phase(atan2(sum_q, sum_i) - M_PI) * 0.25;
12811 }
12812 else {
12813     double mean_i = 0.0;
12814     double mean_q = 0.0;
12815     for (int p = 0; p < output_count; p++) {
12816         mean_i += output[p].i;
12817         mean_q += output[p].q;
12818     }
12819     mean_i /= (double)output_count;
12820     mean_q /= (double)output_count;
12821     double covariance_ii = 0.0;
12822     double covariance_qq = 0.0;
12823     double covariance_iq = 0.0;
12824     for (int p = 0; p < output_count; p++) {
12825         double centered_i = output[p].i - mean_i;
12826         double centered_q = output[p].q - mean_q;
12827         covariance_ii += centered_i * centered_i;
12828         covariance_qq += centered_q * centered_q;
12829         covariance_iq += centered_i * centered_q;
12830     }
12831     rotation_phase = 0.5 * atan2(2.0 * covariance_iq, covariance_ii - covariance_qq);
12832 }
12833 double rotation_i = cos(-rotation_phase);
12834 double rotation_q = sin(-rotation_phase);
12835 Global_Analysis_Const_Count = output_count;
12836 for (int p = 0; p < output_count; p++) {
12837     Global_Analysis_Const_I[p] = (float)(output[p].i * rotation_i - output[p].q * rotation_q);
12838     Global_Analysis_Const_Q[p] = (float)(output[p].i * rotation_q + output[p].q * rotation_i);
12839 }
12840 ANALYSIS_constellation_normalize_output(family == ANALYSIS_CONSTELLATION_MODE_ASK_OOK);

```

23.7 FSK discriminator and timing calculations

```

13072 double average_power = 0.0;
13073 for (size_t n = 0U; n < count; n++) {
13074     average_power += i_data[n] * i_data[n] + q_data[n] * q_data[n];
13075 }
13076 average_power /= (double)count;
13077 double magnitude_gate = sqrt(average_power) * 0.18;
13078 size_t valid_count = 0U;
13079 for (size_t n = 1U; n < count; n++) {
13080     double current_magnitude = hypot(i_data[n], q_data[n]);
13081     double previous_magnitude = hypot(i_data[n - 1U], q_data[n - 1U]);
13082     size_t index = n - 1U;
13083     if (current_magnitude < magnitude_gate || previous_magnitude < magnitude_gate) {
13084         frequency_hz[index] = 0.0;

```

```

13090     continue;
13092 }
13094 double product_i = i_data[n] * i_data[n - 1U] + q_data[n] * q_data[n - 1U];
13095 double product_q = q_data[n] * i_data[n - 1U] - i_data[n] * q_data[n - 1U];
13096 double phase_increment = atan2(product_q, product_i);
13098 frequency_hz[index] = phase_increment * sample_rate / (2.0 * M_PI);
13099 valid[index] = 1U;
13100 valid_count++;

```

```

13300     if (minimum_sps < 4) {
13302         minimum_sps = 4;
13304     }
13306     if (maximum_sps > 2048) {
13308         maximum_sps = 2048;
13310     }
13312     int count_limited_maximum = (int)(frequency_count / 8U);
13314     if (maximum_sps > count_limited_maximum) {
13316         maximum_sps = count_limited_maximum;
13318     }
13320     double best_score = -1.0;
13322     for (int candidate_sps = minimum_sps; candidate_sps <= maximum_sps; candidate_sps++) {
13323         double phase_i = 0.0;
13324         double phase_q = 0.0;
13325         double weight_sum = 0.0;
13327         for (size_t transition = 0U; transition < transition_count; transition++) {
13328             double weight = transition_weights[transition];
13329             double phase = 2.0 * M_PI * (double)transition_samples[transition] / (double)candidate_sps;
13330             phase_i += weight * cos(phase);
13331             phase_q += weight * sin(phase);
13332             weight_sum += weight;
13333         }
13335         if (weight_sum <= 0.0) {
13337             continue;
13339         }
13341         double coherence = hypot(phase_i, phase_q) / weight_sum;
13342         double score = coherence + 0.01 * log((double)candidate_sps);
13344         if (score > best_score) {
13346             best_score = score;
13347             samples_per_symbol = candidate_sps;
13349         }
13350     }
13352     if (samples_per_symbol >= 4) {
13354         double phase_i = 0.0;
13355         double phase_q = 0.0;
13357         for (size_t transition = 0U; transition < transition_count; transition++) {
13358             double weight = transition_weights[transition];
13359             double phase = 2.0 * M_PI * (double)transition_samples[transition] /
13360 (double)samples_per_symbol;
13361             phase_i += weight * cos(phase);
13362             phase_q += weight * sin(phase);
13364         }
13366         double boundary_phase = atan2(phase_q, phase_i);
13368         if (boundary_phase < 0.0) {
13370             boundary_phase += 2.0 * M_PI;
13372         }
13374         boundary_offset = boundary_phase * (double)samples_per_symbol / (2.0 * M_PI);

```

23.8 OFDM cyclic-prefix calculation

```

13728     int symbol_size = fft_size + cp_size;
13729     double sum_i = 0.0;
13730     double sum_q = 0.0;
13731     double power_a = 0.0;
13732     double power_b = 0.0;
13733     int blocks = 0;
13735     for (size_t start = (size_t)offset; start + (size_t)symbol_size <= count && blocks < 48;

```

```

13736     start += (size_t)symbol_size) {
13737     for (int k = 0; k < cp_size; k++) {
13738         size_t a = start + (size_t)k;
13739         size_t b = start + (size_t)fft_size + (size_t)k;
13740         sum_i += i_data[a] * i_data[b] + q_data[a] * q_data[b];
13741         sum_q += q_data[a] * i_data[b] - i_data[a] * q_data[b];
13742         power_a += i_data[a] * i_data[a] + q_data[a] * q_data[a];
13743         power_b += i_data[b] * i_data[b] + q_data[b] * q_data[b];
13744     }
13745     blocks++;
13746 }
13748 if (phase_out) {
13750     *phase_out = atan2(sum_q, sum_i);
13752 }
13754 return power_a > 1e-12 && power_b > 1e-12 ? hypot(sum_i, sum_q) / sqrt(power_a * power_b) : 0.0;
13755 }

```

23.9 Known OFDM pilot correction calculation

```

14615     error_i[pilot] = point_i * reference_i[pilot] + point_q * reference_q[pilot];
14616     error_q[pilot] = point_q * reference_i[pilot] - point_i * reference_q[pilot];
14617     valid_pilots++;
14618 }
14620 if (valid_pilots < 3) {
14622     continue;
14624 }
14626 double best_slope = 0.0;
14627 double best_coherence = -1.0;
14628 double best_sum_i = 0.0;
14629 double best_sum_q = 0.0;
14631 for (int slope_step = -100; slope_step <= 100; slope_step++) {
14632     double slope = (double)slope_step * 0.0025;
14633     double sum_i = 0.0;
14634     double sum_q = 0.0;
14636     for (int pilot = 0; pilot < 4; pilot++) {
14638         if (hypot(error_i[pilot], error_q[pilot]) <= 1e-12) {
14640             continue;
14642         }
14644         double angle = -slope * (double)pilot_carriers[pilot];
14645         double rotation_i = cos(angle);
14646         double rotation_q = sin(angle);
14647         sum_i += error_i[pilot] * rotation_i - error_q[pilot] * rotation_q;
14648         sum_q += error_i[pilot] * rotation_q + error_q[pilot] * rotation_i;
14649     }
14651     double coherence = hypot(sum_i, sum_q);
14653     if (coherence > best_coherence) {
14655         best_coherence = coherence;
14656         best_slope = slope;
14657         best_sum_i = sum_i;
14658         best_sum_q = sum_q;
14660     }
14661 }
14663 if (best_coherence < 0.40 * (double)valid_pilots) {
14665     continue;
14667 }
14669 double common_phase = atan2(best_sum_q, best_sum_i);
14671 for (int bin = 0; bin < fft_size; bin++) {
14672     int carrier = bin - best_shift;
14674     while (carrier > fft_size / 2) {
14675         carrier -= fft_size;
14676     }
14678     while (carrier < -fft_size / 2) {
14679         carrier += fft_size;
14680     }
14682     double phase = common_phase + best_slope * (double)carrier;
14683     double rotation_i = cos(-phase);
14684     double rotation_q = sin(-phase);

```

```
14685     Type_Analysis_Constellation_Point *point = &symbols[(size_t)symbol * (size_t)fft_size +  
14686     (size_t)bin];  
14687     double corrected_i = point->i * rotation_i - point->q * rotation_q;  
14688     double corrected_q = point->i * rotation_q + point->q * rotation_i;  
14689     point->i = corrected_i;  
14690     point->q = corrected_q;  
    }
```